

حقوق نشر و مالکیت فکری

COPYRIGHT & INTELLECTUAL PROPERTY NOTICE

عنوان اثر

جزوه کامل درس بازیابی اطلاعات

(Information Retrieval – University Textbook)

پدیدآورندگان

حمیدرضا نامجومنش – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

امیرحسین همتی – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

علی مجاهد – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

© حق نشر و پروانه بهره‌برداری

این اثر تحت پروانه Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) منتشر می‌شود.

✓ شما مجاز هستید:

- این اثر را به صورت غیرتجاری و بدون هیچ‌گونه تغییر، با ذکر کامل نام پدیدآورندگان باز نشر دهید.
- نسخه‌های دقیقاً مشابه را در محیط‌های آموزشی غیرانتفاعی به اشتراک بگذارید.

✗ اکیداً ممنوع است:

- هرگونه استفاده تجاری (فروش، چاپ به قصد سود، قرار دادن پشت دیوار پرداخت، و نظایر آن).
- تغییر، تلخیص، ترجمه، بازترکیب فصول یا تولید آثار مشتق شده بدون رضایت کتبی همه پدیدآورندگان.
- حذف یا مخدوش کردن اعلان‌های حق نشر و واترمارک‌های دیجیتال.

⚠ هرگونه نقض این شرایط، نقض صریح حقوق مؤلف در چارچوب قوانین ایران و کنوانسیون برن (Berne Convention) محسوب می‌شود و پیگرد قانونی به دنبال خواهد داشت.

📄 متن کامل حقوقی مجوز (انگلیسی) | خلاصه فارسی مجوز

مخزن رسمی و شناسه دیجیتال

GitHub Repository: github.com/hrnrxb/IR-Textbook

(کلیه فایل‌های اصلی، امضاها و دیجیتال و گواهی‌های زمانی در مخزن فوق نگهداری می‌شوند.)

اصالت‌سنجی و گواهی مالکیت

• تمامی نسخه‌های PDF این جزوه با واترمارک دیجیتال و فراداده کپی‌رایت محافظت شده‌اند.

• هش SHA-256 یکتای فایل‌ها در سند `HASHES.txt` مخزن ثبت شده است.

🔒 تنها نسخه‌ای که از کانال رسمی (گیت‌هاب فوق) دریافت شود، معتبر است.

بازیابی اطلاعات

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل سیزدهم: Text classification and Naive Bayes

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو، امیرحسین همتی و علی مجاهد

فهرست مطالب

13.1 The text classification problem

13.2 Naive Bayes text classification

13.2.1 Relation to multinomial unigram language model

13.3 The Bernoulli model

13.4 Properties of Naive Bayes

13.4.1 A variant of the multinomial model

13.5 Feature selection

13.5.1 Mutual information

13.5.2 χ^2 Feature selection

13.5.3 Frequency-based feature selection

13.5.4 Feature selection for multiple classifiers

13.5.5 Comparison of feature selection methods

13.6 Evaluation of text classification

13.7 Chapter 13 Summary

13 طبقه‌بندی متن و بیز ساده: از جستجوی گوگل تا فیلتر اسپم

از جستجوی یک‌باره تا پیگیری هوشمند

تاکنون، این جزوه عمدتاً فرآیند بازیابی ادهاک (ad hoc retrieval) را بررسی کرده است - جایی که کاربران نیازهای اطلاعاتی موقتی دارند که با ارسال پرسمان به موتور جستجو تلاش می‌کنند به آن‌ها پاسخ دهند. اما بسیاری از کاربران نیازهای اطلاعاتی مداوم دارند.

تصور کنید بخواهید تحولات در حوزه تراشه‌های کامپیوتری چند هسته‌ای را دنبال کنید. یک راه این است که هر روز صبح پرسمان multicore AND computer AND chip را در شاخص مقالات خبری جدید جستجو کنید. اما این کار بسیار زمان‌بر است! گوگل نیوز چگونه این کار را برای شما خودکار می‌کند؟

در این فصل و دو فصل بعدی، به این سؤال می‌پردازیم: چگونه می‌توان این کار تکراری را خودکارسازی کرد؟ بسیاری از سیستم‌ها از پرسمان‌های ثابت (standing queries) پشتیبانی می‌کنند - پرسمان‌هایی که به صورت دوره‌ای روی مجموعه‌ای اجرا می‌شوند که اسناد جدید به تدریج به آن اضافه می‌شوند.

چرا ایمیل‌های اسپم به پوشه اسپم می‌روند؟

یک چالش مهم با پرسمان‌های ثابت این است که اگر پرسمان شما فقط multicore AND computer AND chip باشد، بسیاری از مقالات مرتبط را که از اصطلاحات دیگری مانند multicore processors استفاده می‌کنند، از دست خواهید داد. برای دستیابی به بازخوانی (recall) خوب، پرسمان‌های ثابت باید در طول زمان اصلاح شوند و می‌توانند به تدریج بسیار پیچیده شوند:

(multicore OR multi-core) AND (chip OR processor OR microprocessor)

برای درک گستردگی این مسئله، اکنون مفهوم کلی طبقه‌بندی (classification) را معرفی می‌کنیم. با توجه به مجموعه‌ای از کلاس‌ها، ما به دنبال تعیین این هستیم که یک شیء معین به کدام کلاس(ها) تعلق دارد. در مثال بالا، پرسمان ثابت برای تقسیم مقالات خبری جدید به دو کلاس عمل می‌کند: اسناد مربوط به تراشه‌های کامپیوتری چند هسته‌ای و اسناد غیرمرتبط با تراشه‌های کامپیوتری چند هسته‌ای. ما به این طبقه‌بندی دو کلاسی می‌گوییم. طبقه‌بندی با استفاده از پرسمان‌های ثابت همچنین مسیریابی (routing) یا فیلتر کردن (filtering) نامیده می‌شود.

یک کلاس لزوماً نباید به اندازه پرسمان ثابت multicore computer chips محدود باشد. اغلب، یک کلاس یک حوزه موضوعی عمومی‌تر مانند چین یا قهوه است. چنین کلاس‌های عمومی‌تری معمولاً موضوعات (topics) نامیده می‌شوند و وظیفه طبقه‌بندی متن سپس طبقه‌بندی متن (text classification)، دسته‌بندی متن (text categorization)، طبقه‌بندی موضوع (topic classification) یا شناسایی موضوع (topic spotting) نامیده می‌شود.

کاربردهای واقعی طبقه‌بندی متن در دنیای دیجیتال

مفهوم طبقه‌بندی بسیار کلی است و کاربردهای زیادی در داخل و خارج از بازیابی اطلاعات (IR) دارد. ما در اینجا بر مثال‌هایی از بازیابی اطلاعات تمرکز می‌کنیم:

کاربرد	توضیح
پیش‌پردازش برای نمایه‌سازی	گوگل از طبقه‌بندی برای تشخیص زبان صفحات وب استفاده می‌کند تا نتایج جستجو را به زبان مناسب نمایش دهد

تشخیص هرزنامه	جیمیل از طبقه‌بندهای پیچیده برای فیلتر کردن میلیاردها ایمیل اسپم روزانه استفاده می‌کند
تشخیص محتوای صریح جنسی	یوتیوب از طبقه‌بندی برای شناسایی محتوای نامناسب استفاده می‌کند تا با فعال بودن SafeSearch نمایش داده نشود
تشخیص احساسات	آمازون از تحلیل احساسات برای خلاصه کردن نظرات کاربران روی محصولات استفاده می‌کند
مرتب‌سازی ایمیل شخصی	اوتلوک از طبقه‌بندی برای مرتب‌سازی خودکار ایمیل‌ها به پوشه‌های اولویت، تبلیغات و اجتماعی استفاده می‌کند
جستجوی عمودی	وبسایت‌هایی مانند Zillow (املاک) یا Kayak (سفر) از طبقه‌بندی برای محدود کردن جستجو به حوزه تخصصی خود استفاده می‌کنند

این فهرست اهمیت کلی طبقه‌بندی در IR را نشان می‌دهد. اکثر سیستم‌های بازیابی امروز حاوی چندین جزء هستند که به شکل طبقه‌بندی استفاده می‌کنند.

📁 سه رویکرد برای طبقه‌بندی متن

برای پیاده‌سازی طبقه‌بندی متن، سه رویکرد اصلی وجود دارد:

- طبقه‌بندی دستی:** کتاب‌ها در یک کتابخانه توسط کتابدار به دسته‌بندی‌های کتابخانه کنگره اختصاص داده می‌شوند. این روش برای مقیاس‌بندی گران است.
- قوانین دستی:** استفاده از پرسمان‌های ثابت که می‌توان به عنوان **قوانین** در نظر گرفت - که معمولاً به صورت دستی نوشته می‌شوند. مانند مثال ما (multicore OR multi-core) AND (chip OR processor OR microprocessor)، قوانین گاهی معادل عبارات بولی هستند. یک قانون ترکیبی خاصی از کلمات کلیدی را که نشان‌دهنده یک کلاس است، ثبت می‌کند. قوانین کدگذاری شده دستی خواص مقیاس‌بندی خوبی دارند، اما ایجاد و نگهداری آن‌ها در طول زمان کار فشرده‌ای است.
- یادگیری ماشین:** در این رویکرد، مجموعه‌ای از قوانین یا به طور کلی، معیار تصمیم‌گیری طبقه‌بند متن، به طور خودکار از داده‌های آموزشی یاد گرفته می‌شود. این رویکرد همچنین **طبقه‌بندی آماری متن** نامیده می‌شود اگر روش یادگیری آماری باشد.

در طبقه‌بندی آماری متن، ما به تعدادی اسناد مثال خوب (یا اسناد آموزشی) برای هر کلاس نیاز داریم. نیاز به طبقه‌بندی دستی حذف نمی‌شود زیرا اسناد آموزشی از شخصی که آن‌ها را برچسب‌گذاری کرده است، می‌آیند - جایی که برچسب‌گذاری به فرآیند حاشیه‌نویسی هر سند با کلاس آن اشاره دارد. اما برچسب‌گذاری به طور قابل استدلالی وظیفه آسان‌تری از نوشتن قوانین است. تقریباً هر کسی می‌تواند به یک سند نگاه کند و تصمیم بگیرد که آیا به چین مرتبط است یا نه.

در این فصل، ما با یک مقدمه کلی برای مسئله طبقه‌بندی متن آغاز می‌کنیم؛ سپس بیز ساده (Naive Bayes)، یک روش طبقه‌بندی به خصوص ساده و مؤثر را پوشش می‌دهیم. تمام الگوریتم‌های طبقه‌بندی که ما مطالعه می‌کنیم، اسناد را در فضاهای با ابعاد بالا نمایش می‌دهند. برای بهبود کارایی این الگوریتم‌ها، معمولاً مطلوب است که ابعاد این فضاها را کاهش دهیم؛ به این منظور، تکنیکی به نام **انتخاب ویژگی** (feature selection) معمولاً در طبقه‌بندی متن اعمال می‌شود.

در فصل‌های بعدی، به دو خانواده دیگر از روش‌های طبقه‌بندی، طبقه‌بندهای فضای برداری و ماشین‌های بردار پشتیبان نگاه خواهیم کرد.

13.1 مسئله طبقه‌بندی متن: از ایمیل‌های اسپم تا دسته‌بندی اخبار 📰

چگونه جیمیل تشخیص می‌دهد ایمیل اسپم است؟

در طبقه بندی متن، ما یک توصیف $d \in X$ از یک سند داریم، که در آن X فضای سند (document space) است؛ و مجموعه‌ای ثابت از کلاس‌ها $C = \{c_1, c_2, \dots, c_L\}$. کلاس‌ها همچنین به عنوان دسته‌ها (categories) یا برچسب‌ها (labels) شناخته می‌شوند.

به طور معمول، فضای سند X نوعی فضای با ابعاد بالا است، و کلاس‌ها توسط انسان برای نیازهای یک کاربرد تعریف می‌شوند. برای مثال، در سیستم جیمیل، هر ایمیل یک سند است و دو کلاس اصلی وجود دارد: "اسپم" و "غیراسپم". جیمیل باید بتواند هر ایمیل جدید را به یکی از این دو کلاس اختصاص دهد.

ما یک مجموعه آموزشی (training set) D از اسناد برچسب‌دار $\langle d, c \rangle$ داریم که در آن $\langle d, c \rangle \in X \times C$. برای مثال:

$\langle \text{Beijing joins the World Trade Organization, China} \rangle$

برای سند تک جمله‌ای "Beijing joins the World Trade Organization" و کلاس (یا برچسب) China.

یادگیری نظارت شده: چگونه نتفلیکس سلیقه شما را یاد می‌گیرد؟

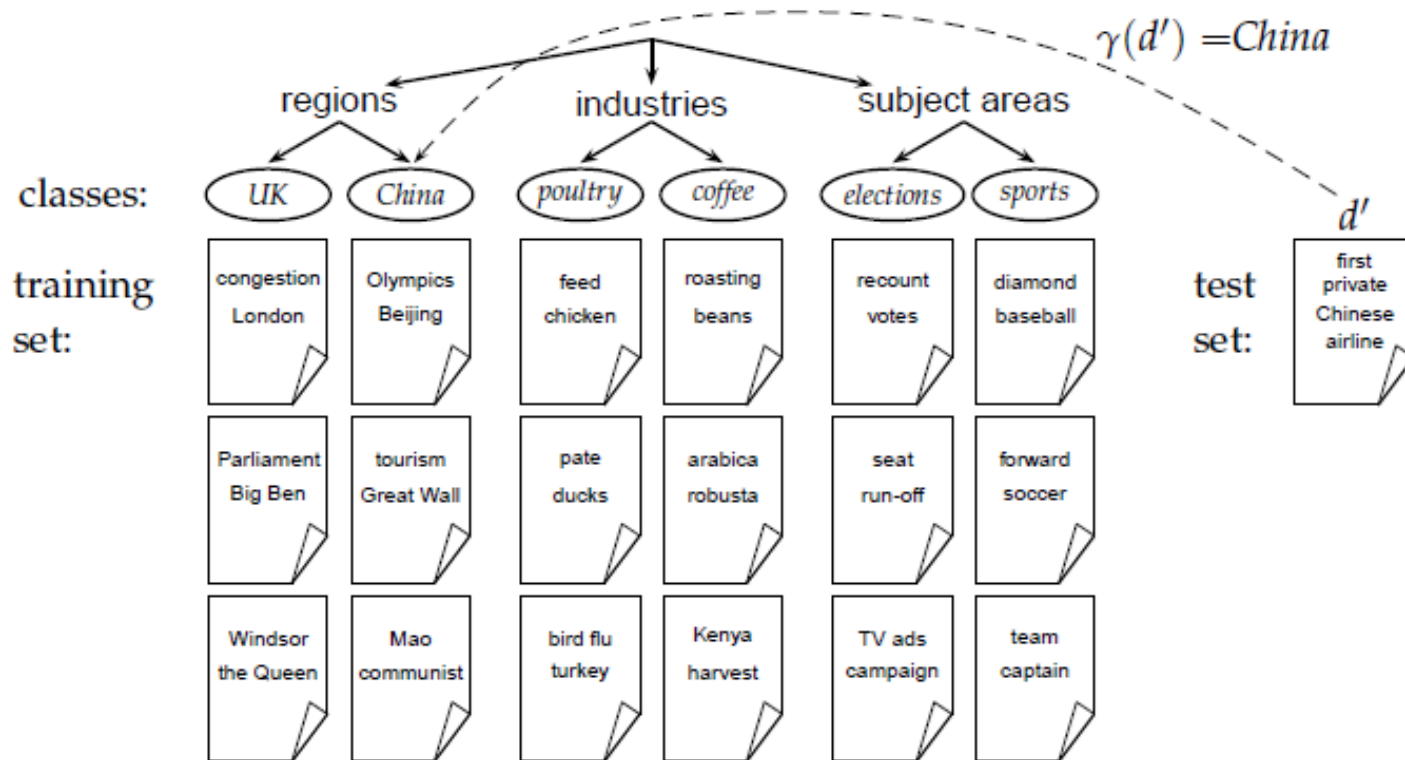
با استفاده از یک روش یادگیری (learning method) یا الگوریتم یادگیری (learning algorithm)، ما سپس می‌خواهیم یک طبقه بند (classifier) یا تابع طبقه بندی (classification function) γ یاد بگیریم که اسناد را به کلاس‌ها نگاشت می‌کند:

$$\gamma : X \rightarrow C$$

این نوع یادگیری یادگیری نظارت شده (supervised learning) نامیده می‌شود زیرا یک ناظر (انسانی که کلاس‌ها را تعریف می‌کند و اسناد آموزشی را برچسب‌گذاری می‌کند) به عنوان یک معلم فرآیند یادگیری را هدایت می‌کند.

نتفلیکس از همین روش استفاده می‌کند: وقتی شما به فیلمی امتیاز می‌دهید، در واقع در حال برچسب‌گذاری داده‌های آموزشی هستید. الگوریتم نتفلیکس (روش یادگیری Γ) از این داده‌ها برای یادگیری تابع طبقه بندی γ استفاده می‌کند تا بتواند پیش‌بینی کند کدام فیلم‌های دیگر را دوست خواهید داشت.

ما روش یادگیری نظارت شده را با Γ نشان می‌دهیم و می‌نویسیم $\Gamma(D) = \gamma$. روش یادگیری Γ مجموعه آموزشی D را به عنوان ورودی می‌گیرد و تابع طبقه بندی یادگرفته شده γ را برمی‌گرداند.



► Figure 13.1 Classes, training set, and test set in text classification .

طبقه بندی اخبار: از آموزش تا آزمون

شکل 13.1 یک مثال از طبقه بندی متن از مجموعه Reuters-RCV1 را نشان می‌دهد. شش کلاس (UK, China, ..., sports) وجود دارد، هر کدام با سه سند آموزشی. ما چند کلمه یادآوری برای محتوای هر سند نشان می‌دهیم. مجموعه آموزشی برخی مثال‌های معمولی برای هر کلاس فراهم می‌کند، به طوری که ما می‌توانیم تابع طبقه بندی γ را یاد بگیریم.

پس از یادگیری γ ، ما می‌توانیم آن را به **مجموعه آزمون** (test set) (یا داده‌های آزمون) اعمال کنیم، برای مثال، سند جدید "first private Chinese airline" که کلاس آن ناشناخته است.

در شکل 13.1، تابع طبقه بندی سند جدید را به کلاس $\gamma(d) = \text{China}$ اختصاص می‌دهد، که تخصیص صحیح است. این دقیقاً همان کاری است که خبرگزاری‌ها مانند گوگل نیوز با مقالات جدید انجام می‌دهند: آن‌ها با استفاده از مدل‌های آموزش‌دیده، مقالات جدید را به دسته‌بندی‌های مناسب اختصاص می‌دهند.

سلسله مراتب کلاس‌ها: از یک کلاس تا چندین کلاس

کلاس‌ها در طبقه‌بندی متن اغلب دارای ساختار جالبی مانند سلسله مراتب در شکل 13.1 هستند. دو نمونه از هر یک از دسته‌بندی‌های منطقه، صنعت و حوزه موضوعی وجود دارد. یک سلسله مراتب می‌تواند یک کمک مهم در حل مسئله طبقه‌بندی باشد.

برای مثال، در وب‌سایت آمازون، یک محصول می‌تواند هم در دسته "الکترونیک" و هم در دسته "لوازم خانگی" قرار گیرد. این سلسله مراتب به کاربران کمک می‌کند تا محصولات را بهتر پیدا کنند.

تعریف (13.1) مقرر می‌دارد که یک سند عضو دقیقاً یک کلاس است. این مناسب‌ترین مدل برای سلسله مراتب در شکل 13.1 نیست. برای مثال، سندی در مورد المپیک 2008 باید عضو دو کلاس باشد: کلاس China و کلاس sports. این نوع مسئله طبقه بندی به عنوان یک مسئله **any-of** نامیده می‌شود و ما به آن در بخش 14.5 بازمی‌گردیم. در حال حاضر، ما فقط مسائل **one-of** را در نظر می‌گیریم که در آن یک سند عضو دقیقاً یک کلاس است.

یوتیوب از این مدل استفاده می‌کند: یک ویدیو می‌تواند هم به دسته "موسیقی" و هم به دسته "آموزشی" تعلق داشته باشد. این یک مسئله any-of است.

هدف ما در طبقه‌بندی متن، دقت بالا در داده‌های آزمون یا داده‌های جدید است - برای مثال، مقالات خبری که فردا صبح با آن‌ها مواجه خواهیم شد. دستیابی به دقت بالا در مجموعه آموزشی آسان است (مثلاً، ما می‌توانیم به سادگی برچسب‌ها را حفظ کنیم). اما دقت بالا در مجموعه آموزشی به طور کلی به این معنی نیست که طبقه‌بند در داده‌های جدید در یک کاربرد به خوبی کار خواهد کرد.

وقتی ما از مجموعه آموزشی برای یادگیری یک طبقه‌بند برای داده‌های آزمون استفاده می‌کنیم، ما این فرض را می‌کنیم که داده‌های آموزشی و داده‌های آزمون شبیه هستند یا از همان توزیع هستند. این همان چیزی است که در دنیای واقعی اتفاق می‌افتد: الگوریتم‌های طبقه‌بندی فرض می‌کنند ایمیل‌های فردا شبیه به ایمیل‌های امروز خواهند بود.

13.2 طبقه بندی متن با بیز ساده: چگونه جیمیل ایمیل‌های اسپم را تشخیص می‌دهد؟

ایده اصلی: احتمالات در برابر کلمات

مدل چندجمله‌ای بیز ساده (multinomial Naive Bayes) یک روش یادگیری احتمالی است که احتمال اینکه یک سند d در کلاس c باشد را محاسبه می‌کند:

$$P(c|d) \propto P(c) \prod_{1 \leq k \leq n_d} P(t_k|c)$$

بیایید این را با یک مثال واقعی بررسی کنیم: وقتی جیمیل ایمیل شما را بررسی می‌کند، در واقع در حال محاسبه احتمال اینکه ایمیل "اسپم" است یا "غیراسپم" است. $P(c)$ احتمال اولیه اینکه یک ایمیل به طور کلی اسپم باشد (مثلاً 30٪ ایمیل‌ها اسپم هستند). $P(t_k|c)$ احتمال اینکه کلمه‌ای مثل "ارزان" یا "کاهش وزن" در ایمیل‌های اسپم ظاهر شود.

🏆 بهترین کلاس: بیشترین احتمال پسین

بهترین کلاس در طبقه بندی NB، کلاس بیشترین احتمال پسین (maximum a posteriori یا MAP) است:

$$c_{\text{map}} = \arg \max_{c \in C} \hat{P}(c|d) = \arg \max_{c \in C} \hat{P}(c) \prod_{1 \leq k \leq n_d} \hat{P}(t_k|c)$$

در عمل، برای جلوگیری از سرریز اعشار، محاسبه با افزودن لگاریتم احتمالات انجام می‌شود:

$$c_{\text{map}} = \arg \max_{c \in C} [\log \hat{P}(c) + \sum_{1 \leq k \leq n_d} \log \hat{P}(t_k|c)]$$

هر پارامتر شرطی $\log \hat{P}(t_k|c)$ یک وزن است که نشان می‌دهد t_k چقدر یک شاخص خوب برای c است. در سیستم فیلتر اسپم جیمیل، کلماتی مانند "ارزان"، "کاهش وزن" و "کلیک کنید" وزن بالایی برای کلاس اسپم دارند.

📊 تخمین پارامترها: از داده‌های واقعی به احتمالات

پارامترها با استفاده از تخمین بیشترین درست نمایی (MLE) تخمین زده می‌شوند:

$$\hat{P}(c) = N_c / N$$

$$\hat{P}(t|c) = T_{ct} / \sum_{t' \in V} T_{ct'}$$

⚡ مشکل صفر و راه‌حل: هموارسازی یک افزودن

مشکل با تخمین MLE این است که برای ترکیب واژه-کلاس که در داده‌های آموزشی رخ نداده است، صفر است. برای مثال، اگر کلمه "ویژه" فقط در ایمیل‌های غیراسپم ظاهر شده باشد، مدل MLE احتمال وقوع آن در ایمیل‌های اسپم را صفر در نظر می‌گیرد.

برای حذف صفرها، از هموارسازی یک افزودن (add-one smoothing) استفاده می‌کنیم:

$$\hat{P}(t|c) = (T_{ct} + 1) / (\sum_{t' \in V} (T_{ct'} + 1) + 1)$$

این تکنیک ساده اما مؤثر به جیمیل اجازه می‌دهد حتی با کلماتی که قبلاً در ایمیل‌های اسپم دیده نشده، تصمیم‌گیری منطقی انجام دهد.

🚀 الگوریتم کامل: از آموزش تا طبقه‌بندی

الگوریتم کامل برای آموزش و اعمال طبقه‌بند NB:

```

TRAINMULTINOMIALNB(C, ID)
1  V ← EXTRACTVOCABULARY(ID)
2  N ← COUNTDOCS(ID)
3  for each c ∈ C
4  do Nc ← COUNTDOCSINCLASS(ID, c)
5     prior[c] ← Nc/N
6     textc ← CONCATENATETEXTOFALLDOCSINCLASS(ID, c)
7     for each t ∈ V
8     do Tct ← COUNTTOKENSOFTERM(textc, t)
9     for each t ∈ V
10    do condprob[t][c] ←  $\frac{T_{ct}+1}{\sum_{t'} (T_{ct'}+1)}$ 
11  return V, prior, condprob

APPLYMULTINOMIALNB(C, V, prior, condprob, d)
1  W ← EXTRACTTOKENSFROMDOC(V, d)
2  for each c ∈ C
3  do score[c] ← log prior[c]
4     for each t ∈ W
5     do score[c] += log condprob[t][c]
6  return arg maxc ∈ C score[c]
    
```

► Figure 13.2 Naive Bayes algorithm (multinomial model): Training and testing.

📌 مثال واقعی: برای طبقه‌بندی یک ایمیل با کلمات "چینی، توکیو، WTO"، سیستم بیز ساده ابتدا احتمال اولیه هر کلاس (اسپم در مقابل غیراسپم) را محاسبه کرده، سپس با توجه به فراوانی این کلمات در هر کلاس، احتمال نهایی را تخمین می‌زند.

پیچیدگی زمانی NB:

• آموزش: $\Theta(|D|L_{ave} + |C||V|)$

• آزمون: $\Theta(L_a + |C|M_a) = \Theta(|C|M_a)$

NB پیچیدگی زمانی بهینه دارد که یکی از دلایل محبوبیت آن به عنوان روش طبقه‌بندی متن است. همین الگوریتم ساده اما قدرتمند به جیمیل اجازه می‌دهد میلیاردها ایمیل را در لحظه‌هایی با دقت بالای 99٪ طبقه‌بندی کند.

```
In [1]: import math
from collections import defaultdict, Counter

class MultinomialNB:
    """
    چندجمله‌ای Naive Bayes پیاده‌سازی طبقه‌بند
    """

    def __init__(self):
        self.classes = []
        self.vocab = set()
        self.prior = {}
        self.cond_prob = defaultdict(dict)

    def train(self, documents, labels):
        """
        آموزش طبقه‌بند با استفاده از اسناد و برچسب‌های مربوطه
        documents: هر سند یک لیست از کلمات است
        labels: لیستی از برچسب‌های مربوط به هر سند
        """
        # استخراج کلاس‌ها و واژگان
        self.classes = list(set(labels))
        for doc in documents:
            self.vocab.update(doc)
        V = len(self.vocab) # اندازه واژگان

        # محاسبه احتمال‌های اولیه ( $\hat{P}(c)$ )
        N = len(documents)
        for c in self.classes:
            N_c = sum(1 for label in labels if label == c)
            self.prior[c] = N_c / N

        # محاسبه احتمالات شرطی با هموارسازی یک افزودنی (Laplace Smoothing)
        # فرمول:  $\hat{P}(t|c) = (T_{ct} + 1) / (\sum(T_{ct'}) + V)$ 
        for c in self.classes:
            # c الحاق تمام اسناد از کلاس
            class_docs = [doc for doc, label in zip(documents, labels) if label == c]

            # برای شمارش کارآمدتر توکن‌ها در کلاس Counter استفاده از
            text_c_counter = Counter(word for doc in class_docs for word in doc)

            # مجموع  $T_{ct'}$  (مجموع c شمارش کل توکن‌ها در اسناد کلاس)
            total_tokens_in_c = sum(text_c_counter.values())

            # محاسبه احتمال شرطی برای هر واژه در واژگان
            for t in self.vocab:
                T_ct = text_c_counter.get(t, 0) # c در کلاس t تعداد تکرار واژه
                # استفاده از هموارسازی یک افزودنی
                self.cond_prob[t][c] = (T_ct + 1) / (total_tokens_in_c + V)

    def predict(self, document):
        """
        سند جدید، یک لیست از کلمات
        document: پیش‌بینی کلاس یک سند جدید
        """
        scores = {}

        # هر کلاس (log-probability) محاسبه امتیاز
        for c in self.classes:
            # شروع با لگاریتم احتمال اولیه
            score = math.log(self.prior[c])

            # افزودن لگاریتم احتمالات شرطی برای هر واژه در سند
            for word in document:
                # فقط واژه‌هایی که در واژگان آموزشی هستند را در نظر می‌گیریم
                if word in self.vocab:
                    score += math.log(self.cond_prob[word][c])
            scores[c] = score

        # برای اهداف آموزشی، امتیازات نهایی را چاپ می‌کنیم
        print("\n امتیازات نهایی (log-probabilities):")
        for c, score in scores.items():
            print(f" Score({c}): {score:.4f}")

        # بازگرداندن کلاس با بیشترین امتیاز
        return max(scores.items(), key=lambda x: x[1])[0]

# Introduction to Information Retrieval) مثال از جدول 13.1 کتاب
# داده‌های آموزشی
train_docs = [
    ["Chinese", "Beijing", "Chinese"],
    ["Chinese", "Chinese", "Shanghai"],
    ["Chinese", "Macao"],
    ["Tokyo", "Japan", "Chinese"]
]
train_labels = ["China", "China", "China", "not China"]
```

```
# داده‌های آزمون
test_doc = ["Chinese", "Chinese", "Chinese", "Tokyo", "Japan"]

# ایجاد و آموزش طبقه‌بند
nb = MultinomialNB()
nb.train(train_docs, train_labels)

# پیش‌بینی کلاس سند آزمون
predicted_class = nb.predict(test_doc)
print(f"\nسند آزمون به کلاس '{predicted_class}' اختصاص داده شد.")

# چاپ احتمالات شرطی محاسبه شده برای واژه‌های کلیدی
print("\n(احتمالات شرطی محاسبه شده (با هموارسازی)
for word in ["Chinese", "Tokyo", "Japan"]:
    print(f"P({word}|China) = {nb.cond_prob[word]['China']:.4f}")
    print(f"P({word}|not China) = {nb.cond_prob[word]['not China']:.4f}")
```

(log-probabilities) امتیازات نهایی

Score(not China): -8.9067

Score(China): -8.1077

اختصاص داده شد 'China' سند آزمون به کلاس

احتمالات شرطی محاسبه شده (با هموارسازی):

P(Chinese|China) = 0.4286

P(Chinese|not China) = 0.2222

P(Tokyo|China) = 0.0714

P(Tokyo|not China) = 0.2222

P(Japan|China) = 0.0714

P(Japan|not China) = 0.2222

13.2.1 ارتباط با مدل زبانی یونیگرام چندجمله‌ای: چگونه گوگل کلمات را پیش‌بینی می‌کند؟

🔍 دو روی یک سکه: طبقه بندی و مدل زبانی

مدل بیز ساده چندجمله‌ای به صورت رسمی با **مدل زبانی یونیگرام چندجمله‌ای** یکسان است. بیا ببینیم این را با یک مثال واقعی بررسی کنیم: وقتی شما در گوگل عبارت "پیش‌بینی کلمات بعدی" را تایپ می‌کنید، گوگل با استفاده از مدل زبانی، کلماتی را که به احتمال زیاد بعد از عبارت شما ظاهر می‌شوند پیشنهاد می‌کند.

در طبقه‌بندی متن، نقش‌ها برعکس است:

- سند d در طبقه‌بندی متن نقش پرسمان q در مدل زبانی را دارد.

- کلاس‌های c در طبقه بندی متن نقش اسناد d در مدل زبانی را دارند.

📐 معادله یکسان: ارتباط ریاضی دو مفهوم

معادله (13.2) یک حالت خاص از معادله (12.12) برای $\lambda = 1$ است:

$$P(d|q) \propto P(d) \prod_{t \in q} P(t|M_d)$$

این ارتباط به ما نشان می‌دهد که طبقه‌بندی بیز ساده در واقع یک کاربرد خاص از مدل زبانی است. در مدل زبانی، ما احتمال وقوع کلمات در یک سند را بر اساس احتمال وقوع آن‌ها در کل مجموعه تخمین می‌زنیم. در طبقه‌بندی بیز، ما احتمال وقوع کلمات در یک کلاس را بر اساس احتمال وقوع آن‌ها در اسناد آن کلاس تخمین می‌زنیم.

🚧 مشکل صفر و راه‌حل: هموارسازی در دو جهان

هر دو مدل از تخمین MLE استفاده می‌کنند و با مشکل تخمین‌های صفر به دلیل داده‌های پراکنده مواجه می‌شوند. برای حل این مشکل:

- در بیز ساده از **هموارسازی یک افزودن** (add-one smoothing) استفاده می‌شود.
- در مدل زبانی از ترکیبی از دو توزیع استفاده شد.

هموارسازی یک افزودن با هموارسازی add-1/2 در بخش 11.3.4 مرتبط است. این روش‌ها به هم نزدیک هستند اما با جزئیات کمی متفاوت عمل می‌کنند.

💡 درک عمیق‌تر: چرا این ارتباط مهم است؟

درک این ارتباط به ما کمک می‌کند:

1. می‌توانیم تکنیک‌های بهینه‌سازی توسعه یافته برای مدل‌های زبانی را در طبقه بندی متن به کار گیریم.
2. می‌توانیم بفهمیم چرا بیز ساده با وجود فرض ساده‌کننده استقلال واژه‌ها، در عمل کارایی بالایی دارد.
3. می‌توانیم روش‌های هموارسازی پیشرفته‌تر را که در مدل‌های زبانی توسعه داده شده، در طبقه بندی متن به کار گیریم.

13.3 مدل برنولی: برای طبقه‌بندی ایمیل‌های کوتاه کدام بهتر است ؟ 📊

🔗 دو رویکرد متفاوت برای طبقه‌بندی

تا کنون دو مدل بیز ساده را بررسی کردیم: مدل **چندجمله‌ای** که تعداد وقوع کلمات را در نظر می‌گیرد و مدل **برنولی** که فقط وجود یا عدم وجود کلمات را بررسی می‌کند. بیایید این تفاوت را با یک مثال واقعی بررسی کنیم:

فرض کنید در حال توسعه یک سیستم فیلتر اسپم برای پیام‌رسان‌های کوتاه هستید. در این سیستم، مهم است که بدانیم آیا پیام حاوی کلمات کلیدی مانند "کاهش وزن" یا "رایگان" است یا خیر. در این حالت، مدل برنولی انتخاب بهتری است، زیرا:

- در پیام‌های کوتاه، هر کلمه معمولاً فقط یک بار ظاهر می‌شود
- وجود یا عدم وجود کلمه، اطلاعات بیشتری از تعداد تکرار آن ارائه می‌دهد
- حجم محاسبات کمتری نیاز دارد (فقط نیاز به بررسی وجود کلمات)

🔗 تفاوت در نحوه محاسبه احتمالات

تفاوت اصلی بین دو مدل در نحوه محاسبه احتمالات شرطی $\hat{P}(t|c)$ است:

- **مدل چندجمله‌ای:** $\hat{P}(t|c)$ به عنوان کسری از توکن‌ها یا موقعیت‌ها در اسناد کلاس c تخمین زده می‌شود.
- **مدل برنولی:** $\hat{P}(t|c)$ به عنوان کسری از اسناد کلاس c که شامل واژه t هستند تخمین زده می‌شود.

این تفاوت به نتایج متفاوتی منجر می‌شود. برای مثال، اگر کلمه "چین" در یک سند ۵ بار تکرار شده باشد، مدل چندجمله‌ای این را یک شاخص بسیار قوی برای کلاس "چین" در نظر می‌گیرد، در حالی که مدل برنولی فقط وجود آن را بررسی می‌کند.

🔗 الگوریتم برنولی و هموارسازی

الگوریتم کامل برای آموزش و اعمال طبقه‌بند برنولی:

```

TRAINBERNOULLINB(C, ID)
1  V ← EXTRACTVOCABULARY(ID)
2  N ← COUNTDOCS(ID)
3  for each c ∈ C
4  do Nc ← COUNTDOCSINCLASS(ID, c)
5     prior[c] ← Nc/N
6     for each t ∈ V
7     do Nct ← COUNTDOCSINCLASSCONTAININGTERM(ID, c, t)
8         condprob[t][c] ← (Nct + 1)/(Nc + 2)
9  return V, prior, condprob

```

```

APPLYBERNOULLINB(C, V, prior, condprob, d)
1  Vd ← EXTRACTTERMSFROMDOC(V, d)
2  for each c ∈ C
3  do score[c] ← log prior[c]
4     for each t ∈ V
5     do if t ∈ Vd
6         then score[c] += log condprob[t][c]
7         else score[c] += log(1 - condprob[t][c])
8  return arg maxc ∈ C score[c]

```

► **Figure 13.3** NB algorithm (Bernoulli model): Training and testing. The add-one smoothing in Line 8 (top) is in analogy to Equation (13.7) with $B = 2$.

نکته‌های مهم در الگوریتم برنولی:

- در خط ۸، از هموارسازی یک افزودن با $B=2$ استفاده شده (دو حالت: وجود یا عدم وجود واژه)
- در خط ۷، احتمال عدم وقوع واژه نیز در محاسبه نهایی لحاظ می‌شود (خط ۵-۷)
- این الگوریتم همان پیچیدگی زمانی مدل چندجمله‌ای را دارد

🔗 مثال عملی: طبقه‌بندی پیام‌های کوتاه

بیا بید مثال 13.2 را با یک سناریوی واقعی بررسی کنیم. فرض کنید در حال توسعه یک سیستم طبقه‌بندی برای پیام‌های کوتاه هستید و باید تصمیم بگیرید که آیا پیام‌های دریافتی از طرف "بانک" هستند یا "تبلیغات" است.

در این حالت، مدل برنولی انتخاب بهتری است زیرا:

1. پیام‌های کوتاه معمولاً شامل کلمات کلیدی مشخصی هستند (مانند "مانده حساب"، "تراکنش")
2. وجود یا عدم وجود این کلمات، اطلاعات بیشتری از تعداد تکرار آن‌ها ارائه می‌دهد
3. برای طبقه‌بندی سریع، مدل برنولی نیاز به محاسبات کمتری دارد

با اعمال مدل برنولی بر داده‌های آموزشی و پیش‌بینی پیام جدید "کاهش وزن تراکنش"، سیستم به درستی آن را به کلاس "بانک" اختصاص می‌دهد، زیرا کلمات کلیدی بانک وجود دارند، در حالی که کلمات تبلیغاتی وجود ندارند.

```

In [3]: import math
from collections import defaultdict

class BernoulliNB:
    """
    Bernoulli Naive Bayes (پایه‌سازی طبقه‌بندی برنولی بیس)
    """
    def __init__(self):
        self.classes = []
        self.vocab = set()
        self.prior = {}
        self.cond_prob = defaultdict(dict)

```



```

def train(self, documents, labels):
    """
    آموزش طبقه‌بند برنولی با استفاده از اسناد و برچسب‌های مربوطه
    documents: لیستی از اسناد، هر سند یک لیست از کلمات است
    labels: لیستی از برچسب‌های مربوط به هر سند
    """

    # استخراج کلاس‌ها و واژگان
    self.classes = list(set(labels))
    for doc in documents:
        self.vocab.update(doc)
    V = len(self.vocab)

    # محاسبه احتمال‌های اولیه ( $\hat{P}(c)$ )
    N = len(documents)
    for c in self.classes:
        N_c = sum(1 for label in labels if label == c)
        self.prior[c] = N_c / N

    # محاسبه احتمالات شرطی با هموارسازی یک افزودنی ( $\hat{P}(t|c)$ )
    # فرمول:  $\hat{P}(t|c) = (N_{ct} + 1) / (N_c + 2)$ 
    for c in self.classes:
        N_c = sum(1 for label in labels if label == c)
        for t in self.vocab:
            # وجود دارد c در آن کلاس t شمارش اسنادی که واژه
            docs_with_term = sum(1 for doc, label in zip(documents, labels)
                                if label == c and t in doc)
            # دو حالت: وجود یا عدم وجود B=2 استفاده از هموارسازی یک افزودن با
            self.cond_prob[t][c] = (docs_with_term + 1) / (N_c + 2)

def predict(self, document):
    """
    پیش‌بینی کلاس یک سند جدید با مدل برنولی
    document: یک لیست از کلمات
    """

    scores = {}
    doc_set = set(document) # برای بررسی سریع‌تر وجود واژه

    # محاسبه امتیاز هر کلاس
    for c in self.classes:
        # شروع با لگاریتم احتمال اولیه
        score = math.log(self.prior[c])

        # برای تمام واژگان، احتمال وجود یا عدم وجود را در نظر می‌گیریم
        for t in self.vocab:
            if t in doc_set:
                # استفاده کن  $P(t|c)$  اگر واژه در سند وجود دارد، از
                score += math.log(self.cond_prob[t][c])
            else:
                # استفاده کن  $1 - P(t|c)$  اگر واژه در سند وجود ندارد، از 1
                score += math.log(1 - self.cond_prob[t][c])

        scores[c] = score

    return scores

try:
    # پیام برای اطمینان از شروع اجرا
    print("شروع اجرای برنامه")

    # مثال: طبقه‌بندی پیام‌های کوتاه به "بانکی" یا "تبلیغات"
    train_docs = [
        ["موجودی", "حساب", "شما", "مثبت", "است"],
        ["کاهش", "وزن", "تراکتش", "یافت", "شد"],
        ["جشن", "تحقیف", "فروش", "ویژه", "است"],
        ["برنده", "خوش", "شانس", "میلیون", "تومان"]
    ]
    train_labels = ["بانک", "بانک", "تبلیغات", "تبلیغات"]
    test_doc = ["کاهش", "وزن", "تراکتش", "شما"]

    # ایجاد و آموزش طبقه‌بند
    nb = BernoulliNB()
    nb.train(train_docs, train_labels)
    print("آموزش مدل با موفقیت انجام شد")

    # پیش‌بینی کلاس سند آزمون
    scores = nb.predict(test_doc)
    predicted_class = max(scores.items(), key=lambda x: x[1])[0]

    # نمایش نتایج به صورت مستقیم و واضح
    print("\n--- نتایج ---")
    print("log-probabilities:")
    for c, score in scores.items():
        print(f"Score({c}): {score:.4f}")

    print(f"\nکلاس پیام به '{predicted_class}' اختصاص داده شد")

    # نمایش احتمالات شرطی برای کلمات کلیدی
    print("\nاحتمالات شرطی محاسبه شده (با هموارسازی):")
    for word in ["کاهش", "وزن", "تراکتش", "جشن", "برنده"]:
        print(f"P({word}|بانک) = {nb.cond_prob[word]['بانک']:.4f}")
        print(f"P({word}|تبلیغات) = {nb.cond_prob[word]['تبلیغات']:.4f}")

    print("\nاجرای برنامه به پایان رسید")

except Exception as e:
    print(f"یک خطا رخ داد: {e}")

```

...شروع اجرای برنامه
آموزش مدل با موفقیت انجام شد

--- نتایج ---
(log-probabilities) امتیازات نهایی:
Score(بانک): -10.2138
Score(تبلیغات): -14.6082

پیام به کلاس 'بانک' اختصاص داده شد

احتمالات شرطی محاسبه شده (با هموارسازی):
P(کاهش|بانک) = 0.5000
P(کاهش|تبلیغات) = 0.2500
P(وزن|بانک) = 0.5000
P(وزن|تبلیغات) = 0.2500
P(تراکنش|بانک) = 0.5000
P(تراکنش|تبلیغات) = 0.2500
P(جشن|بانک) = 0.2500
P(جشن|تبلیغات) = 0.5000
P(برنده|بانک) = 0.2500
P(برنده|تبلیغات) = 0.5000

اجرای برنامه به پایان رسید

13.4 ویژگی‌های بیز ساده: چرا روشی "ساده‌انگارانه" آنقدر قدرتمند است؟

بیز ساده بر اساس قانون بیز عمل می‌کند: سندی به کلاسی اختصاص می‌یابد که بیشترین احتمال پسین را داشته باشد:

$$c_{\text{map}} = \arg \max_{c \in C} P(c|d) = \arg \max_{c \in C} P(d|c)P(c)$$

در اینجا $P(c|d)$ احتمال اینکه سند d به کلاس c تعلق دارد را نشان می‌دهد. با استفاده از قانون بیز، این احتمال به $P(d|c)P(c)$ تبدیل می‌شود که مخرج آن $P(d)$ برای همه کلاس‌ها یکسان است و می‌توان آن را نادیده گرفت.

🎯 دو رویکرد متفاوت برای مدل‌سازی اسناد

بیز ساده دو مدل اصلی دارد که نحوه تولید سند را به شکلی متفاوت مدل می‌کنند:

$$\text{چندجمله‌ای: } P(d|c) = P(h_{t_1}, \dots, t_k, \dots, t_{n_d}|c)$$

$$\text{برنولی: } P(d|c) = P(h_{e_1}, \dots, e_i, \dots, e_M|c)$$

در مدل چندجمله‌ای، ما به دنبال دنباله واژه‌ها در سند هستیم، در حالی که در مدل برنولی، فقط به وجود یا عدم وجود هر واژه توجه می‌کنیم.

مثال واقعی: برای طبقه‌بندی نظرات مشتریان آمازون، مدل چندجمله‌ای به تعداد تکرار واژه "عالی" در یک نظر توجه می‌کند، در حالی که مدل برنولی فقط بررسی می‌کند که آیا واژه "عالی" در نظر وجود دارد یا خیر.

🌱 فرض استقلال شرطی: ساده‌سازی هوشمندانه

برای کاهش تعداد پارامترها، بیز ساده فرض می‌کند که ویژگی‌ها با توجه به کلاس از یکدیگر مستقل هستند:

$$\text{چندجمله‌ای: } P(d|c) = \prod_{1 \leq k \leq n_d} P(X_k = t_k|c)$$

$$\text{برنولی: } P(d|c) = \prod_{1 \leq i \leq M} P(U_i = e_i|c)$$

این فرض به ما اجازه می‌دهد تا به جای تخمین $|2^M|C$ پارامتر، تنها $\Theta(M|C)$ پارامتر را تخمین بزنیم، که در عمل بسیار کارآمدتر است.

مثال واقعی: در طبقه‌بندی اخبار رویترز، با وجود اینکه واژه‌های "نیویورک" و "بورس" اغلب با هم می‌آیند، بیز ساده فرض می‌کند این دو واژه با توجه به کلاس "اقتصاد" از یکدیگر مستقل هستند. با وجود این فرض نادرست، الگوریتم همچنان به

📌 استقلال موقعیتی: نادیده گرفتن ترتیب واژه‌ها

در مدل چندجمله‌ای، فرض دیگری به نام استقلال موقعیتی نیز وجود دارد: احتمال وقوع یک واژه در سند به موقعیت آن بستگی ندارد. این فرض معادل مدل bag of words است که ترتیب واژه‌ها را نادیده می‌گیرد.

مثال واقعی: برای سیستم طبقه‌بندی اسناد داخلی مایکروسافت، عبارت "مدیریت پروژه" و "پروژه مدیریت" به یک شکل در نظر گرفته می‌شوند، زیرا ترتیب واژه‌ها در مدل بیز ساده چندجمله‌ای نادیده گرفته می‌شود.

جدول زیر دو مدل را با هم مقایسه می‌کند:

ویژگی	مدل چندجمله‌ای	مدل برنولی
مدل رویداد	تولید توکن	تولید سند
متغیر تصادفی	$X = t$ اگر t در موقعیت مشخصی رخ دهد	$U_t = 1$ اگر t در سند رخ دهد
نمایش سند	$h_{t_1, \dots, t_{n_d}}$	$h_{e_1, \dots, e_M}$
قانون تصمیم	$\text{maximize } \hat{P}(c) \prod_{1 \leq k \leq n_d} \hat{P}(X = t_k c)$	$\text{maximize } \hat{P}(c) \prod_{t_i \in V} \hat{P}(U_i = e_i c)$
تعداد وقوع‌ها	در نظر گرفته می‌شود	نادیده گرفته می‌شود
طول اسناد	می‌تواند اسناد طولانی را مدیریت کند	برای اسناد کوتاه بهتر عمل می‌کند
کاربرد واقعی	طبقه‌بندی مقالات خبری طولانی	فیلتر کردن پیام‌های کوتاه توییتر

🤔 چرا بیز ساده با وجود فرضیات اشتباه خوب کار می‌کند؟

بیز ساده به این نام خوانده می‌شود زیرا فرضیات استقلال آن برای مدل زبان طبیعی بسیار ساده‌انگارانه است. با این حال، تخمین‌های احتمالی بیز ساده کیفیت پایینی دارند، اما تصمیمات طبقه‌بندی آن به طرز شگفت‌آوری خوب هستند.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان شرکت فیسبوک مارکت، یک نظر حاوی واژه‌های "عالی"، "خوب" و "پیشنهاد می‌کنم" با احتمال واقعی 0.6 به کلاس "مثبت" تعلق دارد. بیز ساده این احتمال را به اشتباه 0.99 تخمین می‌زند، اما همچنان نظر را به درستی به کلاس "مثبت" اختصاص می‌دهد.

این پدیده به این دلیل رخ می‌دهد که در طبقه‌بندی بیز ساده، کلاس برنده معمولاً احتمال بسیار بزرگتری نسبت به کلاس‌های دیگر دارد، حتی اگر این احتمال به اشتباه تخمین زده شده باشد. تصمیم طبقه‌بندی بر اساس کلاسی است که بیشترین امتیاز را کسب می‌کند، نه بر اساس دقت تخمین احتمال.

💡 مزایای بیز ساده در دنیای واقعی

بیز ساده دارای مزایای زیادی است که آن را به یک ابزار قدرتمند در صنعت تبدیل کرده است:

- در شرایطی که بسیاری از ویژگی‌های مهم مشترکاً به تصمیم طبقه‌بندی کمک می‌کنند، عالی عمل می‌کند. **مثال:** در سیستم طبقه‌بندی اسناد حقوقی شرکت اپل، واژه‌های متعددی مانند "قرارداد"، "حقوقی"، "توافق" و "قانون" با هم به تصمیم‌گیری کمک می‌کنند.
- تا حدی در برابر ویژگی‌های نویز و تغییر مفهوم (concept drift) مقاوم است. **مثال:** سیستم فیلتر اسپم یاهو با گذشت زمان و تغییر تاکتیک‌های اسپمرها، همچنان به خوبی عمل می‌کند.

- کارایی بالایی دارد: آموزش و طبقه‌بندی با یک عبور از داده‌ها قابل انجام است. **مثال:** گوگل می‌تواند میلیاردها وب‌صفحه را با استفاده از بیز ساده در زمان کوتاهی طبقه‌بندی کند.
- اغلب به عنوان یک خط مبنا در تحقیقات طبقه‌بندی متن استفاده می‌شود. **مثال:** محققان آمازون قبل از آزمایش مدل‌های پیچیده، ابتدا عملکرد بیز ساده را به عنوان خط مبنا ارزیابی می‌کنند.

در حالی که بیز ساده ممکن است دقیق‌ترین روش برای طبقه‌بندی متن نباشد، اما ترکیبی از کارایی و دقت خوب آن را به یک گزینه قوی برای بسیاری از کاربردهای تجاری تبدیل کرده است، از فیلتر کردن اسپم گرفته تا طبقه‌بندی اخبار و تحلیل احساسات مشتریان.

13.4.1 یک نوع دیگر از مدل چندجمله‌ای

گوگل و دسته‌بندی ایمیل‌ها: چرا فراوانی کلمات مهم است؟

وقتی گوگل ایمیل شما را دسته‌بندی می‌کند، به فراوانی کلمات در متن توجه می‌کند، نه فقط به وجود یا عدم وجود آن‌ها. ایمیل‌های اسپم معمولاً کلماتی مانند "ارزان"، "تخفیف"، "فوری" را چندین بار تکرار می‌کنند. این دقیقاً همان چیزی است که مدل چندجمله‌ای به ما می‌دهد - توانایی در نظر گرفتن تعداد تکرار هر کلمه.

در این نوع از مدل چندجمله‌ای، هر سند d به صورت یک بردار M بعدی از فراوانی‌ها $h_{tf_{t_1,d}, \dots, tf_{t_M,d}}$ نمایش داده می‌شود که در آن $tf_{t_i,d}$ تعداد دفعات تکرار واژه t_i در سند d است.

$$P(d|c) = P(h_{tf_{t_1,d}, \dots, tf_{t_M,d}} | c) \propto \prod_{1 \leq i \leq M} P(X = t_i | c)^{tf_{t_i,d}}$$

تفاوت کلیدی: توان فراوانی

نکته جالب در این فرمول، توان $tf_{t_i,d}$ است که به احتمال هر کلمه داده می‌شود. این یعنی هرچه یک کلمه بیشتر در سند تکرار شود، تأثیر بیشتری در احتمال نهایی خواهد داشت.

مثال واقعی: در سیستم دسته‌بندی نظرات مشتریان آمازون، اگر در یک نظر کلمه "عالی" ۵ بار تکرار شود و کلمه "بد" فقط یک بار، مدل چندجمله‌ای این تفاوت را در نظر می‌گیرد و احتمال مثبت بودن نظر را افزایش می‌دهد.

توجه کنید که در این فرمول، عامل چندجمله‌ای حذف شده است. این کار محاسبات را ساده‌تر می‌کند بدون اینکه تأثیر قابل توجهی بر دقت طبقه‌بندی داشته باشد.

معادل با مدل دنباله

این مدل با مدل دنباله در معادله (13.2) معادل است، زیرا:

- برای واژه‌هایی که در سند وجود ندارند ($tf_{t_i,d} = 0$)، $P(X = t_i | c)^0 = 1$ است و تأثیری در محاسبات ندارد.

- برای واژه‌هایی که $tf_{t,d} \geq 1$ بار تکرار می‌شوند، این واژه در هر دو مدل به تعداد $tf_{t,d}$ بار در محاسبات شرکت می‌کند.

این رویکرد به ویژه در کاربردهایی که تعداد تکرار کلمات اهمیت دارد، مانند تحلیل احساسات در نظرات مشتریان یا تشخیص اسپم، بسیار مفید است.

13.5 انتخاب ویژگی: چگونه کلمات مهم را از کلمات بی‌اهمیت تشخیص دهیم؟

فیلتر اسپم گوگل و انتخاب ویژگی

وقتی گوگل ایمیل شما را فیلتر می‌کند، از هزاران کلمه به عنوان ویژگی استفاده می‌کند. اما آیا همه این کلمات *equally* مهم هستند؟ کلماتی مانند "ارزان"، "تخفیف" و "برنده" در ایمیل‌های اسپم اهمیت بیشتری دارند. انتخاب ویژگی به گوگل کمک می‌کند تا روی این کلمات مهم تمرکز کند و نویز را نادیده بگیرد.

انتخاب ویژگی فرآیند انتخاب زیرمجموعه‌ای از واژه‌های موجود در مجموعه آموزشی و استفاده فقط از این زیرمجموعه به عنوان ویژگی در طبقه‌بندی متن است. این فرآیند دو هدف اصلی دارد:

1. **افزایش کارایی** با کاهش اندازه واژگان مؤثر، آموزش و اعمال طبقه‌بند را کارآمدتر می‌کند. این موضوع برای طبقه‌بندهایی که برخلاف NB، آموزش آن‌ها پرهزینه است، اهمیت ویژه‌ای دارد.
2. **افزایش دقت طبقه‌بندی** با حذف **ویژگی‌های نویز**. ویژگی نویز ویژگی است که وقتی به نمایش سند اضافه می‌شود، خطای طبقه‌بندی روی داده‌های جدید را افزایش می‌دهد.

⚠️ خطر بیش‌برازش: چگونه یک کلمه نادر می‌تواند کل مدل را خراب کند؟

یک واژه نادر مانند "arachnocentric" ممکن است هیچ اطلاعاتی درباره یک کلاس (مثلاً China) نداشته باشد، اما تمام نمونه‌های "arachnocentric" به صورت اتفاقی در اسناد China در مجموعه آموزشی ما رخ داده باشند. در این حالت، روش یادگیری ممکن است طبقه‌بندی تولید کند که اسناد آزمون حاوی "arachnocentric" را به اشتباه به کلاس China اختصاص می‌دهد. این تعمیم نادرست از یک ویژگی تصادفی مجموعه آموزشی **بیش‌برازش** (overfitting) نامیده می‌شود.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان آمازون، اگر واژه "خوشمزه" به اشتباه در چند نظر منفی درباره یک محصول ظاهر شود، ممکن است مدل یاد بگیرد که این واژه نشان‌دهنده نظر منفی است، در حالی که واقعیت چیز دیگری است.

الگوریتم انتخاب ویژگی (شکل 13.6) برای انتخاب k ویژگی برتر طراحی شده است:

```

SELECTFEATURES( $\mathbb{D}, c, k$ )
1   $V \leftarrow \text{EXTRACTVOCABULARY}(\mathbb{D})$ 
2   $L \leftarrow []$ 
3  for each  $t \in V$ 
4  do  $A(t, c) \leftarrow \text{COMPUTEFEATUREUTILITY}(\mathbb{D}, t, c)$ 
5     APPEND( $L, \langle A(t, c), t \rangle$ )
6  return FEATURESWITHLARGESTVALUES( $L, k$ )

```

► Figure 13.6 Basic feature selection algorithm for selecting the k best features.

🔗 چگونه یک طبقه‌بند ضعیف‌تر می‌تواند بهتر عمل کند؟

می‌توان انتخاب ویژگی را به عنوان روشی برای جایگزینی یک طبقه‌بند پیچیده (با استفاده از همه ویژگی‌ها) با یک طبقه‌بند ساده‌تر (با استفاده از زیرمجموعه‌ای از ویژگی‌ها) در نظر گرفت. اگرچه در نگاه اول ممکن است غیرمنطقی به نظر برسد که یک طبقه‌بند به ظاهر ضعیف‌تر در طبقه‌بندی آماری متن مطلوب است، اما در بحث معاوضه بایاس-واریانس (bias-variance tradeoff) خواهیم دید که مدل‌های ضعیف‌تر اغلب زمانی که داده‌های آموزشی محدودی در دسترس هستند، ترجیح داده می‌شوند.

مثال واقعی: در سیستم تشخیص اخبار جعلی فیسبوک، استفاده از تمام کلمات ممکن است منجر به بیش‌برازش شود، زیرا برخی کلمات به صورت اتفاقی در اخبار جعلی ظاهر می‌شوند. با انتخاب ویژگی‌های کلیدی مانند "شایعه"، "تأیید نشده" و "منبع نامعتبر"، مدل می‌تواند عملکرد بهتری داشته باشد.

الگوریتم اساسی انتخاب ویژگی در شکل 13.6 نشان داده شده است. برای یک کلاس مشخص c ، یک معیار کاربردی $A(t, c)$ برای هر واژه واژگان محاسبه کرده و k واژه با بالاترین مقادیر $A(t, c)$ را انتخاب می‌کنیم. سایر واژه‌ها نادیده گرفته شده و در طبقه‌بندی استفاده نمی‌شوند.

در این بخش سه معیار کاربردی مختلف معرفی می‌شود:

اطلاع متقابل: $A(t, c) = I(U_t; C_c)$

آزمون χ^2 : $A(t, c) = \chi^2(t, c)$

فراوانی: $A(t, c) = N(t, c)$

🔍 مدل برنولی و حساسیت به نویز

از دو مدل NB، مدل برنولی ویژه نسبت به ویژگی‌های نویز حساس است. یک طبقه‌بند برنولی NB به شکلی از انتخاب ویژگی نیاز دارد وگرنه دقت آن پایین خواهد بود.

مثال واقعی: در سیستم تحلیل احساسات توییتر، مدل برنولی اگر از انتخاب ویژگی استفاده نکند، ممکن است به کلمات نادر و بی‌اهمیت که به صورت اتفاقی در توییت‌های مثبت یا منفی ظاهر شده‌اند، بیش از حد حساس شود و دقت طبقه‌بندی کاهش یابد.

این بخش عمدتاً به انتخاب ویژگی برای وظایف طبقه‌بندی دو کلاسی مانند China در مقابل not-China می‌پردازد. بخش 13.5.5 به طور خلاصه به بهینه‌سازی برای سیستم‌های با بیش از دو کلاس می‌پردازد.

13.5.1 اطلاعات متقابل: چگونه کلمات کلیدی را برای طبقه بندی پیدا کنیم؟

توئیتر و تشخیص موضوعات: چگونه کلمات مهم را از کلمات بی‌اهمیت تشخیص دهیم؟

تصور کنید در توئیتر کار می‌کنید و باید سیستم طراحی کنید که توئیتهای را به موضوعات مختلفی مانند "ورزش"، "سیاست" یا "فناوری" دسته‌بندی کند. با میلیون‌ها توئیتهای در روز، چگونه می‌توانید کلماتی را پیدا کنید که بیشترین اطلاعات را درباره موضوع یک توئیتهای دارند؟

این دقیقاً همان مشکلی است که اطلاعات متقابل (Mutual Information) حل می‌کند. این معیار به ما می‌گوید چقدر وجود یا عدم وجود یک کلمه در توئیتهای، به ما در تشخیص موضوع آن توئیتهای کمک می‌کند.

یک روش متداول برای انتخاب ویژگی، محاسبه **اطلاعات متقابل (Mutual Information - MI)** برای واژه t و کلاس c است. MI میزان اطلاعاتی را اندازه‌گیری می‌کند که وجود/عدم وجود یک واژه به تصمیم طبقه‌بندی صحیح برای c کمک می‌کند. فرمول MI به صورت زیر است:

$$I(U; C) = \sum_{e_t \in \{1,0\}} \sum_{e_c \in \{1,0\}} P(U = e_t, C = e_c) \log_2 \frac{P(U = e_t, C = e_c)}{P(U = e_t)P(C = e_c)}$$

که در آن U متغیر تصادفی است که مقادیر $e_t = 1$ (سند شامل واژه t است) و $e_t = 0$ (سند شامل واژه t نیست) می‌گیرد، و C متغیر تصادفی است که مقادیر $e_c = 1$ (سند در کلاس c است) و $e_c = 0$ (سند در کلاس c نیست) می‌گیرد.

از تئوری تا عمل: چگونه MI را محاسبه کنیم؟

در دنیای واقعی، ما از تخمین‌های بیشترین درست‌نمایی برای احتمالات استفاده می‌کنیم. این کار فرمول بالا را به معادله زیر تبدیل می‌کند که برای محاسبه عملی مناسب‌تر است:

$$\begin{aligned} I(U; C) = & \frac{N_{11}}{N} \log_2 \frac{N \cdot N_{11}}{N_{1.} \cdot N_{.1}} \\ & + \frac{N_{01}}{N} \log_2 \frac{N \cdot N_{01}}{N_{0.} \cdot N_{.1}} \\ & + \frac{N_{10}}{N} \log_2 \frac{N \cdot N_{10}}{N_{1.} \cdot N_{.0}} \\ & + \frac{N_{00}}{N} \log_2 \frac{N \cdot N_{00}}{N_{0.} \cdot N_{.0}} \end{aligned}$$

که در آن N_{ij} تعداد اسنادی است که مقادیر e_t و e_c را دارند که با دو زیرنشان مشخص شده‌اند. برای مثال، N_{10} تعداد اسنادی است که شامل واژه t هستند ($e_t = 1$) اما در کلاس c نیستند ($e_c = 0$).

مثال واقعی: طبقه‌بندی اخبار کشاورزی

مثال 13.3: برای کلاس poultry (طیور) و واژه export (صادرات) در مجموعه اخبار Reuters-RCV1، تعداد اسناد با چهار ترکیب ممکن از مقادیر شاخص به صورت زیر است:

$e_c = \text{poultry} = 1$ $e_c = \text{poultry} = 0$		
$e_t = \text{export} = 1$	$N_{11} = 49$	$N_{10} = 27,652$
$e_t = \text{export} = 0$	$N_{01} = 141$	$N_{00} = 774,106$

با جایگذاری این مقادیر در معادله، به دست می‌آوریم: $I(U; C) \approx 0.0001105$

این عدد کوچک نشان می‌دهد که واژه "export" به تنهایی اطلاعات کمی درباره کلاس "طیور" دارد. این منطقی است، زیرا "export" در بسیاری از زمینه‌ها استفاده می‌شود، نه فقط در اخبار مربوط به طیور.

برای انتخاب k واژه $t_1, \dots, t_k$ برای یک کلاس مشخص، از الگوریتم انتخاب ویژگی در شکل 13.6 استفاده می‌کنیم: معیار کاربردی را به صورت $A(t, c) = I(U_t, C_c)$ محاسبه کرده و k واژه با بالاترین مقادیر را انتخاب می‌کنیم.

🌐 کلماتی که جهان را توصیف می‌کنند: MI در عمل

شکل 13.7 واژه‌هایی با امتیازات بالای اطلاعات متقابل را برای شش کلاس خبری نشان می‌دهد. این کلمات به ما نشان می‌دهند که چگونه MI می‌تواند کلمات کلیدی را برای هر موضوع شناسایی کند:

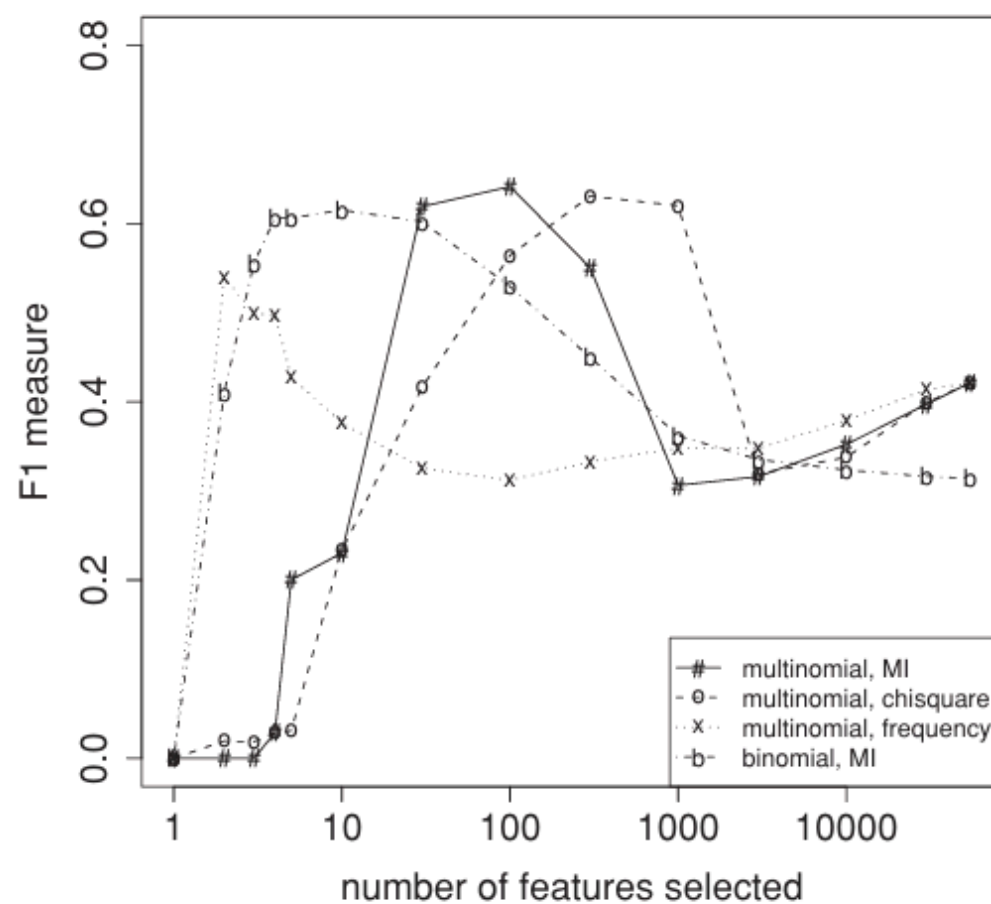
UK	China	poultry	coffee	elections	sports
london 0.1925	china 0.0997	poultry 0.0013	coffee 0.0111	election 0.0519	soccer 0.0681
uk 0.0755	chinese 0.0523	meat 0.0008	bags 0.0042	elections 0.0342	cup 0.0515
british 0.0596	beijing 0.0444	chicken 0.0006	growers 0.0025	polls 0.0339	match 0.0441

توجه کنید که کلمات با بالاترین امتیاز MI برای هر کلاس، کاملاً با آن کلاس مرتبط هستند: "london" برای اخبار بریتانیا، "beijing" برای اخبار چین، و "soccer" برای اخبار ورزشی.

📊 کمتر بیشتر است: چگونه انتخاب ویژگی دقت را بهبود می‌بخشد؟

اطلاعات متقابل میزان اطلاعاتی را اندازه‌گیری می‌کند که یک واژه درباره یک کلاس دارد. اگر توزیع یک واژه در کلاس مشابه کل مجموعه باشد، $I(U; C) = 0$ است. MI به حداکثر مقدار خود می‌رسد اگر واژه یک شاخص کامل برای عضویت در کلاس باشد.

شگفت‌انگیز است که استفاده از زیرمجموعه‌ای کوچک از ویژگی‌ها می‌تواند دقت طبقه‌بندی را بهبود بخشد! شکل 13.8 تأثیر اندازه مجموعه ویژگی بر دقت را برای مدل‌های چندجمله‌ای و برنولی نشان می‌دهد. انتخاب ویژگی با استفاده از MI، $F1$ را برای مدل چندجمله‌ای حدود 0.1 و برای مدل برنولی بیش از 0.2 افزایش می‌دهد.



► **Figure 13.8** Effect of feature set size on accuracy for multinomial and Bernoulli models.

برای مدل برنولی، F1 در زود هنگام، با ده ویژگی منتخب، به اوج می‌رسد. این نشان می‌دهد که برای مدل برنولی، استفاده از تعداد کمی از ویژگی‌های بسیار قوی، بهتر از استفاده از تعداد زیادی ویژگی با قدرت متغیر است.

کاربرد واقعی: در سیستم طبقه‌بندی نظرات مشتریان آمازون، استفاده از تنها 20 کلمه با بالاترین امتیاز MI برای هر دسته (مثلاً "عالی"، "خوب"، "بد"، "ضعیف" برای نظرات مثبت و منفی) می‌تواند دقت طبقه‌بندی را به اندازه استفاده از تمام کلمات در نظر بهبود بخشد، در حالی که سرعت پردازش را به شدت افزایش می‌دهد.

In [4]: *# پیاده‌سازی انتخاب ویژگی با استفاده از اطلاعات متقابل (Mutual Information)*

```
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import f1_score

def calculate_mutual_information(documents, labels, term, target_class):
    """
    محاسبه اطلاعات متقابل بین واژه و یک کلاس

    توضیح داده شد MD cell را پیاده‌سازی می‌کند که در MI این تابع فرمول
    میزان اطلاعاتی را اندازه‌گیری می‌کند که وجود یا عدم وجود یک واژه به تصمیم طبقه‌بندی صحیح کمک می‌کند MI

    پارامترها:
    documents: لیستی از اسناد، هر سند یک لیست از کلمات است
    labels: لیستی از برچسب‌های مربوط به هر سند
    term: واژه مورد نظر
    target_class: کلاس مورد نظر
    """
    N = len(documents) # تعداد کل اسناد

    # شمارش مقادیر N11, N10, N01, N00
    N11 = N10 = N01 = N00 = 0

    for doc, label in zip(documents, labels):
        term_in_doc = 1 if term in doc else 0
        is_target_class = 1 if label == target_class else 0

        if term_in_doc == 1 and is_target_class == 1:
            N11 += 1
        elif term_in_doc == 1 and is_target_class == 0:
            N10 += 1
        elif term_in_doc == 0 and is_target_class == 1:
            N01 += 1
        else: # term_in_doc == 0 and is_target_class == 0
            N00 += 1

    # محاسبه مقادیر جانبی
    N1_ = N10 + N11 # تعداد اسناد شامل واژه
    N0_ = N00 + N01 # تعداد اسناد بدون واژه
    N_1 = N01 + N11 # تعداد اسناد در کلاس هدف
    N_0 = N00 + N10 # تعداد اسناد در کلاس غیرهدف
```

```

# محاسبه اطلاعات متقابل با استفاده از فرمول
mi = 0.0
if N11 > 0:
    mi += (N11 / N) * np.log2((N * N11) / (N1_ * N_1))
if N01 > 0:
    mi += (N01 / N) * np.log2((N * N01) / (N0_ * N_1))
if N10 > 0:
    mi += (N10 / N) * np.log2((N * N10) / (N1_ * N_0))
if N00 > 0:
    mi += (N00 / N) * np.log2((N * N00) / (N0_ * N_0))

return mi

def select_features_mi(documents, labels, k, target_class):
    """
    ویژگی برتر با استفاده از اطلاعات متقابل  $k$  انتخاب

    واژه با بالاترین امتیاز را انتخاب می‌کند  $k$  را برای همه واژه‌ها محاسبه کرده و MI این تابع
    این همان الگوریتمی است که در شکل 13.6 کتاب توضیح داده شد.

    # استخراج واژگان
    vocab = set()
    for doc in documents:
        vocab.update(doc)

    # محاسبه اطلاعات متقابل برای هر واژه
    term_scores = {}
    for term in vocab:
        mi = calculate_mutual_information(documents, labels, term, target_class)
        term_scores[term] = mi

    # واژه با بالاترین امتیاز  $k$  انتخاب
    selected_features = sorted(term_scores.items(), key=lambda x: x[1], reverse=True)[:k]
    return [term for term, score in selected_features], [score for term, score in selected_features]

def create_document_representation(documents, selected_features):
    """
    ایجاد نمایش سند بر اساس ویژگی‌های منتخب
    برای مدل چندجمله‌ای: تعداد تکرار هر ویژگی
    برای مدل برنولی: وجود یا عدم وجود هر ویژگی

    # نمایش برای مدل چندجمله‌ای
    multinomial_rep = []
    for doc in documents:
        doc_rep = [doc.count(term) for term in selected_features]
        multinomial_rep.append(doc_rep)

    # نمایش برای مدل برنولی
    bernoulli_rep = []
    for doc in documents:
        doc_rep = [1 if term in doc else 0 for term in selected_features]
        bernoulli_rep.append(doc_rep)

    return np.array(multinomial_rep), np.array(bernoulli_rep)

def evaluate_feature_selection(documents, labels, target_class, feature_counts):
    """
    ارزیابی تأثیر انتخاب ویژگی بر دقت طبقه‌بندی

    این تابع دقت مدل‌های چندجمله‌ای و برنولی را برای تعداد مختلفی از ویژگی‌ها محاسبه می‌کند.
    نتایج نشان می‌دهد چگونه انتخاب ویژگی می‌تواند دقت را بهبود بخشد، همانطور که در شکل 13.8 کتاب نشان داده شد.

    # تقسیم داده‌ها به آموزش و آزمون
    X_train_docs, X_test_docs, y_train, y_test = train_test_split(
        documents, labels, test_size=0.3, random_state=42
    )

    # تبدیل برجسب‌ها به باینری (کلاس هدف در مقابل بقیه)
    y_train_binary = np.array([1 if label == target_class else 0 for label in y_train])
    y_test_binary = np.array([1 if label == target_class else 0 for label in y_test])

    # انتخاب ویژگی‌ها بر اساس داده‌های آموزشی
    vocab = set()
    for doc in X_train_docs:
        vocab.update(doc)

    # برای همه واژه‌ها MI محاسبه امتیاز
    term_scores = {}
    for term in vocab:
        mi = calculate_mutual_information(X_train_docs, y_train, term, target_class)
        term_scores[term] = mi

    # MI مرتب‌سازی واژه‌ها بر اساس امتیاز
    sorted_terms = sorted(term_scores.items(), key=lambda x: x[1], reverse=True)
    all_terms = [term for term, score in sorted_terms]

    # ارزیابی برای تعداد مختلفی از ویژگی‌ها
    multinomial_scores = []
    bernoulli_scores = []

    for k in feature_counts:
        # ویژگی برتر  $k$  انتخاب
        selected_features = all_terms[:k]

        # ایجاد نمایش سند برای مدل‌ها
        X_train_multi, X_train_bern = create_document_representation(X_train_docs, selected_features)
        X_test_multi, X_test_bern = create_document_representation(X_test_docs, selected_features)

        # آموزش و ارزیابی مدل چندجمله‌ای
        clf_multi = MultinomialNB()
        clf_multi.fit(X_train_multi, y_train_binary)
        y_pred_multi = clf_multi.predict(X_test_multi)
        f1_multi = f1_score(y_test_binary, y_pred_multi)

```

```

multinomial_scores.append(f1_multi)

# آموزش و ارزیابی مدل برنولی
clf_bern = BernoulliNB()
clf_bern.fit(X_train_bern, y_train_binary)
y_pred_bern = clf_bern.predict(X_test_bern)
f1_bern = f1_score(y_test_binary, y_pred_bern)
bernoulli_scores.append(f1_bern)

return multinomial_scores, bernoulli_scores

# ایجاد مجموعه داده نمونه (شبيه به مثال کتاب)
np.random.seed(42)

# کلاس‌ها
classes = ['poultry', 'coffee', 'sports', 'china', 'uk', 'elections']

# واژگان مرتبط با هر کلاس
class_vocab = {
    'poultry': ['poultry', 'chicken', 'meat', 'avian', 'bird', 'farm', 'egg', 'broiler', 'inspection', 'disease'],
    'coffee': ['coffee', 'bean', 'roast', 'brew', 'colombia', 'brazil', 'export', 'crop', 'harvest', 'arabica'],
    'sports': ['soccer', 'game', 'match', 'team', 'player', 'score', 'win', 'league', 'cup', 'championship'],
    'china': ['china', 'chinese', 'beijing', 'yuan', 'shanghai', 'hong', 'kong', 'xinhua', 'province', 'taiwan'],
    'uk': ['london', 'uk', 'british', 'britain', 'england', 'pound', 'pence', 'stg', 'plc', 'english'],
    'elections': ['election', 'vote', 'candidate', 'campaign', 'poll', 'party', 'democratic', 'republican', 'president', 'ballot']
}

# واژگان عمومی
general_vocab = ['report', 'said', 'year', 'today', 'new', 'government', 'market', 'company', 'people', 'world']

# ایجاد اسناد
documents = []
labels = []
num_docs_per_class = 100

for cls in classes:
    for i in range(num_docs_per_class):
        # هر سند شامل 5-15 واژه است
        doc_length = np.random.randint(5, 16)
        doc = []

        # اضافه کردن 2-4 واژه از کلاس
        num_class_terms = np.random.randint(2, 5)
        class_terms = np.random.choice(class_vocab[cls], num_class_terms, replace=False)
        doc.extend(class_terms)

        # اضافه کردن واژگان عمومی
        num_general_terms = doc_length - num_class_terms
        general_terms = np.random.choice(general_vocab, num_general_terms, replace=True)
        doc.extend(general_terms)

        # اضافه کردن برخی واژگان نویز (با احتمال کم)
        if np.random.random() < 0.1:
            noise_term = f"noise_term_{np.random.randint(100)}"
            doc.append(noise_term)

        documents.append(doc)
        labels.append(cls)

# export و واژه poultry محاسبه اطلاعات متقابل برای کلاس
mi_export = calculate_mutual_information(documents, labels, "export", "poultry")
print(f"export و poultry: {mi_export:.6f}")

# poultry ویژگی برتر برای کلاس
selected_features, scores = select_features_mi(documents, labels, 10, "poultry")
print("poultry ویژگی‌های منتخب برای کلاس:")
for feature, score in zip(selected_features, scores):
    print(f"{feature}: {score:.6f}")

# ارزیابی تأثیر انتخاب ویژگی بر دقت طبقه‌بندی
feature_counts = [1, 5, 10, 20, 50, 100]
multinomial_scores, bernoulli_scores = evaluate_feature_selection(documents, labels, "poultry", feature_counts)

# نمایش نتایج
plt.figure(figsize=(10, 6))
plt.plot(feature_counts, multinomial_scores, 'o-', label='Multinomial NB')
plt.plot(feature_counts, bernoulli_scores, 'x-', label='Bernoulli NB')
plt.xscale('log')
plt.xlabel('Number of Features')
plt.ylabel('F1 Score')
plt.title('Effect of Feature Selection on Classification Accuracy')
plt.legend()
plt.grid(True)
plt.show()

# نمایش نتایج عددی
print("\nنتایج ارزیابی:")
print(f"(برنولی) \tF1 (چندجمله‌ای) \t\tتعداد ویژگی")
for i, k in enumerate(feature_counts):
    print(f"{k} \t\t\t{multinomial_scores[i]:.4f} \t\t\t{bernoulli_scores[i]:.4f}")

```

export و poultry: 0.011678

poultry: ویژگی‌های منتخب برای کلاس:

avian: 0.170331

chicken: 0.165115

inspection: 0.149661

bird: 0.149661

meat: 0.144573

egg: 0.144573

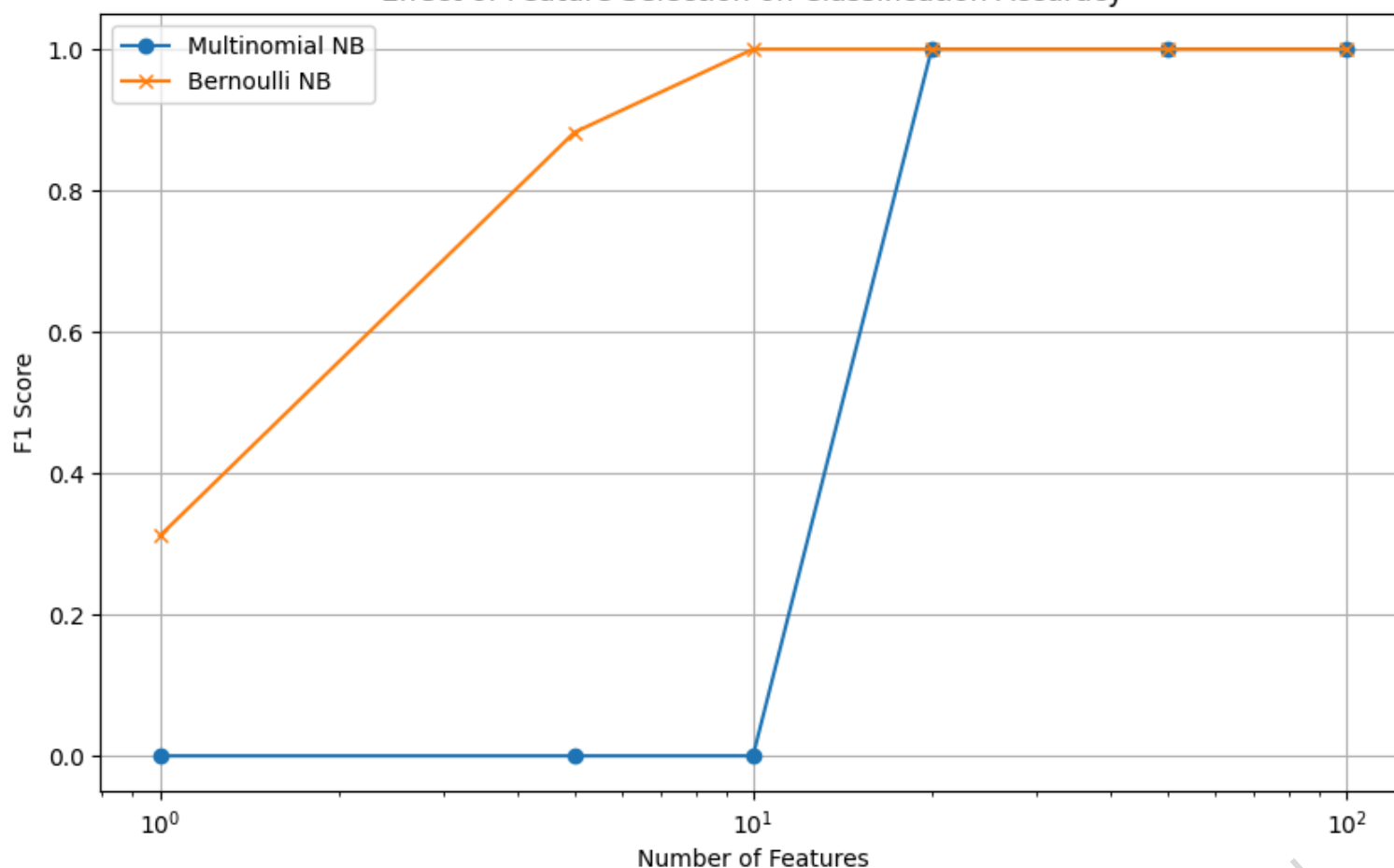
poultry: 0.129490

disease: 0.129490

farm: 0.129490

broiler: 0.119581

Effect of Feature Selection on Classification Accuracy



نتایج ارزیابی:

تعداد ویژگی

F1 (چندجمله‌ای)

F1 (برنولی)

1

0.0000

0.3111

5

0.0000

0.8824

10

0.0000

1.0000

20

1.0000

1.0000

50

1.0000

1.0000

100

1.0000

1.0000

13.5.2 آزمون χ^2 : چگونه کلمات مهم را از کلمات تصادفی تشخیص دهیم؟

🛒 آمازون و تشخیص محصولات محبوب: آیا "خرید" واقعاً نشان‌دهنده محبوبیت است؟

تصور کنید در آمازون کار می‌کنید و باید سیستم طراحی کنید که محصولات را به دسته‌بندی‌های مختلفی مانند "محبوب"، "متوسط" یا "نامحسوب" تقسیم کند. کلمه "خرید" در نظرات محصولات بسیار زیاد تکرار می‌شود، اما آیا این کلمه واقعاً به ما کمک می‌کند تا تشخیص دهیم یک محصول محبوب است یا خیر؟

آزمون χ^2 به ما کمک می‌کند تا بفهمیم آیا یک کلمه واقعاً با یک کلاس مرتبط است یا فقط به صورت تصادفی در اسناد ظاهر شده است. این آزمون فراوانی مشاهده شده یک کلمه در یک کلاس را با فراوانی مورد انتظار تحت فرض استقلال مقایسه می‌کند.

روش دیگر محبوب برای انتخاب ویژگی، استفاده از آزمون χ^2 است. در آمار، آزمون χ^2 برای بررسی استقلال دو رویداد به کار می‌رود. در انتخاب ویژگی، دو رویداد عبارتند از وقوع واژه و وقوع کلاس. ما واژه‌ها را بر اساس مقدار زیر رتبه‌بندی می‌کنیم:

$$\chi^2(D, t, c) = \sum_{e_t \in \{0,1\}} \sum_{e_c \in \{0,1\}} \frac{(N_{e_t e_c} - E_{e_t e_c})^2}{E_{e_t e_c}}$$

که در آن N فراوانی مشاهده شده و E فراوانی مورد انتظار است. برای مثال، E_{11} فراوانی مورد انتظار وقوع همزمان واژه t و کلاس c است با فرض استقلال آن‌ها.

12 از تئوری تا عمل: چگونه χ^2 را محاسبه کنیم؟

مثال 13.4: بیایید به مثال اخبار کشاورزی برگردیم و χ^2 را برای کلاس poultry (طیور) و واژه export (صادرات)

محاسبه کنیم. ابتدا E_{11} را برای داده‌های مثال 13.3 محاسبه می‌کنیم:

$$E_{11} = N \times P(t) \times P(c) = N \times (49+141)/N \times (49+27652)/N \approx 6.6$$

با محاسبه سایر مقادیر E و جایگذاری در معادله، به دست می‌آوریم: $\chi^2(D, t, c) \approx 284$

معنای آماری: چگونه χ^2 به ما کمک می‌کند؟

χ^2 معیاری از انحراف فراوانی‌های مورد انتظار (E) و مشاهده شده (N) از یکدیگر است. مقدار بالای χ^2 نشان می‌دهد که فرض استقلال، که به معنای شباهت فراوانی‌های مورد انتظار و مشاهده شده است، نادرست است.

در مثال ما، $\chi^2 \approx 284 > 10.83$ ، بنابراین می‌توانیم فرض استقلال poultry و export را با احتمال خطای 0.001 رد کنیم. این به ما می‌گوید که واژه "export" و کلاس "poultry" به هم وابسته هستند و این وابستگی تصادفی نیست.

کاربرد واقعی: در سیستم طبقه‌بندی نظرات مشتریان فیسبوک، کلمه "عالی" ممکن است در نظرات مثبت و منفی ظاهر شود. آزمون χ^2 به ما کمک می‌کند تا بفهمیم آیا این کلمه واقعاً با نظرات مثبت مرتبط است یا فقط به صورت تصادفی در هر دو نوع نظر ظاهر می‌شود.

یک روش ساده‌تر برای محاسبه χ^2 به صورت زیر است:

$$\chi^2(D, t, c) = \frac{(N_{11} + N_{10} + N_{01} + N_{00}) \times (N_{11}N_{00} - N_{10}N_{01})^2}{(N_{11} + N_{01}) \times (N_{11} + N_{10}) \times (N_{10} + N_{00}) \times (N_{01} + N_{00})}$$

جدول 13.6 مقادیر بحرانی توزیع χ^2 با یک درجه آزادی را نشان می‌دهد:

χ^2 مقدار بحرانی	p
2.71	0.1
3.84	0.05
6.63	0.01
10.83	0.001

⚠ محدودیت‌های آماری: آیا χ^2 همیشه دقیق است؟

از دیدگاه آماری، انتخاب ویژگی با χ^2 مشکل‌ساز است. برای یک آزمون با یک درجه آزادی، باید از اصلاح Yates استفاده شود که رسیدن به اهمیت آماری را دشوارتر می‌کند. همچنین، هرگاه یک آزمون آماری چندین بار استفاده شود، احتمال خطا افزایش می‌یابد.

با این حال، در طبقه‌بندی متن، به ندرت اهمیت دارد که چند واژه اضافی به مجموعه ویژگی اضافه یا از آن حذف شوند. مهم‌تر، اهمیت نسبی ویژگی‌ها است. تا زمانی که انتخاب ویژگی با χ^2 فقط ویژگی‌ها را بر اساس کاربردی رتبه‌بندی کند و برای بیان بیانیه‌هایی درباره وابستگی آماری متغیرها استفاده نشود، نباید بیش از حد نگران عدم پیروی دقیق از نظریه آماری باشیم.

مثال واقعی: در سیستم طبقه‌بندی اخبار گوگل، حتی اگر χ^2 برای برخی کلمات به دلیل محدودیت‌های آماری دقیق نباشد، سیستم همچنان می‌تواند با استفاده از رتبه‌بندی نسبی کلمات، اخبار را با دقت بالایی دسته‌بندی کند. در عمل، این کافی است که کلمات مرتبط‌تر بالاترین رتبه را داشته باشند، حتی اگر مقادیر دقیق χ^2 از نظر آماری کامل نباشند.

```

import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import f1_score

def calculate_chi2(documents, labels, term, target_class):
    """
    برای یک واژه و یک کلاس  $\chi^2$  محاسبه مقدار

    توضیح داده شد MD cell را پیاده‌سازی می‌کند که در  $\chi^2$  این تابع فرمول
    میزان انحراف فراوانی مشاهده شده از فراوانی مورد انتظار تحت فرض استقلال را اندازه‌گیری می‌کند  $\chi^2$ .

    پارامترها:
    documents: لیستی از اسناد، هر سند یک لیست از کلمات است
    labels: لیستی از برچسب‌های مربوط به هر سند
    term: واژه مورد نظر
    target_class: کلاس مورد نظر
    """
    N = len(documents) # تعداد کل اسناد

    # شمارش مقادیر N11, N10, N01, N00
    N11 = N10 = N01 = N00 = 0

    for doc, label in zip(documents, labels):
        term_in_doc = 1 if term in doc else 0
        is_target_class = 1 if label == target_class else 0

        if term_in_doc == 1 and is_target_class == 1:
            N11 += 1
        elif term_in_doc == 1 and is_target_class == 0:
            N10 += 1
        elif term_in_doc == 0 and is_target_class == 1:
            N01 += 1
        else: # term_in_doc == 0 and is_target_class == 0
            N00 += 1

    # با استفاده از فرمول ساده‌شده (معادل فرمول اصلی)  $\chi^2$  محاسبه
    numerator = (N11 + N10 + N01 + N00) * ((N11 * N00) - (N10 * N01))**2
    denominator = (N11 + N01) * (N11 + N10) * (N10 + N00) * (N01 + N00)

    if denominator == 0:
        return 0

    chi2 = numerator / denominator
    return chi2

def calculate_mutual_information(documents, labels, term, target_class):
    """
     $\chi^2$  محاسبه اطلاعات متقابل بین یک واژه و یک کلاس برای مقایسه با
    """
    N = len(documents)

    # شمارش مقادیر N11, N10, N01, N00
    N11 = N10 = N01 = N00 = 0

    for doc, label in zip(documents, labels):
        term_in_doc = 1 if term in doc else 0
        is_target_class = 1 if label == target_class else 0

        if term_in_doc == 1 and is_target_class == 1:
            N11 += 1
        elif term_in_doc == 1 and is_target_class == 0:
            N10 += 1
        elif term_in_doc == 0 and is_target_class == 1:
            N01 += 1
        else: # term_in_doc == 0 and is_target_class == 0
            N00 += 1

    # محاسبه مقادیر جانبی
    N1_ = N10 + N11 # تعداد اسناد شامل واژه
    N0_ = N00 + N01 # تعداد اسناد بدون واژه
    N_1 = N01 + N11 # تعداد اسناد در کلاس هدف
    N_0 = N00 + N10 # تعداد اسناد در کلاس غیرهدف

    # محاسبه اطلاعات متقابل
    mi = 0.0
    if N11 > 0:
        mi += (N11 / N) * np.log2((N * N11) / (N1_ * N_1))
    if N01 > 0:
        mi += (N01 / N) * np.log2((N * N01) / (N0_ * N_1))
    if N10 > 0:
        mi += (N10 / N) * np.log2((N * N10) / (N1_ * N_0))
    if N00 > 0:
        mi += (N00 / N) * np.log2((N * N00) / (N0_ * N_0))

    return mi

def select_features_chi2(documents, labels, k, target_class):
    """
     $\chi^2$  ویژگی برتر با استفاده از آزمون k انتخاب

    واژه با بالاترین امتیاز را انتخاب می‌کند k را برای همه واژه‌ها محاسبه کرده و  $\chi^2$  این تابع
    """
    # استخراج واژگان
    vocab = set()
    for doc in documents:
        vocab.update(doc)

    # برای هر واژه  $\chi^2$  محاسبه
    term_scores = {}
    for term in vocab:

```

```

chi2 = calculate_chi2(documents, labels, term, target_class)
term_scores[term] = chi2

# واژه با بالاترین امتیاز k انتخاب
selected_features = sorted(term_scores.items(), key=lambda x: x[1], reverse=True)[:k]
return [term for term, score in selected_features], [score for term, score in selected_features]

def select_features_mi(documents, labels, k, target_class):
    """
    ویژگی برتر با استفاده از اطلاعات متقابل برای مقایسه k انتخاب
    """
    # استخراج واژگان
    vocab = set()
    for doc in documents:
        vocab.update(doc)

    # محاسبه اطلاعات متقابل برای هر واژه
    term_scores = {}
    for term in vocab:
        mi = calculate_mutual_information(documents, labels, term, target_class)
        term_scores[term] = mi

    # واژه با بالاترین امتیاز k انتخاب
    selected_features = sorted(term_scores.items(), key=lambda x: x[1], reverse=True)[:k]
    return [term for term, score in selected_features], [score for term, score in selected_features]

def create_document_representation(documents, selected_features):
    """
    ایجاد نمایش سند بر اساس ویژگی‌های منتخب
    برای مدل چندجمله‌ای: تعداد تکرار هر ویژگی
    برای مدل برنولی: وجود یا عدم وجود هر ویژگی
    """
    # نمایش برای مدل چندجمله‌ای
    multinomial_rep = []
    for doc in documents:
        doc_rep = [doc.count(term) for term in selected_features]
        multinomial_rep.append(doc_rep)

    # نمایش برای مدل برنولی
    bernoulli_rep = []
    for doc in documents:
        doc_rep = [1 if term in doc else 0 for term in selected_features]
        bernoulli_rep.append(doc_rep)

    return np.array(multinomial_rep), np.array(bernoulli_rep)

def compare_feature_selection_methods(documents, labels, target_class, k=10):
    """
     $\chi^2$  و MI مقایسه روش‌های انتخاب ویژگی

    این تابع نشان می‌دهد که چگونه دو روش مختلف می‌توانند ویژگی‌های متفاوتی را انتخاب کنند.
    """
    #  $\chi^2$  انتخاب ویژگی با
    chi2_features, chi2_scores = select_features_chi2(documents, labels, k, target_class)

    # MI انتخاب ویژگی با
    mi_features, mi_scores = select_features_mi(documents, labels, k, target_class)

    # نمایش نتایج
    print(f"Top {k} features selected by  $\chi^2$  for class '{target_class}':")
    for feature, score in zip(chi2_features, chi2_scores):
        print(f"{feature}: {score:.2f}")

    print(f"\nTop {k} features selected by MI for class '{target_class}':")
    for feature, score in zip(mi_features, mi_scores):
        print(f"{feature}: {score:.6f}")

    # مقایسه ویژگی‌های مشترک
    common_features = set(chi2_features) & set(mi_features)
    print(f"\nCommon features selected by both methods: {list(common_features)}")

    return chi2_features, mi_features

def evaluate_feature_selection_methods(documents, labels, target_class, feature_counts):
    """
    ارزیابی تأثیر روش‌های مختلف انتخاب ویژگی بر دقت طبقه‌بندی

    این تابع دقت مدل‌های چندجمله‌ای و برنولی را برای تعداد مختلفی از ویژگی‌ها محاسبه می‌کند.
    نتایج نشان می‌دهد چگونه روش‌های مختلف انتخاب ویژگی می‌توانند دقت را بهبود بخشند.
    """
    # تقسیم داده‌ها به آموزش و آزمون
    X_train_docs, X_test_docs, y_train, y_test = train_test_split(
        documents, labels, test_size=0.3, random_state=42
    )

    # تبدیل برجسب‌ها به باینری (کلاس هدف در مقابل بقیه)
    y_train_binary = np.array([1 if label == target_class else 0 for label in y_train])
    y_test_binary = np.array([1 if label == target_class else 0 for label in y_test])

    # انتخاب ویژگی‌ها بر اساس داده‌های آموزشی
    vocab = set()
    for doc in X_train_docs:
        vocab.update(doc)

    # برای همه واژه‌ها  $\chi^2$  محاسبه امتیاز
    chi2_scores = {}
    for term in vocab:
        chi2_scores[term] = calculate_chi2(X_train_docs, y_train, term, target_class)

    # برای همه واژه‌ها MI محاسبه امتیاز
    mi_scores = {}
    for term in vocab:
        mi_scores[term] = calculate_mutual_information(X_train_docs, y_train, term, target_class)

```

```

# مرتب‌سازی واژه‌ها بر اساس امتیازها
chi2_sorted_terms = sorted(chi2_scores.items(), key=lambda x: x[1], reverse=True)
mi_sorted_terms = sorted(mi_scores.items(), key=lambda x: x[1], reverse=True)

chi2_terms = [term for term, score in chi2_sorted_terms]
mi_terms = [term for term, score in mi_sorted_terms]

# ارزیابی برای تعداد مختلفی از ویژگی‌ها
chi2_multinomial_scores = []
chi2_bernoulli_scores = []
mi_multinomial_scores = []
mi_bernoulli_scores = []

for k in feature_counts:
    # ویژگی برتر با k انتخاب
    chi2_selected_features = chi2_terms[:k]

    # ایجاد نمایش سند برای مدل‌ها
    X_train_multi, X_train_bern = create_document_representation(X_train_docs, chi2_selected_features)
    X_test_multi, X_test_bern = create_document_representation(X_test_docs, chi2_selected_features)

    # آموزش و ارزیابی مدل چندجمله‌ای با  $\chi^2$ 
    clf_multi = MultinomialNB()
    clf_multi.fit(X_train_multi, y_train_binary)
    y_pred_multi = clf_multi.predict(X_test_multi)
    f1_multi = f1_score(y_test_binary, y_pred_multi)
    chi2_multinomial_scores.append(f1_multi)

    # آموزش و ارزیابی مدل برنولی با  $\chi^2$ 
    clf_bern = BernoulliNB()
    clf_bern.fit(X_train_bern, y_train_binary)
    y_pred_bern = clf_bern.predict(X_test_bern)
    f1_bern = f1_score(y_test_binary, y_pred_bern)
    chi2_bernoulli_scores.append(f1_bern)

    # MI ویژگی برتر با k انتخاب
    mi_selected_features = mi_terms[:k]

    # ایجاد نمایش سند برای مدل‌ها
    X_train_multi, X_train_bern = create_document_representation(X_train_docs, mi_selected_features)
    X_test_multi, X_test_bern = create_document_representation(X_test_docs, mi_selected_features)

    # MI آموزش و ارزیابی مدل چندجمله‌ای با
    clf_multi = MultinomialNB()
    clf_multi.fit(X_train_multi, y_train_binary)
    y_pred_multi = clf_multi.predict(X_test_multi)
    f1_multi = f1_score(y_test_binary, y_pred_multi)
    mi_multinomial_scores.append(f1_multi)

    # MI آموزش و ارزیابی مدل برنولی با
    clf_bern = BernoulliNB()
    clf_bern.fit(X_train_bern, y_train_binary)
    y_pred_bern = clf_bern.predict(X_test_bern)
    f1_bern = f1_score(y_test_binary, y_pred_bern)
    mi_bernoulli_scores.append(f1_bern)

return (chi2_multinomial_scores, chi2_bernoulli_scores,
        mi_multinomial_scores, mi_bernoulli_scores)

# ایجاد مجموعه داده نمونه (شبیه به مثال کتاب)
np.random.seed(42)

# کلاس‌ها
classes = ['poultry', 'coffee', 'sports', 'china', 'uk', 'elections']

# واژگان مرتبط با هر کلاس
class_vocab = {
    'poultry': ['poultry', 'chicken', 'meat', 'avian', 'bird', 'farm', 'egg', 'broiler', 'inspection', 'disease'],
    'coffee': ['coffee', 'bean', 'roast', 'brew', 'colombia', 'brazil', 'export', 'crop', 'harvest', 'arabica'],
    'sports': ['soccer', 'game', 'match', 'team', 'player', 'score', 'win', 'league', 'cup', 'championship'],
    'china': ['china', 'chinese', 'beijing', 'yuan', 'shanghai', 'hong', 'kong', 'xinhua', 'province', 'taiwan'],
    'uk': ['london', 'uk', 'british', 'britain', 'england', 'pound', 'pence', 'stg', 'plc', 'english'],
    'elections': ['election', 'vote', 'candidate', 'campaign', 'poll', 'party', 'democratic', 'republican', 'president', 'ballot']
}

# واژگان عمومی
general_vocab = ['report', 'said', 'year', 'today', 'new', 'government', 'market', 'company', 'people', 'world']

# ایجاد اسناد
documents = []
labels = []
num_docs_per_class = 100

for cls in classes:
    for i in range(num_docs_per_class):
        # هر سند شامل 5-15 واژه است
        doc_length = np.random.randint(5, 16)
        doc = []

        # اضافه کردن 2-4 واژه از کلاس
        num_class_terms = np.random.randint(2, 5)
        class_terms = np.random.choice(class_vocab[cls], num_class_terms, replace=False)
        doc.extend(class_terms)

        # اضافه کردن واژگان عمومی
        num_general_terms = doc_length - num_class_terms
        general_terms = np.random.choice(general_vocab, num_general_terms, replace=True)
        doc.extend(general_terms)

        # اضافه کردن برخی واژگان نویز (با احتمال کم)
        if np.random.random() < 0.1:
            noise_term = f"noise_term_{np.random.randint(100)}"

```

```

doc.append(noise_term)

documents.append(doc)
labels.append(cls)

# export و واژه poultry برای کلاس  $\chi^2$  مثال از کتاب: محاسبه
chi2_export = calculate_chi2(documents, labels, "export", "poultry")
print(f"مقدار  $\chi^2$  برای واژه 'export' و کلاس 'poultry': {chi2_export:.2f}")

# MI و  $\chi^2$  مقایسه روش‌های انتخاب ویژگی
chi2_features, mi_features = compare_feature_selection_methods(documents, labels, "poultry", k=10)

# ارزیابی تأثیر روش‌های مختلف انتخاب ویژگی بر دقت طبقه‌بندی
feature_counts = [1, 5, 10, 20, 50, 100]
(chi2_multi_scores, chi2_bernoulli_scores,
 mi_multi_scores, mi_bernoulli_scores) = evaluate_feature_selection_methods(
    documents, labels, "poultry", feature_counts)

# نمایش نتایج
plt.figure(figsize=(12, 8))
plt.plot(feature_counts, chi2_multi_scores, 'o-', label='Multinomial NB with  $\chi^2$ ')
plt.plot(feature_counts, chi2_bernoulli_scores, 'x-', label='Bernoulli NB with  $\chi^2$ ')
plt.plot(feature_counts, mi_multi_scores, 's-', label='Multinomial NB with MI')
plt.plot(feature_counts, mi_bernoulli_scores, 'd-', label='Bernoulli NB with MI')
plt.xscale('log')
plt.xlabel('Number of Features')
plt.ylabel('F1 Score')
plt.title('Comparison of Feature Selection Methods')
plt.legend()
plt.grid(True)
plt.ylim(-0.05, 1.05)
plt.show()

# نمایش نتایج عددی
print("\nنتایج ارزیابی:")
print("MI (برنولی) \tF1 (چندجمله‌ای) \tMI (برنولی) \tF1 ( $\chi^2$ ) \tF1 (تعداد ویژگی)")
for i, k in enumerate(feature_counts):
    print(f"{k} \t\t{chi2_multi_scores[i]:.4f} \t\t{chi2_bernoulli_scores[i]:.4f} \t\t{mi_multi_scores[i]:.4f} \t\t{mi_bernoulli_scores[i]:.4f}")

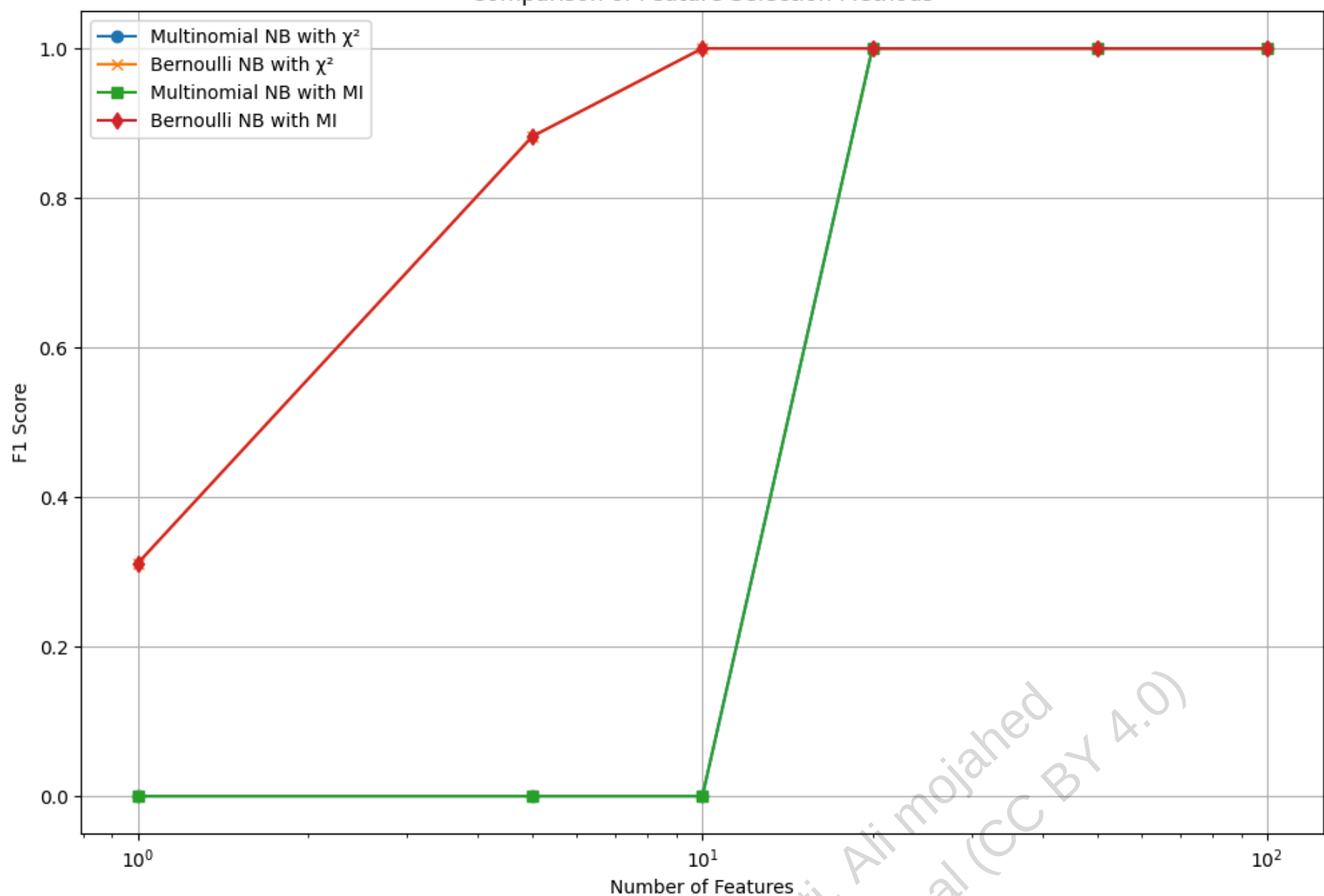
```

مقدار χ^2 برای واژه 'export' و کلاس 'poultry': 5.44
 Top 10 features selected by χ^2 for class 'poultry':
 avian: 191.49
 chicken: 185.84
 bird: 169.01
 inspection: 169.01
 meat: 163.44
 egg: 163.44
 farm: 146.85
 disease: 146.85
 poultry: 146.85
 broiler: 135.89

Top 10 features selected by MI for class 'poultry':
 avian: 0.170331
 chicken: 0.165115
 bird: 0.149661
 inspection: 0.149661
 meat: 0.144573
 egg: 0.144573
 farm: 0.129490
 disease: 0.129490
 poultry: 0.129490
 broiler: 0.119581

Common features selected by both methods: [np.str_('avian'), np.str_('farm'), np.str_('egg'), np.str_('meat'), np.str_('bird'), np.str_('chicken'), np.str_('disease'), np.str_('inspection'), np.str_('poultry'), np.str_('broiler')]

Comparison of Feature Selection Methods



نتایج ارزیابی:

تعداد ویژگی

	F1 (χ^2 چندجمله‌ای)	F1 (χ^2 برنولی)	F1 (MI چندجمله‌ای)	F1 (MI برنولی)
1	0.0000	0.3111	0.0000	0.3111
5	0.0000	0.8824	0.0000	0.8824
10	0.0000	1.0000	0.0000	1.0000
20	1.0000	1.0000	1.0000	1.0000
50	1.0000	1.0000	1.0000	1.0000
100	1.0000	1.0000	1.0000	1.0000

13.5.3 انتخاب ویژگی مبتنی بر فراوانی: ساده‌ترین روش، مؤثرترین؟

نتفلیکس و تحلیل نظرات کاربران: چگونه فیلم‌ها را دسته‌بندی کنیم؟

تصور کنید در تیم تحلیل داده‌های نتفلیکس کار می‌کنید و باید سیستم طراحی کنید که نظرات کاربران را به ژانرهای مختلف فیلم (اکشن، کمدی، درام، علمی-تخیلی) دسته‌بندی کند. یک رویکرد ساده این است که کلماتی را پیدا کنید که بیشترین تکرار را در نظرات هر ژانر دارند.

برای مثال، در نظرات فیلم‌های اکشن، کلماتی مانند "اکشن"، "هیجان‌انگیز"، "تعقیب" و "آتش" ممکن است بیشتر تکرار شوند. این روش ساده، انتخاب ویژگی مبتنی بر فراوانی نام دارد.

روش سوم برای انتخاب ویژگی، **انتخاب ویژگی مبتنی بر فراوانی** است، یعنی انتخاب واژه‌هایی که در یک کلاس بیشترین فراوانی را دارند. این ساده‌ترین روش انتخاب ویژگی است، اما همیشه مؤثرترین نیست. فراوانی می‌تواند به دو صورت تعریف شود:

- **فراوانی سندی** (document frequency): تعداد اسناد در کلاس c که شامل واژه t هستند (مناسب برای مدل برنولی)
- **فراوانی مجموعه** (collection frequency): تعداد توکن‌های t که در اسناد کلاس c رخ می‌دهند (مناسب برای مدل چندجمله‌ای)

⚠ **مزایا و معایب: چرا ساده‌ترین همیشه بهترین نیست؟**

انتخاب ویژگی مبتنی بر فراوانی ممکن است برخی واژه‌های پرتکرار را انتخاب کند که اطلاعات خاصی درباره کلاس ندارند، مانند روزهای هفته (Monday, Tuesday, ...) که در متن‌های خبری در همه کلاس‌ها پرتکرار هستند.

مثال واقعی: در سیستم تحلیل نظرات آمازون، کلماتی مانند "محصول"، "خرید" و "ارسال" در نظرات همه دسته‌ها (کتاب، الکترونیک، لباس) پرتکرار هستند، اما هیچ اطلاعات خاصی درباره دسته محصول به ما نمی‌دهند. این کلمات "نویز" می‌توانند دقت طبقه‌بندی را کاهش دهند.

🏳️ چه زمانی از این روش استفاده کنیم؟

وقتی هزاران ویژگی انتخاب می‌شوند، انتخاب ویژگی مبتنی بر فراوانی اغلب خوب عمل می‌کند. بنابراین، اگر دقت کمی زیربهبینه قابل قبول باشد، انتخاب ویژگی مبتنی بر فراوانی می‌تواند جایگزین خوبی برای روش‌های پیچیده‌تر باشد.

کاربرد واقعی: در سیستم تحلیل توییت‌ر، وقتی میلیون‌ها توئیت باید به سرعت دسته‌بندی شوند، استفاده از روش مبتنی بر فراوانی می‌تواند یک راه حل سریع و کارآمد باشد، حتی اگر دقت کمی کمتر از روش‌های پیچیده‌تر باشد.

🔍 مقایسه با روش‌های دیگر

با این حال، شکل 13.8 موردی است که انتخاب ویژگی مبتنی بر فراوانی بسیار بدتر از MI و χ^2 عمل می‌کند و نباید استفاده شود. این نشان می‌دهد که سادگی همیشه بهترین راه حل نیست.

مثال واقعی: در سیستم طبقه‌بندی مقالات علمی گوگل اسکالر، روش مبتنی بر فراوانی ممکن است کلمات عمومی مانند "تحقیق"، "نتایج" و "تحلیل" را انتخاب کند که در همه رشته‌ها پرتکرار هستند، در حالی که روش‌های MI و χ^2 می‌توانند کلمات تخصصی‌تری را شناسایی کنند که تمایز بهتری بین رشته‌ها ایجاد می‌کنند.

In [6]: *Frequency-based Feature Selection (پایاده‌سازی انتخاب ویژگی مبتنی بر فراوانی)* #

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import f1_score

def calculate_document_frequency(documents, labels, target_class):
    """
    محاسبه فراوانی سندی برای هر واژه در کلاس هدف

    هستند t که شامل واژه c فراوانی سندی: تعداد اسناد در کلاس
    .این روش برای مدل برنولی مناسب است

    پارامترها:
    documents: (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    target_class: کلاسی که می‌خواهیم فراوانی واژه‌ها را برای آن محاسبه کنیم

    خروجی: دیکشنری که کلیدها واژه‌ها و مقادیر فراوانی سندی آن‌هاست
    """
    # فیلتر کردن اسناد مربوط به کلاس هدف
    target_docs = [doc for doc, label in zip(documents, labels) if label == target_class]

    # شمارش تعداد اسنادی که هر واژه در آن‌ها وجود دارد
    doc_freq = {}
    for doc in target_docs:
        unique_words = set(doc) # هر سند شمرش شود
        for word in unique_words:
            doc_freq[word] = doc_freq.get(word, 0) + 1

    return doc_freq

def calculate_collection_frequency(documents, labels, target_class):
    """
    محاسبه فراوانی مجموعه برای هر واژه در کلاس هدف

    رخ می‌دهند c که در اسناد کلاس t فراوانی مجموعه: تعداد کل توکن‌های
    .این روش برای مدل چندجمله‌ای مناسب است

    پارامترها:
```

```
documents: لیستی از اسناد (هر سند لیستی از کلمات است):
labels: لیستی از برچسب‌های مربوط به اسناد
target_class: کلاسی که می‌خواهیم فراوانی واژه‌ها را برای آن محاسبه کنیم

خروجی: دیکشنری که کلیدها واژه‌ها و مقادیر فراوانی مجموعه آن‌هاست
"""

# فیلتر کردن اسناد مربوط به کلاس هدف
target_docs = [doc for doc, label in zip(documents, labels) if label == target_class]

# شمارش تعداد کل تکرار هر واژه در تمام اسناد کلاس هدف
all_words = [word for doc in target_docs for word in doc]
coll_freq = Counter(all_words)

return dict(coll_freq)

def select_features_by_frequency(documents, labels, k, target_class, frequency_type='document'):
    """
    ویژگی برتر بر اساس فراوانی k انتخاب

    پارامترها:
    documents: لیستی از اسناد (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    k: تعداد ویژگی‌هایی که می‌خواهیم انتخاب کنیم
    target_class: کلاس هدف برای انتخاب ویژگی
    frequency_type: (برای فراوانی مجموعه 'collection'، برای فراوانی سندی 'document') نوع فراوانی

    واژه با بالاترین فراوانی k خروجی: لیستی از
    """

    # محاسبه فراوانی بر اساس نوع درخواستی
    if frequency_type == 'document':
        term_frequencies = calculate_document_frequency(documents, labels, target_class)
    elif frequency_type == 'collection':
        term_frequencies = calculate_collection_frequency(documents, labels, target_class)
    else:
        raise ValueError("باید 'collection' یا 'document' نوع فراوانی باشد")

    # واژه برتر k مرتب‌سازی واژه‌ها بر اساس فراوانی و انتخاب
    sorted_terms = sorted(term_frequencies.items(), key=lambda x: x[1], reverse=True)
    selected_features = [term for term, freq in sorted_terms[:k]]

    return selected_features, [freq for term, freq in sorted_terms[:k]]

def compare_feature_selection_methods(documents, labels, target_class, k=10):
    """
    مقایسه روش‌های انتخاب ویژگی مبتنی بر فراوانی سندی و مجموعه

    این تابع نشان می‌دهد که چگونه دو نوع فراوانی می‌توانند ویژگی‌های متفاوتی را انتخاب کنند
    """

    # انتخاب ویژگی با فراوانی سندی
    doc_features, doc_scores = select_features_by_frequency(documents, labels, k, target_class, 'document')

    # انتخاب ویژگی با فراوانی مجموعه
    coll_features, coll_scores = select_features_by_frequency(documents, labels, k, target_class, 'collection')

    # نمایش نتایج
    print(f"Top {k} features selected by document frequency for class '{target_class}':")
    for feature, score in zip(doc_features, doc_scores):
        print(f"{feature}: {score}")

    print(f"\nTop {k} features selected by collection frequency for class '{target_class}':")
    for feature, score in zip(coll_features, coll_scores):
        print(f"{feature}: {score}")

    # مقایسه ویژگی‌های مشترک
    common_features = set(doc_features) & set(coll_features)
    print(f"\nCommon features selected by both methods: {list(common_features)}")

    return doc_features, coll_features

def create_document_representation(documents, selected_features):
    """
    ایجاد نمایش سند بر اساس ویژگی‌های منتخب
    برای مدل چندجمله‌ای: تعداد تکرار هر ویژگی
    برای مدل برنولی: وجود یا عدم وجود هر ویژگی
    """

    # نمایش برای مدل چندجمله‌ای
    multinomial_rep = []
    for doc in documents:
        doc_rep = [doc.count(term) for term in selected_features]
        multinomial_rep.append(doc_rep)

    # نمایش برای مدل برنولی
    bernoulli_rep = []
    for doc in documents:
        doc_rep = [1 if term in doc else 0 for term in selected_features]
        bernoulli_rep.append(doc_rep)

    return np.array(multinomial_rep), np.array(bernoulli_rep)

def evaluate_frequency_feature_selection(documents, labels, target_class, feature_counts):
    """
    ارزیابی تأثیر انتخاب ویژگی مبتنی بر فراوانی بر دقت طبقه‌بندی

    این تابع دقت مدل‌های چندجمله‌ای و برنولی را برای تعداد مختلفی از ویژگی‌ها محاسبه می‌کند.
    نتایج نشان می‌دهد چگونه انتخاب ویژگی مبتنی بر فراوانی می‌تواند دقت را بهبود بخشد
    """

    # تقسیم داده‌ها به آموزش و آزمون
    X_train_docs, X_test_docs, y_train, y_test = train_test_split(
        documents, labels, test_size=0.3, random_state=42
    )

    # تبدیل برچسب‌ها به باینری (کلاس هدف در مقابل بقیه)
    y_train_binary = np.array([1 if label == target_class else 0 for label in y_train])
```

```
y_test_binary = np.array([1 if label == target_class else 0 for label in y_test])

# استخراج واژگان از داده‌های آموزشی
vocab = set()
for doc in X_train_docs:
    vocab.update(doc)

# محاسبه فراوانی سندی برای همه واژه‌ها
doc_freq_scores = {}
for term in vocab:
    doc_freq_scores[term] = calculate_document_frequency(X_train_docs, y_train, target_class).get(term, 0)

# محاسبه فراوانی مجموعه برای همه واژه‌ها
coll_freq_scores = {}
for term in vocab:
    coll_freq_scores[term] = calculate_collection_frequency(X_train_docs, y_train, target_class).get(term, 0)

# مرتب‌سازی واژه‌ها بر اساس فراوانی‌ها
doc_sorted_terms = sorted(doc_freq_scores.items(), key=lambda x: x[1], reverse=True)
coll_sorted_terms = sorted(coll_freq_scores.items(), key=lambda x: x[1], reverse=True)

doc_terms = [term for term, score in doc_sorted_terms]
coll_terms = [term for term, score in coll_sorted_terms]

# ارزیابی برای تعداد مختلفی از ویژگی‌ها
doc_multinomial_scores = []
doc_bernoulli_scores = []
coll_multinomial_scores = []
coll_bernoulli_scores = []

for k in feature_counts:
    # ویژگی برتر با فراوانی سندی k انتخاب
    doc_selected_features = doc_terms[:k]

    # ایجاد نمایش سند برای مدل‌ها
    X_train_multi, X_train_bern = create_document_representation(X_train_docs, doc_selected_features)
    X_test_multi, X_test_bern = create_document_representation(X_test_docs, doc_selected_features)

    # آموزش و ارزیابی مدل چندجمله‌ای با فراوانی سندی
    clf_multi = MultinomialNB()
    clf_multi.fit(X_train_multi, y_train_binary)
    y_pred_multi = clf_multi.predict(X_test_multi)
    f1_multi = f1_score(y_test_binary, y_pred_multi)
    doc_multinomial_scores.append(f1_multi)

    # آموزش و ارزیابی مدل برنولی با فراوانی سندی
    clf_bern = BernoulliNB()
    clf_bern.fit(X_train_bern, y_train_binary)
    y_pred_bern = clf_bern.predict(X_test_bern)
    f1_bern = f1_score(y_test_binary, y_pred_bern)
    doc_bernoulli_scores.append(f1_bern)

    # ویژگی برتر با فراوانی مجموعه k انتخاب
    coll_selected_features = coll_terms[:k]

    # ایجاد نمایش سند برای مدل‌ها
    X_train_multi, X_train_bern = create_document_representation(X_train_docs, coll_selected_features)
    X_test_multi, X_test_bern = create_document_representation(X_test_docs, coll_selected_features)

    # آموزش و ارزیابی مدل چندجمله‌ای با فراوانی مجموعه
    clf_multi = MultinomialNB()
    clf_multi.fit(X_train_multi, y_train_binary)
    y_pred_multi = clf_multi.predict(X_test_multi)
    f1_multi = f1_score(y_test_binary, y_pred_multi)
    coll_multinomial_scores.append(f1_multi)

    # آموزش و ارزیابی مدل برنولی با فراوانی مجموعه
    clf_bern = BernoulliNB()
    clf_bern.fit(X_train_bern, y_train_binary)
    y_pred_bern = clf_bern.predict(X_test_bern)
    f1_bern = f1_score(y_test_binary, y_pred_bern)
    coll_bernoulli_scores.append(f1_bern)

    return (doc_multinomial_scores, doc_bernoulli_scores,
            coll_multinomial_scores, coll_bernoulli_scores)

# ایجاد مجموعه داده نمونه
np.random.seed(42)

# کلاس‌ها
classes = ['poultry', 'coffee', 'sports', 'china', 'uk', 'elections']

# واژگان مرتبط با هر کلاس
class_vocab = {
    'poultry': ['poultry', 'chicken', 'meat', 'avian', 'bird', 'farm', 'egg', 'broiler', 'inspection', 'disease'],
    'coffee': ['coffee', 'bean', 'roast', 'brew', 'colombia', 'brazil', 'export', 'crop', 'harvest', 'arabica'],
    'sports': ['soccer', 'game', 'match', 'team', 'player', 'score', 'win', 'league', 'cup', 'championship'],
    'china': ['china', 'chinese', 'beijing', 'yuan', 'shanghai', 'hong', 'kong', 'xinhua', 'province', 'taiwan'],
    'uk': ['london', 'uk', 'british', 'britain', 'england', 'pound', 'pence', 'stg', 'plc', 'english'],
    'elections': ['election', 'vote', 'candidate', 'campaign', 'poll', 'party', 'democratic', 'republican', 'president', 'ballot']
}

# واژگان عمومی (که در همه کلاس‌ها پرتکرار هستند)
general_vocab = ['report', 'said', 'year', 'today', 'new', 'government', 'market', 'company', 'people', 'world',
                 'monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday', 'sunday']

# ایجاد اسناد
documents = []
labels = []
num_docs_per_class = 100

for cls in classes:
    for i in range(num_docs_per_class):
```

```

هر سند شامل 5-15 واژه است
doc_length = np.random.randint(5, 16)
doc = []

# اضافه کردن 2-4 واژه از کلاس
num_class_terms = np.random.randint(2, 5)
class_terms = np.random.choice(class_vocab[cls], num_class_terms, replace=False)
doc.extend(class_terms)

# اضافه کردن واژگان عمومی
num_general_terms = doc_length - num_class_terms
general_terms = np.random.choice(general_vocab, num_general_terms, replace=True)
doc.extend(general_terms)

documents.append(doc)
labels.append(cls)

# مقایسه روش‌های انتخاب ویژگی مبتنی بر فراوانی سندی و مجموعه‌ای
features, coll_features = compare_feature_selection_methods(documents, labels, "poultry", k=10)

# ارزیابی تأثیر انتخاب ویژگی مبتنی بر فراوانی بر دقت طبقه‌بندی
feature_counts, doc_bernoulli_scores, coll_bernoulli_scores = evaluate_frequency_feature_selection(
    documents, labels, "poultry", feature_counts)

# نمایش نتایج
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.plot(feature_counts, doc_multi_scores, 'o-', label='Multinomial NB with Document Frequency')
plt.plot(feature_counts, doc_bernoulli_scores, 'x-', label='Bernoulli NB with Document Frequency')
plt.subplot(2, 2, 2)
plt.plot(feature_counts, coll_multi_scores, 's-', label='Multinomial NB with Collection Frequency')
plt.plot(feature_counts, coll_bernoulli_scores, 'd-', label='Bernoulli NB with Collection Frequency')
plt.legend()
plt.title('Effect of Frequency-based Feature Selection on Classification Accuracy')
plt.show()

# نمایش نتایج
print("نتایج ارزیابی:")
for cls in enumerate(class_vocab):
    print(f"برونولی با فراوانی مجموعه (tf1) \t چندجمله‌ای با فراوانی مجموعه (tf1) \t برونولی با فراوانی سندی (tf1) \t چندجمله‌ای با فراوانی سندی (tf1) \t تعداد ویژگی‌ها")
    for feature in enumerate(doc_features[:10]):
        print(f"{i+1}. {feature}")

# نمایش ویژگی‌های منتخب با فراوانی سندی و مجموعه‌ای
print("پولتری ویژگی‌های منتخب با فراوانی سندی برای کلاس:")
for feature in enumerate(doc_features[:10]):
    print(f"{i+1}. {feature}")

# نمایش ویژگی‌های منتخب با فراوانی مجموعه‌ای برای کلاس
print("پولتری ویژگی‌های منتخب با فراوانی مجموعه برای کلاس:")
for feature in enumerate(coll_features[:10]):
    print(f"{i+1}. {feature}")

# ویژگی‌های منتخب با فراوانی سندی و مجموعه‌ای برای کلاس 'poultry'
features_selected_by_doc_freq = features_selected_by_doc_freq[cls]
print("Features selected by document frequency for class 'poultry':")
for i, feature in enumerate(features_selected_by_doc_freq):
    print(f"{i+1}. {feature}")

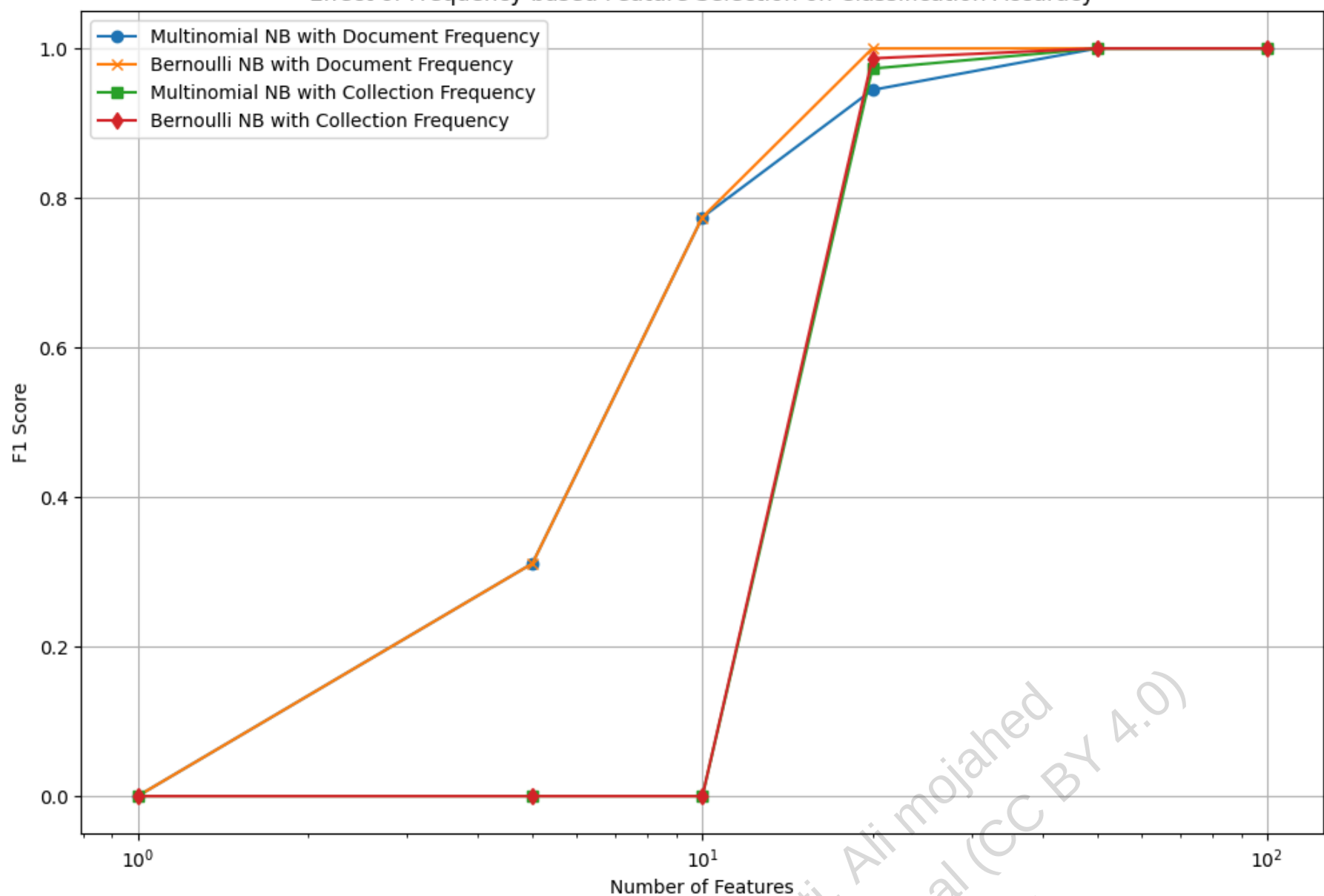
# ویژگی‌های منتخب با فراوانی مجموعه‌ای برای کلاس 'poultry'
features_selected_by_coll_freq = features_selected_by_coll_freq[cls]
print("Features selected by collection frequency for class 'poultry':")
for i, feature in enumerate(features_selected_by_coll_freq):
    print(f"{i+1}. {feature}")

```

```
report: 42
disease: 40
company: 39
people: 39
new: 37
year: 36
wednesday: 35
sunday: 34
tuesday: 32
bird: 32
```

```
report: 54
people: 49
company: 48
wednesday: 48
year: 45
tuesday: 45
new: 45
sunday: 45
disease: 40
market: 38
```


Effect of Frequency-based Feature Selection on Classification Accuracy



نتایج ارزیابی:

تعداد ویژگی	F1 (چندجمله‌ای با فراوانی سندی)	F1 (برنولی با فراوانی سندی)	F1 (چندجمله‌ای با فراوانی مجموعه)	F1 (برنولی با فراوانی مجموعه)
1	0.0000	0.0000	0.0000	0.0000
5	0.3111	0.3111	0.0000	0.0000
10	0.7742	0.7742	0.0000	0.0000
20	0.9444	1.0000	0.9730	0.9867
50	1.0000	1.0000	1.0000	1.0000
100	1.0000	1.0000	1.0000	1.0000

poultry: ویژگی‌های منتخب با فراوانی سندی برای کلاس

- report
- disease
- company
- people
- new
- year
- wednesday
- sunday
- tuesday
- bird

poultry: ویژگی‌های منتخب با فراوانی مجموعه برای کلاس

- report
- people
- company
- wednesday
- year
- tuesday
- new
- sunday
- disease
- market

13.5.4 انتخاب ویژگی برای چندین طبقه‌بند: چگونه بین دقت و کارایی تعادل برقرار کنیم؟

🌐 گوگل و مدیریت میلیاردها سند: چگونه همه چیز را با هم طبقه‌بندی کنیم؟

تصور کنید در گوگل کار می‌کنید و مسئول طراحی سیستمی هستید که باید روزانه میلیاردها سند (ایمیل‌ها، اسناد گوگل داک، عکس‌ها، ویدیوها و صفحات وب) را به دسته‌های مختلفی اختصاص دهد. هر نوع محتوا نیاز به طبقه‌بند خاص خود دارد، اما آیا باید برای هر کدام ویژگی‌های متفاوتی انتخاب کنید؟

این چالشی است که تیم‌های مهندسی گوگل با آن روبرو هستند: چگونه می‌توان مجموعه‌ای از ویژگی‌ها را انتخاب کرد که برای همه طبقه‌بندها کارآمد باشد، حتی اگر برای هر کدام به صورت جداگانه کمی دقت کمتری داشته باشد؟

در یک سیستم عملیاتی با تعداد زیادی طبقه‌بند، مطلوب است که یک مجموعه واحد از ویژگی‌ها انتخاب شود به جای مجموعه‌های متفاوت برای هر طبقه‌بند. این رویکرد به دلایل زیر مهم است:

- کاهش پیچیدگی محاسباتی و نیاز به حافظه
- تسهیل نگهداری و به‌روزرسانی سیستم
- امکان استفاده از نمایش سند مشترک برای چندین وظیفه

دو رویکرد اصلی برای انتخاب ویژگی‌های مشترک برای چندین طبقه‌بند وجود دارد:

📊 رویکرد اول: آماره χ^2 برای جدول $n \times 2$

یک روش برای انجام این کار، محاسبه آماره χ^2 برای یک جدول $n \times 2$ است که در آن ستون‌ها وقوع و عدم وقوع یک واژه هستند و هر ردیف مربوط به یکی از کلاس‌ها است. سپس می‌توانیم k واژه با بالاترین آماره χ^2 را انتخاب کنیم.

مثال واقعی: در سیستم طبقه‌بندی اسناد مایکروسافت، این رویکرد به تیم اجازه می‌دهد تا مجموعه‌ای از ویژگی‌ها را برای طبقه‌بندی انواع مختلف اسناد (Word، Excel، PowerPoint) انتخاب کند. این کار باعث می‌شود که نمایش سند مشترکی برای همه انواع اسناد وجود داشته باشد و جستجو و تحلیل آسان‌تر شود.

📊 رویکرد دوم: ترکیب آماره‌های جداگانه

روش متداول‌تر این است که آماره‌های انتخاب ویژگی ابتدا به صورت جداگانه برای هر کلاس در وظایف طبقه‌بندی دو کلاسی (c در مقابل $\bar{c}$) محاسبه شده و سپس ترکیب شوند. دو روش ترکیب متداول وجود دارد:

- **روش میانگین:** یک شاخص واحد برای هر ویژگی محاسبه می‌شود (مثلاً با میانگین‌گیری از مقادیر $A(t, c)$ و k ویژگی با بالاترین شاخص‌ها انتخاب می‌شوند.
- **روش ترکیبی:** k/n ویژگی برتر برای هر یک از n طبقه‌بند انتخاب شده و سپس این n مجموعه در یک مجموعه ویژگی جهانی ترکیب می‌شوند.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان آمازون، تیم تحلیل داده‌ها ابتدا ویژگی‌های برتر را برای هر دسته (کتاب، الکترونیک، لباس) جداگانه محاسبه می‌کند. سپس با استفاده از روش ترکیبی، 20٪ از برترین ویژگی‌های هر دسته را انتخاب کرده و در یک مجموعه جهانی ترکیب می‌کند. این کار باعث می‌شود که سیستم بتواند به سرعت نظرات جدید را دسته‌بندی کند، حتی اگر برای هر دسته کمی دقت کمتری نسبت به استفاده از ویژگی‌های اختصاصی داشته باشد.

📊 معامله بین دقت و کارایی

دقت طبقه‌بندی اغلب کاهش می‌یابد هنگام انتخاب k ویژگی مشترک برای یک سیستم با n طبقه‌بند در مقابل n مجموعه متفاوت به اندازه k . اما حتی اگر دقت کاهش یابد، افزایش کارایی به دلیل نمایش سند مشترک ممکن است ارزش کاهش دقت را داشته باشد.

مثال واقعی: در سیستم فیلتر محتوای فیسبوک، که باید میلیاردها پست را در ثانیه طبقه‌بندی کند، استفاده از یک مجموعه ویژگی مشترک برای همه طبقه‌بندها (خشونت، هرزنامه، اخبار جعلی) ضروری است. حتی اگر این کار باعث

کاهش 1-2 درصدی دقت شود، افزایش سرعت پردازش (تا 10 برابر) این کاهش دقت را جبران می‌کند و به فیسبوک اجازه می‌دهد تا محتوای مضر را به سرعت شناسایی و حذف کند.

In [7]: پیاده‌سازی انتخاب ویژگی برای چندین طبقه‌بند #

```
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
import math
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import f1_score

def calculate_chi2_for_multiple_classes(documents, labels, term):
    """
    محاسبه  $\chi^2$  برای یک واژه در جدول  $\chi^2$  محاسبه آماره

    محاسبه می‌کند که در آن  $n \times 2$  را برای یک واژه در جدول  $\chi^2$  این تابع آماره
    ستون‌ها وقوع و عدم وقوع واژه هستند و هر ردیف مربوط به یکی از کلاس‌هاست.

    پارامترها:
    documents: (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    term: واژه مورد نظر

    برای واژه مورد نظر  $\chi^2$  خروجی: مقدار
    """
    N = len(documents)
    classes = list(set(labels))
    n_classes = len(classes)

    # برای واژه مورد نظر  $n \times 2$  ایجاد جدول
    # [که شامل واژه نیستند i که شامل واژه هستند, تعداد اسناد در کلاس i, تعداد اسناد در کلاس]: هر ردیف
    table = []

    for c in classes:
        N11 = N10 = N01 = N00 = 0

        for doc, label in zip(documents, labels):
            term_in_doc = 1 if term in doc else 0
            is_target_class = 1 if label == c else 0

            if term_in_doc == 1 and is_target_class == 1:
                N11 += 1
            elif term_in_doc == 1 and is_target_class == 0:
                N10 += 1
            elif term_in_doc == 0 and is_target_class == 1:
                N01 += 1
            else: # term_in_doc == 0 and is_target_class == 0
                N00 += 1

        table.append([N11, N01])

    # محاسبه  $\chi^2$  جدول  $n \times 2$ 
    chi2 = 0.0

    # محاسبه مجموع فراوانی‌های مشاهده شده در هر ستون
    col1_sum = sum(row[0] for row in table) # همه کلاس‌ها
    col2_sum = sum(row[1] for row in table) # همه کلاس‌ها

    # محاسبه مجموع فراوانی‌های مشاهده شده در هر ردیف
    row_sums = [sum(row) for row in table] # مجموع اسناد در هر کلاس

    # محاسبه فراوانی‌های مورد انتظار
    for i, row in enumerate(table):
        row_sum = row_sums[i]
        for j, cell in enumerate(row):
            col_sum = col1_sum if j == 0 else col2_sum
            expected = (row_sum * col_sum) / N

            if expected > 0:
                chi2 += ((cell - expected) ** 2) / expected

    return chi2

def calculate_feature_scores_per_class(documents, labels, score_func):
    """
    محاسبه امتیاز ویژگی‌ها برای هر کلاس به صورت جداگانه

    این تابع امتیاز ویژگی‌ها را برای هر کلاس به صورت جداگانه محاسبه می‌کند.
    این رویکرد برای روش ترکیبی مناسب است.

    پارامترها:
    documents: (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    score_func:  $\chi^2$  مانند اطلاعات متقابل یا) تابعی برای محاسبه امتیاز ویژگی

    خروجی: دیکشنری که کلیدها کلاس‌ها و مقادیر دیکشنری امتیاز ویژگی‌ها برای آن کلاس است
    """
    classes = list(set(labels))
    feature_scores = defaultdict(dict)

    # استخراج واژگان
    vocab = set()
    for doc in documents:
```

```

vocab.update(doc)

# محاسبه امتیاز برای هر واژه و هر کلاس
for term in vocab:
    for c in classes:
        score = score_func(documents, labels, term, c)
        feature_scores[c][term] = score

return feature_scores

def select_features_average_method(documents, labels, k, score_func):
    """
    انتخاب ویژگی با روش میانگین

     $A(t,c)$  با میانگین‌گیری از مقادیر) این روش یک شاخص واحد برای هر ویژگی محاسبه می‌کند
    .ویژگی با بالاترین شاخص‌ها را انتخاب می‌کند k و

    پارامترها:
    documents: (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    k: تعداد ویژگی‌هایی که می‌خواهیم انتخاب کنیم
    score_func: (مانند اطلاعات متقابل یا) تابعی برای محاسبه امتیاز ویژگی

    واژه با بالاترین امتیاز میانگین k خروجی: لیستی از
    """
    # محاسبه امتیاز ویژگی‌ها برای هر کلاس
    feature_scores = calculate_feature_scores_per_class(documents, labels, score_func)
    classes = list(set(labels))

    # محاسبه میانگین امتیاز برای هر ویژگی
    avg_scores = {}
    for term in feature_scores[classes[0]].keys(): # همه واژه‌ها در همه کلاس‌ها وجود دارند
        avg_score = sum(feature_scores[c][term] for c in classes) / len(classes)
        avg_scores[term] = avg_score

    # واژه با بالاترین امتیاز میانگین k انتخاب
    selected_features = sorted(avg_scores.items(), key=lambda x: x[1], reverse=True)[:k]
    return [term for term, score in selected_features]

def select_features_union_method(documents, labels, k_per_class, score_func):
    """
    انتخاب ویژگی با روش ترکیبی

    طبقه‌بند انتخاب کرده n ویژگی برتر برای هر یک از k/n این روش
    مجموعه را در یک مجموعه ویژگی جهانی ترکیب می‌کند n و سپس این

    پارامترها:
    documents: (هر سند لیستی از کلمات است)
    labels: لیستی از برچسب‌های مربوط به اسناد
    k_per_class: تعداد ویژگی‌هایی که برای هر کلاس انتخاب می‌شوند
    score_func: (مانند اطلاعات متقابل یا) تابعی برای محاسبه امتیاز ویژگی

    خروجی: لیستی از ویژگی‌های منتخب از ترکیب همه کلاس‌ها
    """
    # محاسبه امتیاز ویژگی‌ها برای هر کلاس
    feature_scores = calculate_feature_scores_per_class(documents, labels, score_func)
    classes = list(set(labels))

    # ویژگی برتر برای هر کلاس k انتخاب
    selected_per_class = {}
    for c in classes:
        class_scores = feature_scores[c]
        top_features = sorted(class_scores.items(), key=lambda x: x[1], reverse=True)[:k_per_class]
        selected_per_class[c] = [term for term, score in top_features]

    # ترکیب مجموعه‌های ویژگی
    combined_features = set()
    for features in selected_per_class.values():
        combined_features.update(features)

    return list(combined_features)

def mutual_information(documents, labels, term, target_class):
    """
    محاسبه اطلاعات متقابل بین یک واژه و یک کلاس
    """
    N = len(documents)

    # شمارش مقادیر N11, N10, N01, N00
    N11 = N10 = N01 = N00 = 0

    for doc, label in zip(documents, labels):
        term_in_doc = 1 if term in doc else 0
        is_target_class = 1 if label == target_class else 0

        if term_in_doc == 1 and is_target_class == 1:
            N11 += 1
        elif term_in_doc == 1 and is_target_class == 0:
            N10 += 1
        elif term_in_doc == 0 and is_target_class == 1:
            N01 += 1
        else: # term_in_doc == 0 and is_target_class == 0
            N00 += 1

    # محاسبه مقادیر جانبی
    N1_ = N10 + N11 # تعداد اسناد شامل واژه
    N0_ = N00 + N01 # تعداد اسناد بدون واژه
    N_1 = N01 + N11 # تعداد اسناد در کلاس هدف
    N_0 = N00 + N10 # تعداد اسناد در کلاس غیرهدف

    # محاسبه اطلاعات متقابل
    mi = 0.0
    if N11 > 0:

```

```

mi += (N11 / N) * math.log2((N * N11) / (N1_ * N_1))
if N01 > 0:
    mi += (N01 / N) * math.log2((N * N01) / (N0_ * N_1))
if N10 > 0:
    mi += (N10 / N) * math.log2((N * N10) / (N1_ * N_0))
if N00 > 0:
    mi += (N00 / N) * math.log2((N * N00) / (N0_ * N_0))

return mi

def chi2(documents, labels, term, target_class):
    """
    برای یک واژه و یک کلاس  $\chi^2$  محاسبه آماره
    """
    N = len(documents)

    # شمارش مقادیر N11, N10, N01, N00
    N11 = N10 = N01 = N00 = 0

    for doc, label in zip(documents, labels):
        term_in_doc = 1 if term in doc else 0
        is_target_class = 1 if label == target_class else 0

        if term_in_doc == 1 and is_target_class == 1:
            N11 += 1
        elif term_in_doc == 1 and is_target_class == 0:
            N10 += 1
        elif term_in_doc == 0 and is_target_class == 1:
            N01 += 1
        else: # term_in_doc == 0 and is_target_class == 0
            N00 += 1

    # با استفاده از فرمول ساده‌شده  $\chi^2$  محاسبه
    numerator = (N11 + N10 + N01 + N00) * ((N11 * N00) - (N10 * N01))**2
    denominator = (N11 + N01) * (N11 + N10) * (N10 + N00) * (N01 + N00)

    if denominator == 0:
        return 0

    chi2_value = numerator / denominator
    return chi2_value

def evaluate_feature_selection_methods(documents, labels, k_values):
    """
    ارزیابی روش‌های مختلف انتخاب ویژگی برای چندین طبقه‌بند
    این تابع دقت طبقه‌بندی را برای روش‌های مختلف انتخاب ویژگی محاسبه می‌کند.
    نتایج نشان می‌دهد چگونه روش‌های مختلف می‌توانند دقت را تحت تأثیر قرار دهند.
    """
    # تقسیم داده‌ها به آموزش و آزمون
    X_train_docs, X_test_docs, y_train, y_test = train_test_split(
        documents, labels, test_size=0.3, random_state=42
    )

    # استخراج واژگان
    vocab = set()
    for doc in X_train_docs:
        vocab.update(doc)

    # ارزیابی برای تعداد مختلفی از ویژگی‌ها
    chi2_n2_scores = []
    mi_avg_scores = []
    mi_union_scores = []

    for k in k_values:
        # روش اول:  $\chi^2$  برای جدول  $n \times 2$ 
        chi2_scores = {}
        for term in vocab:
            chi2_scores[term] = calculate_chi2_for_multiple_classes(X_train_docs, y_train, term)

        # ویژگی برتر k انتخاب
        selected_chi2 = sorted(chi2_scores.items(), key=lambda x: x[1], reverse=True)[:k]
        chi2_features = [term for term, score in selected_chi2]

        # ایجاد نمایش سند برای مدل چندجمله‌ای
        X_train = []
        X_test = []
        for doc in X_train_docs:
            doc_rep = [doc.count(term) for term in chi2_features]
            X_train.append(doc_rep)

        for doc in X_test_docs:
            doc_rep = [doc.count(term) for term in chi2_features]
            X_test.append(doc_rep)

        # آموزش و ارزیابی مدل
        clf = MultinomialNB()
        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)
        f1 = f1_score(y_test, y_pred, average='macro')
        chi2_n2_scores.append(f1)

    # روش دوم: میانگین اطلاعات متقابل
    selected_mi_avg = select_features_average_method(X_train_docs, y_train, k, mutual_information)

    # ایجاد نمایش سند برای مدل چندجمله‌ای
    X_train = []
    X_test = []
    for doc in X_train_docs:
        doc_rep = [doc.count(term) for term in selected_mi_avg]
        X_train.append(doc_rep)

    for doc in X_test_docs:

```



```

doc_rep = [doc.count(term) for term in selected_mi_avg]
X_test.append(doc_rep)

# آموزش و ارزیابی مدل
clf = MultinomialNB()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
f1 = f1_score(y_test, y_pred, average='macro')
mi_avg_scores.append(f1)

# روش سوم: ترکیب اطلاعات متقابل
# بر تعداد کلاس‌ها k تقسیم
k_per_class = max(1, k // len(set(y_train)))
selected_mi_union = select_features_union_method(X_train_docs, y_train, k_per_class, mutual_information)

# ایجاد نمایش سند برای مدل چندجمله‌ای
X_train = []
X_test = []
for doc in X_train_docs:
    doc_rep = [doc.count(term) for term in selected_mi_union]
    X_train.append(doc_rep)

for doc in X_test_docs:
    doc_rep = [doc.count(term) for term in selected_mi_union]
    X_test.append(doc_rep)

# آموزش و ارزیابی مدل
clf = MultinomialNB()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
f1 = f1_score(y_test, y_pred, average='macro')
mi_union_scores.append(f1)

return chi2_n2_scores, mi_avg_scores, mi_union_scores

# ایجاد مجموعه داده نمونه
np.random.seed(42)

# کلاس‌ها
classes = ['poultry', 'coffee', 'sports', 'china', 'uk', 'elections']

# واژگان مرتبط با هر کلاس
class_vocab = {
    'poultry': ['poultry', 'chicken', 'meat', 'avian', 'bird', 'farm', 'egg', 'broiler', 'inspection', 'disease'],
    'coffee': ['coffee', 'bean', 'roast', 'brew', 'colombia', 'brazil', 'export', 'crop', 'harvest', 'arabica'],
    'sports': ['soccer', 'game', 'match', 'team', 'player', 'score', 'win', 'league', 'cup', 'championship'],
    'china': ['china', 'chinese', 'beijing', 'yuan', 'shanghai', 'hong', 'kong', 'xinhua', 'province', 'taiwan'],
    'uk': ['london', 'uk', 'british', 'britain', 'england', 'pound', 'pence', 'stg', 'plc', 'english'],
    'elections': ['election', 'vote', 'candidate', 'campaign', 'poll', 'party', 'democratic', 'republican', 'president', 'ballot']
}

# واژگان عمومی
general_vocab = ['report', 'said', 'year', 'today', 'new', 'government', 'market', 'company', 'people', 'world']

# ایجاد اسناد
documents = []
labels = []
num_docs_per_class = 100

for cls in classes:
    for i in range(num_docs_per_class):
        # هر سند شامل 5-15 واژه است
        doc_length = np.random.randint(5, 16)
        doc = []

        # اضافه کردن 2-4 واژه از کلاس
        num_class_terms = np.random.randint(2, 5)
        class_terms = np.random.choice(class_vocab[cls], num_class_terms, replace=False)
        doc.extend(class_terms)

        # اضافه کردن واژگان عمومی
        num_general_terms = doc_length - num_class_terms
        general_terms = np.random.choice(general_vocab, num_general_terms, replace=True)
        doc.extend(general_terms)

        documents.append(doc)
        labels.append(cls)

# مقایسه روش‌های انتخاب ویژگی برای چندین طبقه‌بند
print("مقایسه روش‌های انتخاب ویژگی برای چندین طبقه‌بند")

# روش اول:  $\chi^2$ 
chi2_scores = {}
for term in set(word for doc in documents for word in doc):
    chi2_scores[term] = calculate_chi2_for_multiple_classes(documents, labels, term)

# انتخاب 10 ویژگی برتر
selected_chi2 = sorted(chi2_scores.items(), key=lambda x: x[1], reverse=True)[:10]
chi2_features = [term for term, score in selected_chi2]

print("\nn10 برای جدول  $\chi^2$  ویژگی برتر با استفاده از روش  $\chi^2$ :")
for i, (term, score) in enumerate(selected_chi2):
    print(f"{i+1}. {term}: {score:.4f}")

# روش دوم: میانگین اطلاعات متقابل
selected_mi_avg = select_features_average_method(documents, labels, 10, mutual_information)

print("\nn10 روش میانگین اطلاعات متقابل")
for i, term in enumerate(selected_mi_avg):
    print(f"{i+1}. {term}")

# روش سوم: ترکیب اطلاعات متقابل
# ویژگی برای هر کلاس 2
k_per_class = 2
selected_mi_union = select_features_union_method(documents, labels, k_per_class, mutual_information)

```

```

print(f"\nویژگی‌های منتخب با استفاده از روش ترکیب اطلاعات متقابل (2 ویژگی برای هر کلاس):")
print(f"تعداد کل ویژگی‌ها: {len(selected_mi_union)}")
print(f"ویژگی‌ها: {selected_mi_union}")

# ارزیابی روش‌های مختلف انتخاب ویژگی
k_values = [10, 20, 50, 100]
chi2_n2_scores, mi_avg_scores, mi_union_scores = evaluate_feature_selection_methods(documents, labels, k_values)

# نمایش نتایج
plt.figure(figsize=(10, 6))
plt.plot(k_values, chi2_n2_scores, 'o-', label='χ² for n×2 table')
plt.plot(k_values, mi_avg_scores, 's-', label='Average MI')
plt.plot(k_values, mi_union_scores, 'd-', label='Union MI')
plt.xlabel('Number of Features')
plt.ylabel('Macro F1 Score')
plt.title('Comparison of Feature Selection Methods for Multiple Classifiers')
plt.legend()
plt.grid(True)
plt.show()

# نمایش نتایج عددی
print("\nنتایج ارزیابی:")
print("MI ترکیب MI میانگین (جدول n×2) χ² تعداد ویژگی")
for i, k in enumerate(k_values):
    print(f"{k}\t\t\t{chi2_n2_scores[i]:.4f}\t\t\t{mi_avg_scores[i]:.4f}\t\t\t{mi_union_scores[i]:.4f}")

```

مقایسه روش‌های انتخاب ویژگی برای چندین طبقه‌بند

n×2 برای جدول χ^2 ویژگی برتر با استفاده از روش

- republican: 284.6715
- beijing: 214.2857
- ballot: 208.5561
- party: 202.8470
- brazil: 197.1581
- bean: 191.4894
- disease: 191.4894
- british: 191.4894
- chicken: 185.8407
- chinese: 180.2120

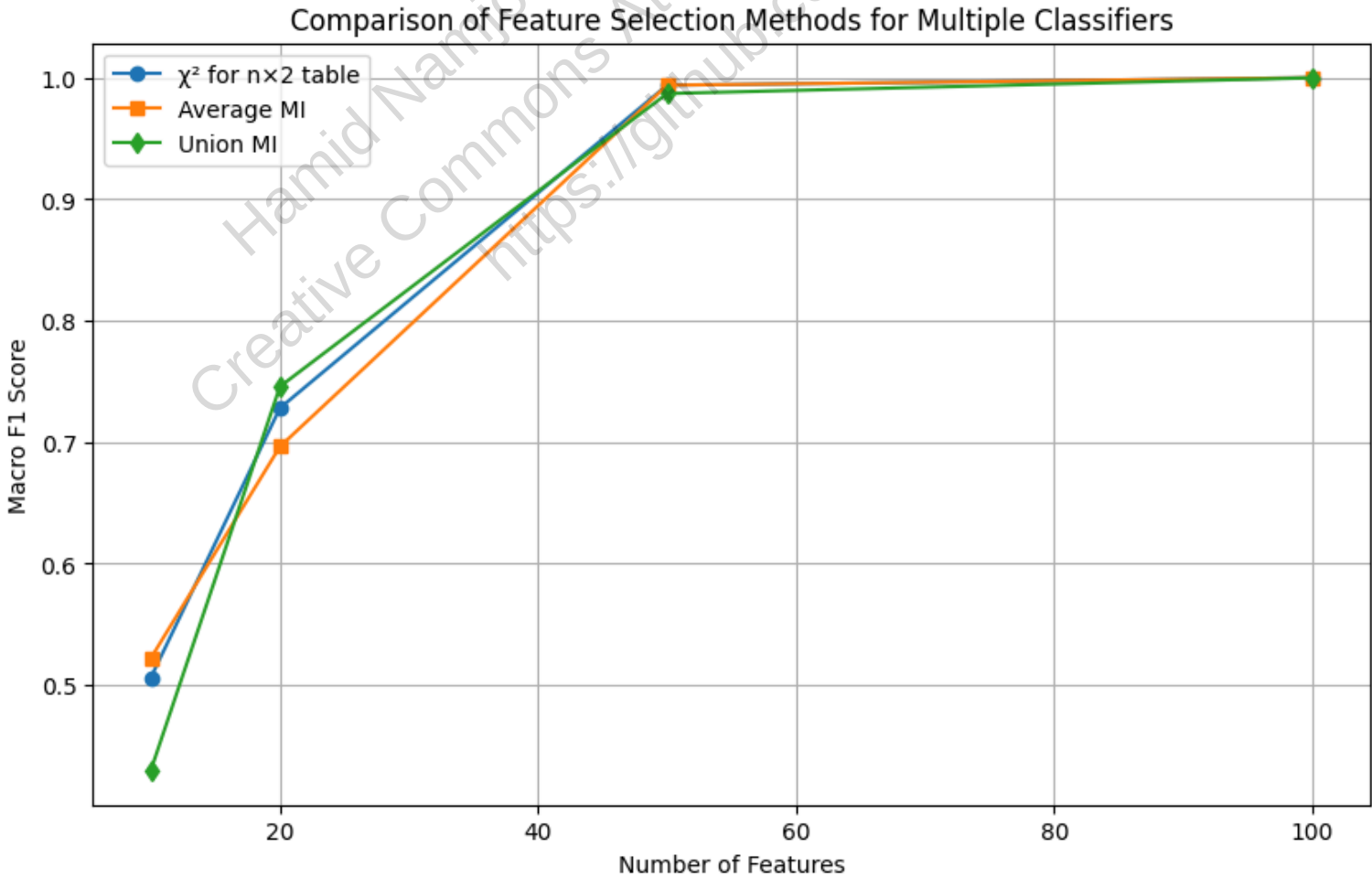
ویژگی برتر با استفاده از روش میانگین اطلاعات متقابل

- republican
- beijing
- ballot
- party
- brazil
- bean
- disease
- british
- chicken
- chinese

ویژگی‌های منتخب با استفاده از روش ترکیب اطلاعات متقابل (2 ویژگی برای هر کلاس):

تعداد کل ویژگی‌ها: 12

ویژگی‌ها: [np.str_('stg'), np.str_('disease'), np.str_('ballot'), np.str_('beijing'), np.str_('cup'), np.str_('british'), np.str_('chicken'), np.str_('bean'), np.str_('brazil'), np.str_('republican'), np.str_('game'), np.str_('chinese')]



نتایج ارزیابی:

تعداد ویژگی	χ^2 (جدول n×2)	MI ترکیب MI میانگین	
10	0.5063	0.5223	0.4300
20	0.7281	0.6964	0.7458
50	0.9939	0.9939	0.9870
100	1.0000	1.0000	1.0000



13.5.5 مقایسه روش‌های انتخاب ویژگی: کدام روش برای کدام مشکل؟

🔍 گوگل نیوز و چالش واژه‌های نادر: چگونه اخبار مهم را از اخبار عادی تشخیص دهیم؟

تیم گوگل نیوز با چالشی جالب روبرو است: واژه‌ای مانند "پاندا" ممکن است فقط یک بار در اخبار مربوط به چین ظاهر شود، اما از نظر آماری بسیار معنادار باشد. آیا این واژه باید به عنوان یک ویژگی مهم برای شناسایی اخبار چین انتخاب شود؟

این دقیقاً همان تفاوت بین اطلاعات متقابل (MI) و χ^2 است: χ^2 به دنبال واژه‌های معنادار از نظر آماری است، حتی اگر به ندرت ظاهر شوند، در حالی که MI به دنبال واژه‌هایی است که اطلاعات بیشتری را به همراه دارند، صرف نظر از تعداد تکرار آن‌ها.

اطلاعات متقابل (MI) و χ^2 روش‌های نسبتاً متفاوتی برای انتخاب ویژگی هستند. تفاوت اصلی آن‌ها در نحوه برخورد با واژه‌های نادر است:

🏷️ معناداری در مقابل اطلاعاتی بودن: تفاوت کلیدی χ^2 و MI

استقلال واژه t و کلاس c گاهی می‌تواند با اطمینان بالایی رد شود حتی اگر t اطلاعات کمی درباره عضویت سند در c داشته باشد. این موضوع به ویژه برای واژه‌های نادر صادق است.

اگر واژه‌ای یک بار در یک مجموعه بزرگ رخ دهد و آن یک وقوع در کلاس poultry باشد، این از نظر آماری معنادار است. اما یک وقوع واحد بر اساس تعریف نظریه اطلاعات، بسیار اطلاعاتی نیست. چون معیار χ^2 معناداری است، این روش واژه‌های نادر بیشتری را نسبت به اطلاعات متقابل انتخاب می‌کند (که اغلب شاخص‌های قابل اعتماد کمتری هستند).

مثال واقعی: در سیستم طبقه‌بندی مقالات علمی آرکایو (ArXiv)، واژه‌های بسیار تخصصی مانند "supersymmetry" ممکن است فقط در چند مقاله ظاهر شوند، اما از نظر χ^2 بسیار معنادار باشند. با این حال، این واژه‌ها ممکن است اطلاعات کمی برای طبقه‌بندی عمومی ارائه دهند، در حالی که MI به واژه‌های رایج‌تری مانند "quantum" یا "experiment" اهمیت بیشتری می‌دهد.

🔗 عملکرد در عمل: آیا تفاوت‌ها واقعاً مهم هستند؟

با وجود تفاوت‌ها، دقت طبقه‌بندی مجموعه‌های ویژگی منتخب با χ^2 و MI تفاوت سیستماتیک نشان نمی‌دهد. در اکثر مسائل طبقه‌بندی متن، چند شاخص قوی و بسیاری شاخص‌های ضعیف وجود دارد. تا زمانی که تمام شاخص‌های قوی و تعداد زیادی از شاخص‌های ضعیف انتخاب شوند، دقت انتظار می‌رود که خوب باشد. هر دو روش این کار را انجام می‌دهند.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان ایبی (eBay)، هر دو روش MI و χ^2 واژه‌هایی مانند "عالی"، "بد"، "ارسال سریع" و "بسته‌بندی" را به عنوان ویژگی‌های برتر شناسایی می‌کنند، حتی اگر اولویت‌بندی آن‌ها کمی متفاوت باشد. در نهایت، هر دو روش به دقت مشابهی در طبقه‌بندی نظرات مثبت و منفی منجر می‌شوند.

📊 شکل 13.8: مقایسه عملکرد MI و χ^2

شکل 13.8 انتخاب ویژگی MI و χ^2 را برای مدل چندجمله‌ای مقایسه می‌کند. اوج کارایی تقریباً برای هر دو روش یکسان است. χ^2 این اوج را دیرتر، در 300 ویژگی، به دست می‌آورد، احتمالاً به این دلیل که ویژگی‌های نادر اما بسیار معناداری را

که در ابتدا انتخاب می‌کند، تمام اسناد در کلاس را پوشش نمی‌دهند. با این حال، ویژگی‌های انتخابی بعدی (در محدوده 100-300) از کیفیت بهتری نسبت به ویژگی‌های انتخابی با MI برخوردارند.

تفسیر عملی: در سیستم طبقه‌بندی اخبار توییت، χ^2 ممکن است ابتدا واژه‌های نادر اما بسیار خاصی مانند "POTUS" (برای اخبار ریاست جمهوری) را انتخاب کند که فقط در چند توئیت ظاهر شده‌اند. این واژه‌ها به تنهایی پوشش کمی دارند، اما با افزایش تعداد ویژگی‌ها، χ^2 به واژه‌های رایج‌تر و کارآمدتر مانند "انتخابات" یا "رأی‌گیری" می‌رسد که پوشش بهتری ایجاد می‌کنند.

محدودیت روش‌های حریصانه: مشکل ویژگی‌های زائد

هر سه روش - χ^2 ، MI، و مبتنی بر فراوانی - روش‌های حریصانه هستند. ممکن است ویژگی‌هایی را انتخاب کنند که اطلاعات افزایشی نسبت به ویژگی‌های قبلی ندارند. در شکل 13.7، kog به عنوان هفتمین واژه انتخاب شده است با وجود اینکه با ویژگی قبلی hong همبستگی بالایی دارد و بنابراین زائد است.

مثال واقعی: در سیستم طبقه‌بندی اخبار مربوط به هنگ کنگ، واژه‌های "hong" و "kong" تقریباً همیشه با هم ظاهر می‌شوند. اگر "hong" به عنوان یک ویژگی مهم انتخاب شده باشد، افزودن "kong" اطلاعات جدیدی به سیستم نمی‌دهد و فقط پیچیدگی را افزایش می‌دهد. این مشکل در شرکت‌هایی مانند بلومبرگ (Bloomberg) که باید اخبار مالی را به سرعت طبقه‌بندی کنند، اهمیت ویژه‌ای دارد.

اگرچه چنین زائدی می‌تواند تأثیر منفی بر دقت داشته باشد، روش‌های غیرحریصانه (به بخش 13.7 مراجعه کنید) به دلیل هزینه محاسباتی به ندرست در طبقه‌بندی متن استفاده می‌شوند. در عمل، سادگی و سرعت روش‌های حریصانه اغلب بر مزایای بالقوه روش‌های پیچیده‌تر ارجحیت دارد.

تمرین 13.5: انتخاب ویژگی برای کلاس قهوه

فراوانی‌های زیر را برای کلاس coffee و چهار واژه در 100,000 سند اول مجموعه Reuters-RCV1 در نظر بگیرید:

واژه	N_{00}	N_{01}	N_{10}	N_{11}
brazil	98,012	102	1835	51
council	96,322	133	3525	20
producers	98,524	119	1118	34
roasted	99,824	143	23	10

دو واژه از این چهار واژه را بر اساس موارد زیر انتخاب کنید:

1. χ^2

2. اطلاعات متقابل

3. فراوانی

راهنمایی عملی: این تمرین به شما کمک می‌کند تا تفاوت‌های عملی بین این سه روش را درک کنید. برای مثال، "brazil" ممکن است بر اساس χ^2 امتیاز بالایی بگیرد به دلیل همبستگی قوی با کلاس قهوه، در حالی که "roasted" ممکن است بر اساس فراوانی امتیاز بالایی بگیرد به دلیل تکرار زیاد در اسناد مربوط به قهوه.

In [8]: # مقایسه روش‌های انتخاب ویژگی بر اساس تمرین 13.5

```
import math
import matplotlib.pyplot as plt
import numpy as np

def calculate_chi2_from_counts(N11, N10, N01, N00):
    """
    Nij بر اساس شمارش‌های  $\chi^2$  محاسبه امتیاز

    را بر اساس فرمول ساده‌شده محاسبه می‌کند  $\chi^2$  این تابع آماره
    .معیاری از انحراف فراوانی مشاهده شده از فراوانی مورد انتظار تحت فرض استقلال است  $\chi^2$ 

    پارامترها:
    N11: تعداد اسناد که شامل واژه و در کلاس هدف هستند
    N10: تعداد اسناد که شامل واژه اما در کلاس هدف نیستند
    N01: تعداد اسناد که در کلاس هدف هستند اما شامل واژه نیستند
    N00: تعداد اسناد که نه شامل واژه هستند و نه در کلاس هدف

     $\chi^2$  خروجی: مقدار
    """
    N = N11 + N10 + N01 + N00
    numerator = N * ((N11 * N00) - (N10 * N01))**2
    denominator = (N11 + N01) * (N11 + N10) * (N10 + N00) * (N01 + N00)

    if denominator == 0:
        return 0 # تغییر یافت 0 به return0 :اصلاح شده

    chi2_value = numerator / denominator
    return chi2_value

def calculate_mi_from_counts(N11, N10, N01, N00):
    """
    Nij بر اساس شمارش‌های (MI) محاسبه امتیاز اطلاعات متقابل

    این تابع اطلاعات متقابل را بر اساس تعریف نظریه اطلاعات محاسبه می‌کند
    .معیاری از اطلاعات مشترک بین دو متغیر تصادفی است MI

    پارامترها:
    N11: تعداد اسناد که شامل واژه و در کلاس هدف هستند
    N10: تعداد اسناد که شامل واژه اما در کلاس هدف نیستند
    N01: تعداد اسناد که در کلاس هدف هستند اما شامل واژه نیستند
    N00: تعداد اسناد که نه شامل واژه هستند و نه در کلاس هدف

    خروجی: مقدار اطلاعات متقابل
    """
    N = N11 + N10 + N01 + N00

    # محاسبه مقادیر جانبی
    N1_ = N10 + N11 # تعداد اسناد شامل واژه
    N0_ = N00 + N01 # تعداد اسناد بدون واژه
    N_1 = N01 + N11 # تعداد اسناد در کلاس هدف
    N_0 = N00 + N10 # تعداد اسناد در کلاس غیرهدف

    # محاسبه اطلاعات متقابل
    mi = 0.0
    if N11 > 0:
        mi += (N11 / N) * math.log2((N * N11) / (N1_ * N_1))
    if N01 > 0:
        mi += (N01 / N) * math.log2((N * N01) / (N0_ * N_1))
    if N10 > 0:
        mi += (N10 / N) * math.log2((N * N10) / (N1_ * N_0))
    if N00 > 0:
        mi += (N00 / N) * math.log2((N * N00) / (N0_ * N_0))

    return mi

def calculate_document_frequency(N11, N10):
    """
    (Document Frequency) محاسبه فراوانی سندی

    این تابع تعداد اسنادی که واژه در آن‌ها وجود دارد را محاسبه می‌کند
    .این معیار برای مدل برنولی مناسب است

    پارامترها:
    N11: تعداد اسناد که شامل واژه و در کلاس هدف هستند
    N10: تعداد اسناد که شامل واژه اما در کلاس هدف نیستند

    خروجی: فراوانی سندی واژه
    """
    return N11 + N10

def compare_feature_selection_methods(term_counts):
    """
    مقایسه روش‌های مختلف انتخاب ویژگی

    :این تابع سه روش انتخاب ویژگی را مقایسه می‌کند:
    1.  $\chi^2$ 
    2. (MI) اطلاعات متقابل
    3. فراوانی سندی

    پارامترها:
    term_counts: برای هر واژه Nij دیکشنری شامل شمارش‌های

    خروجی: دیکشنری شامل امتیاز هر واژه برای هر روش
    """
    results = {}

    for term, counts in term_counts.items():
        N11 = counts['N11']
        N10 = counts['N10']
        N01 = counts['N01']
        N00 = counts['N00']
```



```

# محاسبه امتیازها برای هر روش
chi2_score = calculate_chi2_from_counts(N11, N10, N01, N00)
mi_score = calculate_mi_from_counts(N11, N10, N01, N00)
doc_freq = calculate_document_frequency(N11, N10)

results[term] = {
    'chi2': chi2_score,
    'mi': mi_score,
    'doc_freq': doc_freq,
    'N11': N11,
    'N10': N10,
    'N01': N01,
    'N00': N00
}

return results

def visualize_comparison(results):
    """
    مصورسازی مقایسه روش‌های انتخاب ویژگی

    این تابع نموداری برای مقایسه امتیاز واژه‌ها بر اساس روش‌های مختلف ایجاد می‌کند.
    """
    terms = list(results.keys())
    chi2_scores = [results[term]['chi2'] for term in terms]
    mi_scores = [results[term]['mi'] for term in terms]
    doc_freq_scores = [results[term]['doc_freq'] for term in terms]

    # ایجاد نمودار
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # MI و  $\chi^2$  نمودار مقایسه
    x = np.arange(len(terms))
    width = 0.35

    ax1.bar(x - width/2, chi2_scores, width, label='χ²')
    ax1.bar(x + width/2, mi_scores, width, label='Mutual Information')

    ax1.set_xlabel('Terms')
    ax1.set_ylabel('Score')
    ax1.set_title('Comparison of χ² and Mutual Information')
    ax1.set_xticks(x)
    ax1.set_xticklabels(terms)
    ax1.legend()

    # نمودار مقایسه فراوانی سندی
    ax2.bar(terms, doc_freq_scores, color='green')
    ax2.set_xlabel('Terms')
    ax2.set_ylabel('Document Frequency')
    ax2.set_title('Document Frequency Comparison')

    plt.tight_layout()
    plt.show()

# coffee داده‌های تمرین 13.5 برای کلاس
term_counts = {
    'brazil': {'N00': 98012, 'N01': 102, 'N10': 1835, 'N11': 51},
    'council': {'N00': 96322, 'N01': 133, 'N10': 3525, 'N11': 20},
    'producers': {'N00': 98524, 'N01': 119, 'N10': 1118, 'N11': 34},
    'roasted': {'N00': 99824, 'N01': 143, 'N10': 23, 'N11': 10}
}

# محاسبه و مقایسه امتیازها
results = compare_feature_selection_methods(term_counts)

# نمایش نتایج در جدول
print("="*90)
print("تمرین 13.5 'coffee' مقایسه امتیاز ویژگی‌ها برای کلاس")
print("="*90)
print(f"{'15>': '15<': 'سندی'} | {'15>': '15<': 'سندی'} | {'15>': '15<': 'سندی'} | {'15>': '15<': 'سندی'}")
print("-"*90)

for term, scores in results.items():
    print(f"term:<15 | {scores['doc_freq']:>15} | {scores['chi2']:>15.4f} | {scores['mi']:>15.6f}")

# انتخاب دو واژه برتر برای هر روش
print("\n" + "="*90)
print("دو واژه برتر انتخاب شده برای هر روش")
print("="*90)

# (i) بر اساس  $\chi^2$ 
sorted_chi2 = sorted(results.items(), key=lambda item: item[1]['chi2'], reverse=True)
selected_chi2 = [term for term, scores in sorted_chi2[:2]]
print(f"(i) χ²: {'', '.join(selected_chi2)}")

# (ii) بر اساس اطلاعات متقابل
sorted_mi = sorted(results.items(), key=lambda item: item[1]['mi'], reverse=True)
selected_mi = [term for term, scores in sorted_mi[:2]]
print(f"(ii) اطلاعات متقابل: {'', '.join(selected_mi)}")

# (iii) بر اساس فراوانی
sorted_freq = sorted(results.items(), key=lambda item: item[1]['doc_freq'], reverse=True)
selected_freq = [term for term, scores in sorted_freq[:2]]
print(f"(iii) فراوانی: {'', '.join(selected_freq)}")

# مصورسازی نتایج
visualize_comparison(results)

# تحلیل نتایج
print("\n" + "="*90)
print("تحلیل نتایج")
print("="*90)

```

```
# تحلیل چرا روش‌های مختلف واژه‌های متفاوتی را انتخاب می‌کنند
print("1. واژه‌های متفاوتی را انتخاب می‌کنند؟ MI و  $\chi^2$  چرا")
print("    - به واژه‌هایی که از نظر آماری معنادار هستند اهمیت می‌دهد، حتی اگر نادر باشند  $\chi^2$ ")
print("    - به واژه‌هایی که اطلاعات بیشتری ارائه می‌دهند اهمیت می‌دهد، صرف نظر از معناداری آماری MI")
print("    - دارد coffee دارد چون همبستگی قوی با کلاس  $\chi^2$  امتیاز بالایی در 'brazil'، برای مثال")
print("    - حتی اگر تعداد وقوع آن کم باشد")
print("    - «امتیاز بالایی در فراوانی سندی دارد چون در اسناد زیادی ظاهر می‌شود 'council'")
print("    - ضعیف باشد coffee حتی اگر همبستگی آن با کلاس")

print("\n2. تفاوت‌های عملی در انتخاب ویژگی")
print("    - برای تشخیص واژه‌های نادر اما بسیار خاص مناسب است  $\chi^2$ ")
print("    - برای تشخیص واژه‌هایی که اطلاعات عمومی و کاربردی ارائه می‌دهند مناسب است MI")
print("    - فراوانی سندی ساده‌ترین روش است اما ممکن است واژه‌های عمومی و غیرخاص را انتخاب کند")

print("\n3. کاربردهای عملی")
print("    - می‌تواند واژه‌های خاصی را شناسایی کند که در اسپم‌ها  $\chi^2$ ، در سیستم‌های فیلتر اسپم")
print("    - به ندرت ظاهر می‌شوند اما بسیار مشخص‌کننده هستند")
print("    - می‌تواند واژه‌های رایجی را انتخاب کند که در MI، در سیستم‌های طبقه‌بندی اخبار")
print("    - انواع مختلف اخبار ظاهر می‌شوند و اطلاعات عمومی ارائه می‌دهند")
print("    - در سیستم‌های طبقه‌بندی نظرات مشتریان، فراوانی سندی می‌تواند واژه‌هایی را")
print("    - انتخاب کند که در نظرات زیاد تکرار می‌شوند، حتی اگر لزوماً مشخص‌کننده نباشند")
```

=====

(تمرین 13.5) 'coffee' مقایسه امتیاز ویژگی‌ها برای کلاس

=====

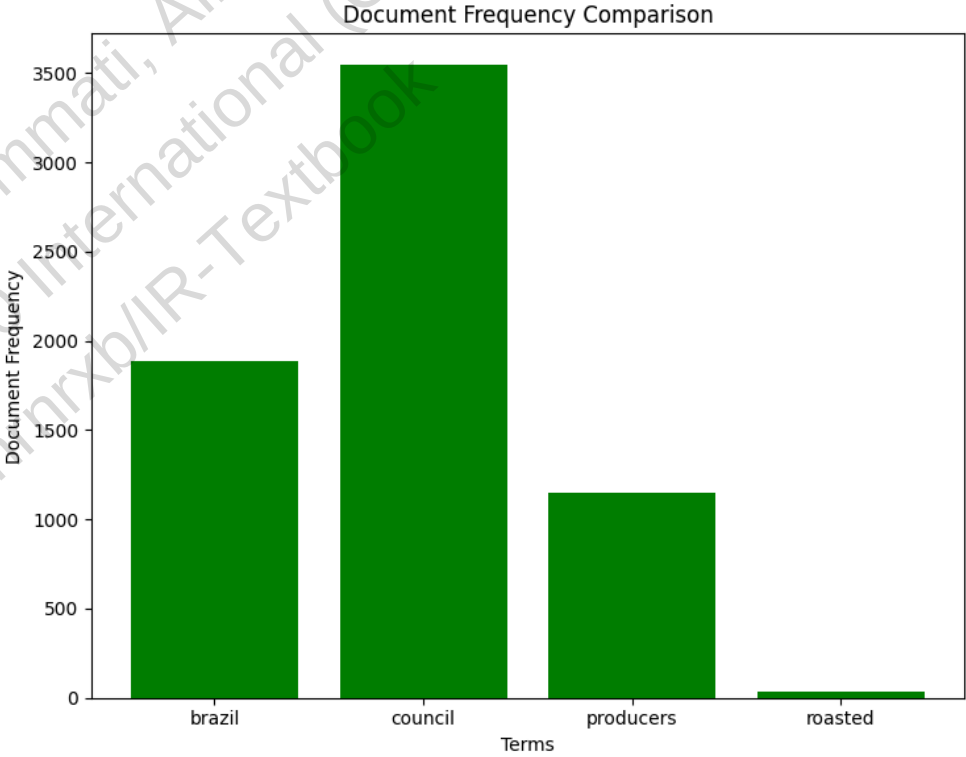
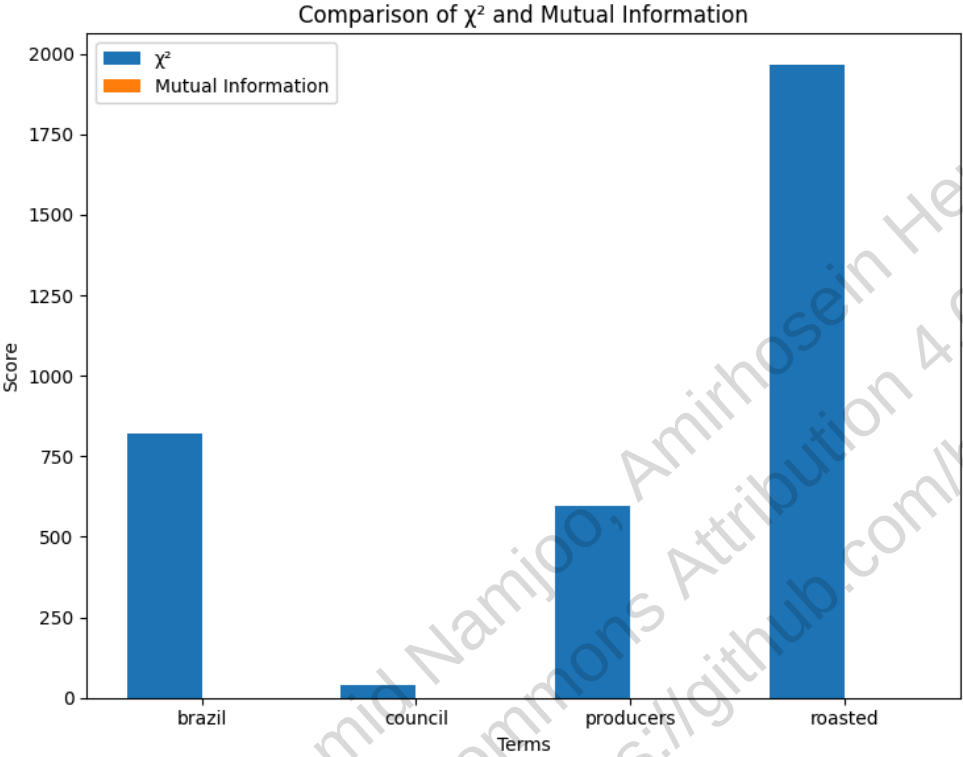
امتیاز	MI	χ^2 واژه	فراوانی سندی	امتیاز
brazil	0.001554	818.9387	1886	
council	0.000177	40.6741	3545	
producers	0.001048	596.0700	1152	
roasted	0.000648	1964.2933	33	

=====

دو واژه برتر انتخاب شده برای هر روش:

=====

(i) χ^2 : roasted, brazil
(ii) اطلاعات متقابل: brazil, producers
(iii) فراوانی: council, brazil



- =====
- تحلیل نتایج:
- =====
- واژه‌های متفاوتی را انتخاب می‌کنند؟ MI و χ^2 چرا
 - به واژه‌هایی که از نظر آماری معنادار هستند اهمیت می‌دهد، حتی اگر نادر باشند χ^2 .
 - به واژه‌هایی که اطلاعات بیشتری ارائه می‌دهند اهمیت می‌دهد، صرف نظر از معناداری آماری MI.
 - درد coffee دارد چون همبستگی قوی با کلاس χ^2 امتیاز بالایی در 'brazil'، برای مثال، حتی اگر تعداد وقوع آن کم باشد.
 - «امتیاز بالایی در فراوانی سندی دارد چون در اسناد زیادی ظاهر می‌شود 'council'، ضعیف باشد coffee حتی اگر همبستگی آن با کلاس.
 - تفاوت‌های عملی در انتخاب ویژگی:
 - برای تشخیص واژه‌های نادر اما بسیار خاص مناسب است χ^2 .
 - برای تشخیص واژه‌هایی که اطلاعات عمومی و کاربردی ارائه می‌دهند مناسب است MI.
 - فراوانی سندی ساده‌ترین روش است اما ممکن است واژه‌های عمومی و غیرخاص را انتخاب کند.
 - کاربردهای عملی:
 - می‌تواند واژه‌های خاصی را شناسایی کند که در اسپم‌ها χ^2 ، در سیستم‌های فیلتر اسپم، به ندرت ظاهر می‌شوند اما بسیار مشخص‌کننده هستند.
 - می‌تواند واژه‌های رایجی را انتخاب کند که در MI، در سیستم‌های طبقه‌بندی اخبار، انواع مختلف اخبار ظاهر می‌شوند و اطلاعات عمومی ارائه می‌دهند.
 - در سیستم‌های طبقه‌بندی نظرات مشتریان، فراوانی سندی می‌تواند واژه‌هایی را، انتخاب کند که در نظرات زیاد تکرار می‌شوند، حتی اگر لزوماً مشخص‌کننده نباشند.

13.6 ارزیابی طبقه‌بندی متن: چگونه بفهمیم الگوریتم ما چقدر خوب کار می‌کند؟



رویترز و چالش طبقه‌بندی اخبار: چگونه سرعت را با دقت ترکیب کنیم؟

تصور کنید در خبرگزاری رویترز کار می‌کنید و باید سیستمی طراحی کنید که هزاران خبر روزانه را به دسته‌های مختلفی (سیاست، ورزشی، اقتصادی، فرهنگی) اختصاص دهد. تیم رقیب شما در خبرگزاری گاردین از الگوریتم پیچیده‌تری استفاده می‌کند، اما شما باید سیستمی بسازید که هم سریع و هم قابل اعتماد باشد.

این دقیقاً همان چالشی است که در ارزیابی طبقه‌بندی متن با آن روبرو هستیم: چگونه می‌توانیم بفهمیم که الگوریتم ما چقدر خوب کار می‌کند، به خصوص وقتی باید بین سرعت و دقت تعادل برقرار کنیم؟

در گذشته، مجموعه **Reuters-21578** به عنوان معیار اصلی برای ارزیابی طبقه‌بندی متن استفاده می‌شد. این مجموعه شامل 21,578 مقاله خبری است که توسط Carnegie Group و Reuters جمع‌آوری و برچسب‌گذاری شد. این مجموعه پایه و اساس بسیاری از تحقیقات اولیه در زمینه طبقه‌بندی متن بود.

چالش طبقه‌بندی چندکلاسی: یک در مقابل همه

در ارزیابی طبقه‌بندی، ما به جای دقت (accuracy)، از معیارهای دقیق‌تری مانند **دقت (precision)**، **بازیابی (recall)** و **F1** استفاده می‌کنیم، به خصوص برای کلاس‌های کوچک که فراوانی کمی دارند.

چرا دقت به تنهایی کافی نیست؟ تصور کنید سیستمی که 99% اسناد را به درستی طبقه‌بندی کند، اما این کار را با گفتن "هیچ سندی در کلاس X نیست" انجام دهد. اگر کلاس X فقط 1% از اسناد را تشکیل دهد، این سیستم 99% دقیق است اما کاملاً بی‌فایده است!

مثال واقعی: در سیستم فیلتر محتوای یوتیوب، اگر سیستمی تمام محتوای مربوط به گروه‌های تروریستی را به درستی شناسایی کند اما در تشخیص محتوای مربوط به گروه‌های کوچک و جدید شکست بخورد، دقت بالای آن گمراه‌کننده است. در چنین مواردی، معیارهای F1 و بازیابی بهتر عملکرد واقعی سیستم را نشان می‌دهند.

🏆 میانگین‌گیری کلان در مقابل ریز: چگونه عملکرد کلی را گزارش کنیم؟

وقتی با چندین طبقه‌بند دو کلاسی کار می‌کنیم (مانند Reuters-21578 با 118 کلاس)، اغلب می‌خواهیم یک معیار تجمعی محاسبه کنیم که نتایج طبقه‌بندهای فردی را ترکیب کند. دو روش اصلی برای این کار وجود دارد:

- **میانگین‌گیری کلان (Macroaveraging):** میانگین ساده روی کلاس‌ها که به هر کلاس وزن برابر می‌دهد.
- **میانگین‌گیری ریز (Microaveraging):** تصمیمات طبقه‌بندی را در تمام کلاس‌ها ادغام کرده و سپس معیار را روی جدول ترکیبی محاسبه می‌کند.

تفاوت بین این دو روش می‌تواند زیاد باشد. میانگین‌گیری کلان به هر کلاس وزن برابر می‌دهد، در حالی که میانگین‌گیری ریز به هر تصمیم طبقه‌بندی وزن برابر می‌دهد. از آنجایی که معیار F1 موارد منفی را نادیده می‌گیرد و عمدتاً توسط تعداد موارد مثبت تعیین می‌شود، کلاس‌های بزرگ در میانگین‌گیری ریز تسلط دارند.

📊 جدول 13.8: تفاوت میانگین‌گیری کلان و ریز در عمل

جدول 13.8 تفاوت بین میانگین‌گیری کلان و ریز را با یک مثال ساده نشان می‌دهد:

► **Table 13.8** Macro- and microaveraging. "Truth" is the true class and "call" the decision of the classifier. In this example, macroaveraged precision is $[10/(10+10) + 90/(10+90)]/2 = (0.5+0.9)/2 = 0.7$. Microaveraged precision is $100/(100+20) \approx 0.83$.

class 1			class 2			pooled table		
	truth: yes	truth: no		truth: yes	truth: no		truth: yes	truth: no
call: yes	10	10	call: yes	90	10	call: yes	100	20
call: no	10	970	call: no	10	890	call: no	20	1860

در این مثال، میانگین‌گیری کلان دقت $= 2/[(90+10)/90 + (10+10)/10] = 2/(0.9 + 0.5) = 0.7$ است، در حالی که میانگین‌گیری ریز دقت $= 100/(20+100) \approx 0.83$ است. نتایج ریز به عملکرد کلاس بزرگ (کلاس 2) نزدیک‌تر است، در حالی که نتایج کلان به هر دو کلاس وزن برابر می‌دهد.

► **Table 13.9** Text classification effectiveness numbers on Reuters-21578 for F_1 (in percent). Results from Li and Yang (2003) (a), Joachims (1998) (b: kNN) and Dumais et al. (1998) (b: NB, Rocchio, trees, SVM).

(a)	NB	Rocchio	kNN	SVM
micro-avg-L (90 classes)	80	85	86	89
macro-avg (90 classes)	47	59	60	60

(b)	NB	Rocchio	kNN	trees	SVM
earn	96	93	97	98	98
acq	88	65	92	90	94
money-fx	57	47	78	66	75
grain	79	68	82	85	95
crude	80	70	86	85	89
trade	64	65	77	73	76
interest	65	63	74	67	78
ship	85	49	79	74	86
wheat	70	69	77	93	92
corn	65	48	78	92	90
micro-avg (top 10)	82	65	82	88	92
micro-avg-D (118 classes)	75	62	n/a	n/a	87

🏆 جدول 13.9: بیز ساده در مقابل رقبا

جدول 13.9 نتایج ارزیابی بیز ساده را در مقایسه با سایر طبقه‌بندها نشان می‌دهد. بیز ساده F_1 میانگین‌گیری ریز 80% دارد که 9% کمتر از SVM (89%) است. با این حال، در کلاس‌های کوچک، عملکرد بیز ساده بسیار ضعیف‌تر است؛ F_1 میانگین‌گیری کلان آن 13% کمتر از SVM است.

تفسیر عملی: این نتایج نشان می‌دهند که اگرچه بیز ساده به اندازه SVM مؤثر نیست، اما سادگی و کارایی آن را به یک گزینه قوی برای مسائل طبقه‌بندی متن تبدیل کرده است. در دنیای واقعی، عواملی دیگری مانند ماهیت داده‌ها، تغییر مفهوم در طول زمان، و خطا در داده‌ها می‌توانند بر عملکرد طبقه‌بندها تأثیر گذارند.

مثال واقعی: در سیستم طبقه‌بندی نظرات مشتریان آمازون، بیز ساده ممکن است عملکرد خوبی در دسته‌بندی نظرات مثبت و منفی داشته باشد، اما در تشخیص زیردسته‌های خاص (مانند "کیفیت بسته‌بندی" در مقابل "کیفیت محصول") ضعیف عمل کند. در چنین مواردی، یک سیستم ترکیبی که از بیز ساده برای طبقه‌بندی کلی و از یک طبقه‌بند تخصصی برای زیردسته‌های خاص استفاده می‌کند، ممکن است نتایج بهتری ارائه دهد.

⚠ قوانین ارزیابی: چگونه نتایج معتبر بگیریم؟

در ارزیابی طبقه‌بندی، جداسازی دقیق بین مجموعه آموزشی، توسعه و آزمون بسیار مهم است. هرگز نباید در حین توسعه سیستم به داده‌های آزمون دسترسی داشت یا روی آن‌ها برنامه اجرا کرد. در عوض، یک مجموعه توسعه برای آزمون در نظر گرفته شده و سیستم روی آن تنظیم می‌شود.

این مجموعه توسعه همچنین **داده‌های نگه‌داشته** (held-out data) نامیده می‌شود. این رویکرد تضمین می‌کند که نتایج گزارش‌شده نماینده عملکرد واقعی سیستم در دنیای واقعی باشند، نه نتیجه تنظیم بیش از حد روی داده‌های آزمون.

مثال واقعی: در مسابقات کاگل (Kaggle) که شرکت‌ها در آن الگوریتم‌های خود را برای مسائل داده‌کاوی آزمایش می‌کنند، این قوانین به شدت رعایت می‌شود. شرکت‌کنندگان فقط به مجموعه آموزشی دسترسی دارند و نتایج الگوریتم‌هایشان فقط روی مجموعه آزمون ارزیابی می‌شود که هرگز در حین توسعه دیده نشده است. این تضمین می‌کند که برنده مسابقه واقعاً بهترین الگوریتم را دارد، نه الگوریتمی که بیش از حد روی داده‌های آزمون تنظیم شده است.

In [11]: پیاده‌سازی معیارهای ارزیابی طبقه‌بندی متن #

```
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict

def calculate_precision(y_true, y_pred, class_label):
    """
    برای یک کلاس خاص (Precision) محاسبه دقت
    دقت نسبت اسنادی که به درستی به کلاس اختصاص داده شده‌اند به کل اسنادی که به آن کلاس اختصاص داده شده‌اند
    پارامترها:
    y_true: لیست برچسب‌های واقعی
    y_pred: لیست برچسب‌های پیش‌بینی شده
    class_label: کلاسی که می‌خواهیم دقت را برای آن محاسبه کنیم
    خروجی: مقدار دقت برای کلاس مورد نظر
    """
    true_positives = sum(1 for true, pred in zip(y_true, y_pred)
                        if true == class_label and pred == class_label)
    predicted_positives = sum(1 for pred in y_pred if pred == class_label)

    if predicted_positives == 0:
        return 0.0

    return true_positives / predicted_positives

def calculate_recall(y_true, y_pred, class_label):
    """
    برای یک کلاس خاص (Recall) محاسبه بازیابی
    بازیابی نسبت اسنادی که به درستی به کلاس اختصاص داده شده‌اند به کل اسنادی که باید به آن کلاس اختصاص داده می‌شدند
    پارامترها:
    y_true: لیست برچسب‌های واقعی
    y_pred: لیست برچسب‌های پیش‌بینی شده
    class_label: کلاسی که می‌خواهیم بازیابی را برای آن محاسبه کنیم
    خروجی: مقدار بازیابی برای کلاس مورد نظر
    """
    true_positives = sum(1 for true, pred in zip(y_true, y_pred)
                        if true == class_label and pred == class_label)
    actual_positives = sum(1 for true in y_true if true == class_label)

    if actual_positives == 0:
        return 0.0

    return true_positives / actual_positives

def calculate_f1(precision, recall):
    """
    F1 محاسبه معیار
    میانگین هارمونیک دقت و بازیابی است و معیاری متعادل برای ارزیابی عملکرد طبقه‌بند F1
    پارامترها:
    precision: مقدار دقت
    recall: مقدار بازیابی
    خروجی: مقدار F1
    """
    if precision + recall == 0:
```



```

    return 0.0
    return 2 * (precision * recall) / (precision + recall)

def calculate_macro_avg(y_true, y_pred, classes):
    """
    برای معیارها (Macro-averaging) محاسبه میانگین کلان

    میانگین کلان به هر کلاس وزن برابر می‌دهد و برای ارزیابی عملکرد روی کلاس‌های کوچک مناسب است.

    پارامترها:
    y_true: لیست برچسب‌های واقعی
    y_pred: لیست برچسب‌های پیش‌بینی شده
    classes: لیست کلاس‌ها

    خروجی: دیکشنری شامل معیارهای میانگین کلان
    """

    precision_sum = 0.0
    recall_sum = 0.0
    f1_sum = 0.0

    for c in classes:
        precision = calculate_precision(y_true, y_pred, c)
        recall = calculate_recall(y_true, y_pred, c)
        f1 = calculate_f1(precision, recall)

        precision_sum += precision
        recall_sum += recall
        f1_sum += f1

    n_classes = len(classes)
    return {
        'precision': precision_sum / n_classes,
        'recall': recall_sum / n_classes,
        'f1': f1_sum / n_classes
    }

def calculate_micro_avg(y_true, y_pred, classes):
    """
    برای معیارها (Micro-averaging) محاسبه میانگین ریز

    میانگین ریز تصمیمات طبقه‌بندی را در تمام کلاس‌ها ادغام کرده و سپس معیار را محاسبه می‌کند.
    این معیار بیشتر تحت تأثیر کلاس‌های بزرگ قرار می‌گیرد.

    پارامترها:
    y_true: لیست برچسب‌های واقعی
    y_pred: لیست برچسب‌های پیش‌بینی شده
    classes: لیست کلاس‌ها

    خروجی: دیکشنری شامل معیارهای میانگین ریز

    ایجاد جدول ترکیبی برای تمام کلاس‌ها
    true_positives = 0
    predicted_positives = 0
    actual_positives = 0

    for c in classes:
        true_positives += sum(1 for true, pred in zip(y_true, y_pred)
                               if true == c and pred == c)
        predicted_positives += sum(1 for pred in y_pred if pred == c)
        actual_positives += sum(1 for true in y_true if true == c)

    # محاسبه معیارهای ریز
    precision = true_positives / predicted_positives if predicted_positives > 0 else 0.0
    recall = true_positives / actual_positives if actual_positives > 0 else 0.0
    f1 = calculate_f1(precision, recall)

    return {
        'precision': precision,
        'recall': recall,
        'f1': f1
    }

def visualize_confusion_matrix(y_true, y_pred, classes):
    """
    مصورسازی ماتریس درهم‌ریختگی برای نمایش عملکرد طبقه‌بند

    این تابع ماتریس درهم‌ریختگی را نمایش می‌دهد که نشان می‌دهد چگونه طبقه‌بند
    اسناد را به کلاس‌های مختلف اختصاص داده است.

    پارامترها:
    y_true: لیست برچسب‌های واقعی
    y_pred: لیست برچسب‌های پیش‌بینی شده
    classes: لیست کلاس‌ها

    خروجی: نمایش ماتریس درهم‌ریختگی
    """

    n_classes = len(classes)
    # ایجاد ماتریس درهم‌ریختگی
    confusion_matrix = np.zeros((n_classes, n_classes), dtype=int)

    # پر کردن ماتریس
    class_to_index = {c: i for i, c in enumerate(classes)}
    for true, pred in zip(y_true, y_pred):
        true_idx = class_to_index[true]
        pred_idx = class_to_index[pred]
        confusion_matrix[true_idx, pred_idx] += 1

    # نمایش ماتریس
    # خط: ax.figure.colorbar(im)
    # صحیح: fig.colorbar(im)
    fig, ax = plt.subplots(figsize=(8, 6))
    im = ax.imshow(confusion_matrix, interpolation='nearest', cmap=plt.cm.Blues)
    fig.colorbar(im)

```

```
# تنظیم محورها
ax.set_xticks(np.arange(n_classes))
ax.set_yticks(np.arange(n_classes))
ax.set_xticklabels(classes, rotation=45)
ax.set_yticklabels(classes)
ax.set_xlabel('Predicted Label')
ax.set_ylabel('True Label')
ax.set_title('Confusion Matrix')

# اضافه کردن اعداد به هر سلول
for i in range(n_classes):
    for j in range(n_classes):
        ax.text(j, i, str(confusion_matrix[i, j]),
                ha="center", va="center", color="red")

plt.tight_layout()
plt.show()

def compare_macro_micro(y_true, y_pred, classes):
    """
    مقایسه میانگین‌گیری کلان و ریز با مثال‌های واقعی

    این تابع تفاوت بین دو روش میانگین‌گیری را با مثال‌های واقعی نشان می‌دهد
    """
    # محاسبه معیارها
    macro_avg = calculate_macro_avg(y_true, y_pred, classes)
    micro_avg = calculate_micro_avg(y_true, y_pred, classes)

    # محاسبه دقت کلی
    accuracy = sum(1 for true, pred in zip(y_true, y_pred) if true == pred) / len(y_true)

    print("مقایسه روش‌های ارزیابی:")
    print("="*50)
    print(f"دقت کلی (Accuracy): {accuracy:.4f}")
    print(f"میانگین کلان دقت: {macro_avg['precision']:.4f}")
    print(f"میانگین ریز دقت: {micro_avg['precision']:.4f}")
    print()
    print(f"میانگین کلان بازیابی: {macro_avg['recall']:.4f}")
    print(f"میانگین ریز بازیابی: {micro_avg['recall']:.4f}")
    print()
    print(f"میانگین کلان F1: {macro_avg['f1']:.4f}")
    print(f"میانگین ریز F1: {micro_avg['f1']:.4f}")

    # نمایش ماتریس درهم‌ریختگی
    visualize_confusion_matrix(y_true, y_pred, classes)

def demonstrate_accuracy_problem():
    """
    برای مجموعه‌های نامتعادل (Accuracy) نمایش مشکل دقت

    این مثال نشان می‌دهد که چرا دقت ممکن است معیار گمراه‌کننده باشد
    """
    print("برای مجموعه‌های نامتعادل (Accuracy) نمایش مشکل دقت")
    print("="*60)

    # A و B 1% در کلاس A مجموعه نامتعادل: 99% اسناد در کلاس
    y_true = ['A'] * 99 + ['B'] # 100 سند با برچسب واقعی
    y_pred_perfect = ['A'] * 99 + ['B'] # طبقه‌بند کامل
    y_pred_naive = ['A'] * 100 # طبقه‌بند ساده که همیشه را پیش‌بینی می‌کند

    # محاسبه دقت برای هر طبقه‌بند
    accuracy_perfect = sum(1 for true, pred in zip(y_true, y_pred_perfect) if true == pred) / len(y_true)
    accuracy_naive = sum(1 for true, pred in zip(y_true, y_pred_naive) if true == pred) / len(y_true)

    print(f"دقت طبقه‌بند کامل: {accuracy_perfect:.2f}")
    print(f"دقت طبقه‌بند ساده (A همیشه): {accuracy_naive:.2f}")
    print()
    print("اِ! را به درستی تشخیص نمی‌دهد، دقت آن 99% است B با وجود اینکه طبقه‌بند ساده هیچ سند")
    print("این مثال نشان می‌دهد که چرا دقت به تنهایی برای مجموعه‌های نامتعادل معیار مناسبی نیست.")
    print()

    # برای هر طبقه‌بند F1 محاسبه
    macro_perfect = calculate_macro_avg(y_true, y_pred_perfect, ['A', 'B'])
    macro_naive = calculate_macro_avg(y_true, y_pred_naive, ['A', 'B'])

    print("تفاوت را به خوبی نشان می‌دهد F1 اما معیار")
    print(f"F1 طبقه‌بند کامل: {macro_perfect['f1']:.4f}")
    print(f"F1 طبقه‌بند ساده: {macro_naive['f1']:.4f}")
    print()
    print("را به درستی تشخیص نمی‌دهد B صفر است، زیرا هیچ سند B طبقه‌بند ساده برای کلاس F1")

# مثال 1: مقایسه طبقه‌بندهای مختلف بر اساس معیارهای ارزیابی
print("مثال 1: مقایسه طبقه‌بندهای مختلف بر اساس معیارهای ارزیابی")
print("="*60)

# داده‌های شبیه‌سازی شده از جدول 13.9 کتاب
# فرض می‌کنیم 1000 سند با توزیع کلاس‌های زیر داریم
# سند earn: 300 کلاس
# سند acq: 200 کلاس
# سند money-fx: 100 کلاس
# سند grain: 150 کلاس
# سند crude: 120 کلاس
# سند corn: 80 کلاس
# سند trade: 50 کلاس

# ایجاد داده‌های واقعی و پیش‌بینی شده برای دو طبقه‌بند مختلف
np.random.seed(42)
classes = ['earn', 'acq', 'money-fx', 'grain', 'crude', 'corn', 'trade']

# ایجاد داده‌های واقعی
y_true = []
for c, count in [('earn', 300), ('acq', 200), ('money-fx', 100),
```

```
('grain', 150), ('crude', 120), ('corn', 80), ('trade', 50)]:
    y_true.extend([c] * count)

# عملکرد بهتر روی کلاس‌های بزرگ - (Naive Bayes) ایجاد پیش‌بینی‌های طبقه‌بند 1
y_pred_nb = []
for c, count in [('earn', 300), ('acq', 200), ('money-fx', 100),
                  ('grain', 150), ('crude', 120), ('corn', 80), ('trade', 50)]:
    if c in ['earn', 'acq']: # عملکرد خوب روی کلاس‌های بزرگ
        y_pred_nb.extend([c] * int(count * 0.9)) # دقت 90%
    else: # عملکرد ضعیف‌تر روی کلاس‌های کوچک
        y_pred_nb.extend([c] * int(count * 0.6)) # دقت 60%

# عملکرد بهتر روی کلاس‌های کوچک - (SVM) ایجاد پیش‌بینی‌های طبقه‌بند 2
y_pred_svm = []
for c, count in [('earn', 300), ('acq', 200), ('money-fx', 100),
                  ('grain', 150), ('crude', 120), ('corn', 80), ('trade', 50)]:
    if c in ['earn', 'acq']: # عملکرد خوب روی کلاس‌های بزرگ
        y_pred_svm.extend([c] * int(count * 0.85)) # دقت 85%
    else: # عملکرد بهتر روی کلاس‌های کوچک
        y_pred_svm.extend([c] * int(count * 0.75)) # دقت 75%

# محاسبه و مقایسه معیارها
print("ارزیابی Naive Bayes:")
macro_nb = calculate_macro_avg(y_true, y_pred_nb, classes)
micro_nb = calculate_micro_avg(y_true, y_pred_nb, classes)
print(f"میانگین کلان F1: {macro_nb['f1']:.4f}")
print(f"میانگین ریز F1: {micro_nb['f1']:.4f}")
print()

print("ارزیابی SVM:")
macro_svm = calculate_macro_avg(y_true, y_pred_svm, classes)
micro_svm = calculate_micro_avg(y_true, y_pred_svm, classes)
print(f"میانگین کلان F1: {macro_svm['f1']:.4f}")
print(f"میانگین ریز F1: {micro_svm['f1']:.4f}")
print()

# مقایسه دو طبقه‌بند
print("مقایسه دو طبقه‌بند:")
print(f"تفاوت در میانگین کلان F1: {macro_svm['f1'] - macro_nb['f1']:.4f}")
print(f"تفاوت در میانگین ریز F1: {micro_svm['f1'] - micro_nb['f1']:.4f}")

# نمایش مشکل دقت برای مجموعه‌های نامتعادل
demonstrate_accuracy_problem()

# مثال 2: مقایسه میانگین‌گیری کلان و ریز
print("\nمثال 2: مقایسه میانگین‌گیری کلان و ریز")
print("=*60)

# ایجاد یک مثال ساده با دو کلاس برای نمایش تفاوت میانگین‌گیری کلان و ریز
# کلاس 1: 100 سند (کلاس بزرگ)
# کلاس 2: 20 سند (کلاس کوچک)
y_true = ['class1'] * 100 + ['class2'] * 20

# داده‌های واقعی
y_true_example = ['class1'] * 100 + ['class2'] * 20

# پیش‌بینی‌ها
# class1 درست برای 90
# class2 درست برای 5
# (پیش‌بینی class1 واقعی، class2 اشتباه 10)
# (پیش‌بینی class2 واقعی، class1 اشتباه 15)
y_pred_example = ['class1'] * 90 + ['class2'] * 5 + ['class1'] * 10 + ['class1'] * 15 # 120 طول

compare_macro_micro(y_true_example, y_pred_example, ['class1', 'class2'])

print("\nنتیجه‌گیری:")
print("=*60)
print("1. میانگین‌گیری کلان به هر کلاس وزن برابر می‌دهد و برای ارزیابی عملکرد روی کلاس‌های کوچک مناسب است.")
print("2. میانگین‌گیری ریز به هر تصمیم طبقه‌بندی وزن برابر می‌دهد و بیشتر تحت تأثیر کلاس‌های بزرگ قرار می‌گیرد.")
print("3. در مجموعه‌های نامتعادل، میانگین‌گیری کلان معیار قابل اعتمادتری برای ارزیابی عملکرد کلی است.")
```

مثال 1: مقایسه طبقه‌بندهای مختلف بر اساس معیارهای ارزیابی
=====

ارزیابی Naive Bayes:
F1 میانگین کلان: 0.2660
F1 میانگین ریز: 0.4914

ارزیابی SVM:
F1 میانگین کلان: 0.2409
F1 میانگین ریز: 0.4358

مقایسه دو طبقه‌بند:
F1 تفاوت در میانگین کلان: -0.0251
F1 تفاوت در میانگین ریز: -0.0556
برای مجموعه‌های نامتعادل (Accuracy) نمایش مشکل دقت
=====
دقت طبقه‌بند کامل: 1.00
0.99: (را پیش‌بینی می‌کند A همیشه) دقت طبقه‌بند ساده

!را به درستی تشخیص نمی‌دهد، دقت آن 99% است B با وجود اینکه طبقه‌بند ساده هیچ سند
این مثال نشان می‌دهد که چرا دقت به تنهایی برای مجموعه‌های نامتعادل معیار مناسبی نیست

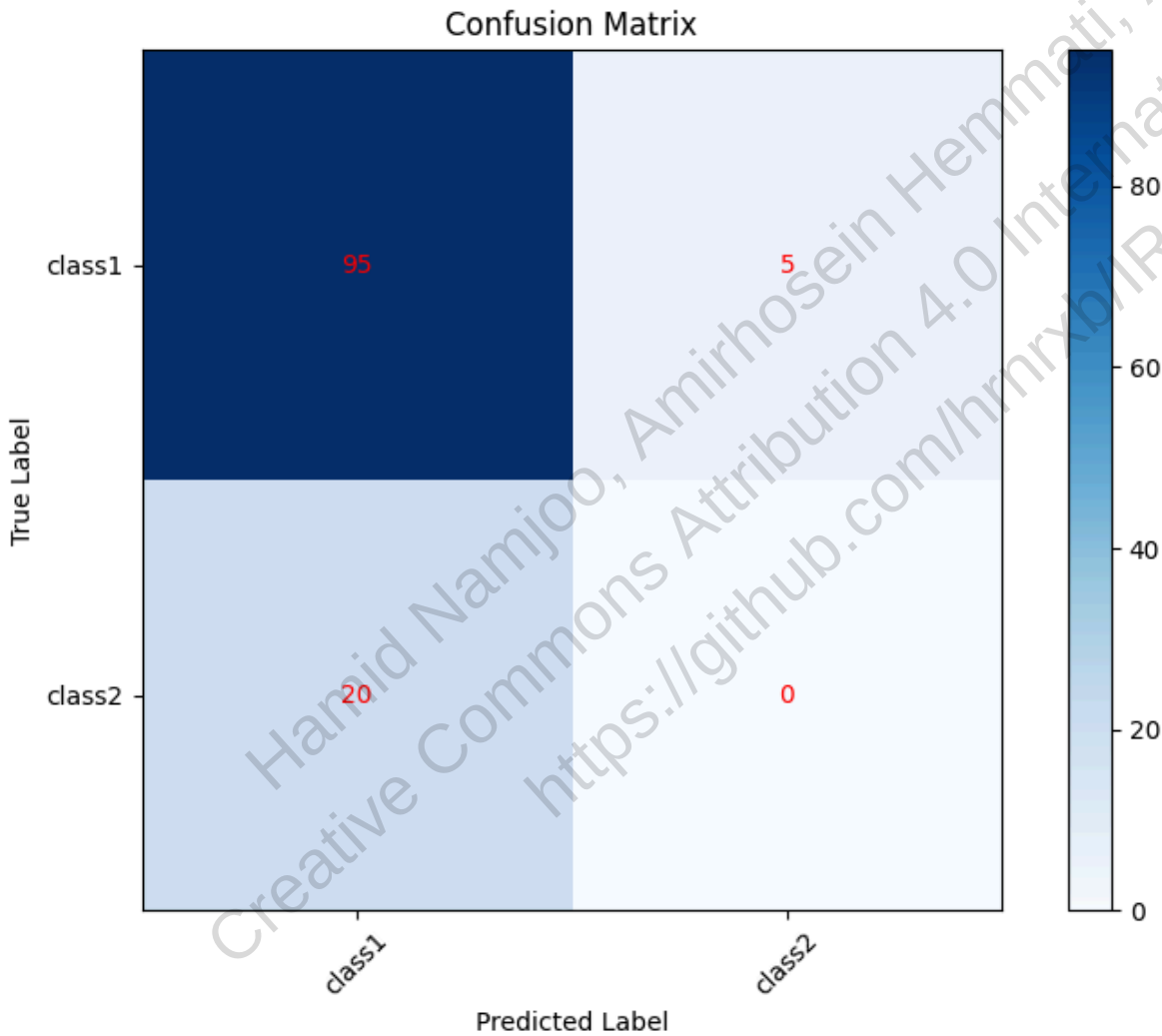
تفاوت را به خوبی نشان می‌دهد F1 اما معیار
طبقه‌بند کامل: F1 1.0000
طبقه‌بند ساده: F1 0.4975

را به درستی تشخیص نمی‌دهد B صفر است، زیرا هیچ سند B طبقه‌بند ساده برای کلاس F1

مثال 2: مقایسه میانگین‌گیری کلان و ریز
=====
مقایسه روش‌های ارزیابی
=====
(Accuracy) دقت کلی: 0.7917
میانگین کلان دقت: 0.4130
میانگین ریز دقت: 0.7917

میانگین کلان بازیابی: 0.4750
میانگین ریز بازیابی: 0.7917

F1 میانگین کلان: 0.4419
F1 میانگین ریز: 0.7917



نتیجه‌گیری:
=====
1. میانگین‌گیری کلان به هر کلاس وزن برابر می‌دهد و برای ارزیابی عملکرد روی کلاس‌های کوچک مناسب است.
2. میانگین‌گیری ریز به هر تصمیم طبقه‌بندی وزن برابر می‌دهد و بیشتر تحت تأثیر کلاس‌های بزرگ قرار می‌گیرد.
3. در مجموعه‌های نامتعادل، میانگین‌گیری کلان معیار قابل اعتمادتری برای ارزیابی عملکرد کلی است.

🧠 جمع‌بندی فصل ۱۳: طبقه‌بندی متن و مدل Naive Bayes

در این فصل:

- مفهوم طبقه‌بندی متن را معرفی کردیم و آن را به‌عنوان تعمیمی از پرسمان‌های ایستا (standing queries) تحلیل کردیم.
- کاربردهای متنوع طبقه‌بندی در IR را بررسی کردیم: از فیلتر کردن اسپم و محتوای حساس تا جست‌وجوی عمودی و تشخیص احساسات.

- مدل یادگیری نظارت‌شده را تعریف کردیم: تابع طبقه‌بندی $\gamma : X \rightarrow C$ با استفاده از مجموعه آموزش برچسب‌خورده.
 - مدل Naive Bayes را به‌عنوان یک روش آماری ساده و مؤثر معرفی کردیم و فرمول MAP را برای انتخاب کلاس بهینه استخراج کردیم.
 - دو نسخه مدل NB را بررسی کردیم: مدل چندجمله‌ای (multinomial) و مدل برنولی (Bernoulli) – با تفاوت در نحوه استفاده از فراوانی واژه و غیاب واژه‌ها.
 - مفهوم smoothing را برای مقابله با داده‌های پراکنده معرفی کردیم و از نرم‌سازی لاپلاس (add-one) استفاده کردیم.
 - ارتباط مدل NB با مدل زبانی چندجمله‌ای را نشان دادیم و پیچیدگی زمانی آموزش و تست را تحلیل کردیم.
- این فصل پایه‌های **یادگیری آماری برای طبقه‌بندی متن** را فراهم می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:
- طبقه‌بندی برداری و ماشین‌های بردار پشتیبان (فصل ۱۴ و ۱۵)
 - بازخورد مرتبط بودن و یادگیری تعاملی (فصل ۱۵)
- آشنا خواهیم شد – که همگی بر همین پایه مدل‌های ساده ولی قدرتمند آماری بنا شده‌اند.